

Notes in CS229-18fall

Sean

stonebreaker365@163.com

Abstract — This note records Sean's notes of Stanford course CS229 taught by Andrew Ng (18fall). (recordings from bilibili)

§ Lec I

1. What is ML ?

- Field of study that gives computers the ability to learn without being explicitly programmed. (Arthur Samuel 1959)
- Well-posed Learning Problem: A computer program is said to learn from experience E with respect to some task T and some performance measure P , if its performance on T , as measured by P , improves with experience E . (Tom Mitchell)
- $AI > ML > DL$

2. Supervised Learning

- Regression Problem:

Given a dataset of (X, Y) (inputs X and label Y), the goal is to learn a mapping from X to Y .
(Regression: the value y that we're trying to predict is continuous)

- Classification Problem:

Similar to above, but the term classification refers to that Y takes on a discrete number of variables.

3. Unsupervised Learning

Only inputs X and no outputs Y are given, asked to figure out interesting structure in the given data.

- Cocktail Party Problem
- ICA : independent component analysis

4. Reinforcement Learning

Widely used in game playing and robotic applications.

§ Lec II

Linear Regression

1. Notations

θ : parameters

m : input numbers (rows in the table)

x : inputs / features

y : output / target variable

(x, y) : training

$(x^{(i)}, y^{(i)})$: i^{th} training example

$$h_{\theta(x)} = \theta_0 + \theta_1 \times x_1 + \theta_2 \times x_2 + \dots + \theta_n \times x_n$$

$$h(x) = \sum_{i=0}^n \theta_i x_i = \theta^T x$$

Goal: select θ to output a hypothesis : function h

To minimize cost function:

$$J(\theta) = \frac{1}{2} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

p.s: linear regression is a special case in the regression family, and squared error corresponds to a Gaussian !

Gradient Descent

Start with $\vec{0}$, keep changing θ to reduce $J(\theta)$.

In fact, when we run gradient descent on linear regression, there will not be local optimum!

• LMS(Least Mean Squares) algorithm

$$\theta_j := \theta_j - \alpha \frac{\partial J(\theta)}{\partial \theta_j}, \quad j = 0, 1, \dots, n$$

α : learning rate

$:=$: This corresponds to an assignment statement in programming, where the left-hand side (lvalue) is the value being modified.

Substitute the previous $J(\theta)$ into the derivative:

$$\begin{aligned} \frac{\partial}{\partial \theta_j} J(\theta) &= \frac{\partial}{\partial \theta_j} \left(\frac{1}{2} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 \right) \\ &= (h_{\theta}(x^{(i)}) - y^{(i)}) \times \frac{\partial}{\partial \theta_j} (h_{\theta}(x) - y) \\ &= (h_{\theta}(x) - y) \cdot \frac{\partial}{\partial \theta_j} \left(\sum_{i=0}^n \theta_i x_i - y \right) \\ &= (h_{\theta}(x) - y) x_j \end{aligned}$$

that is, for single training example, the update to θ_j is:

$$\theta_j := \theta_j + \alpha (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

As the derivative of the sum is the sum of the derivatives, the overall update is (using all the training examples, that's called *batch gradient descent* !, check for all the training examples):

$$\theta_j := \theta_j + \alpha \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

$j = 0, 1, \dots, n$

repeat until convergence.

Usually the learning rate α should be tried out several times on an exponential scale to find out the best value.

Intuition about this update algorithm :

The magnitude of the update is proportional to $(h_{\theta}(x^{(i)}) - y^{(i)})$. Moreover, when the predicted value for a training example is very close to the true value $y^{(i)}$, we find that there is essentially no need to further modify the parameters. Conversely, if our prediction $h_{\theta}(x^{(i)})$

differs greatly from the true value $y^{(i)}$ (for instance, if it is far off), then a larger adjustment to the parameters is required.

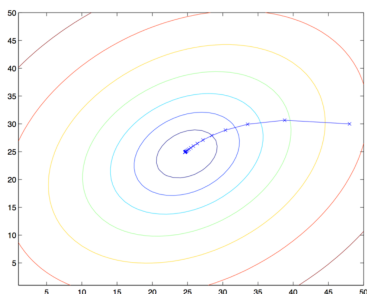


Figure 1: LMS run on the contours of the quadratic function (ellipses)

• Batch Gradient Descent

In each step, we examine *all* the samples in the entire training set.

drawback: if we have a huge dataset, each single update (the sum operation) becomes very slow.

• Stochastic Gradient Descent

```
Repeat:
{
  from  $i = 1$  to  $n$ 
  {
     $\theta_j := \theta_j + \alpha(h_{\theta}(x^{(i)}) - y^{(i)})x_j^{(i)}$ 
    (for every  $j$ )
  }
}
```

In other words, SGD update the whole θ with only *one* current training example !

Typically, stochastic gradient descent finds a value of θ that is sufficiently close to the minimum much faster than batch gradient

descent. (Also note that it may sometimes fail to converge completely to the minimum; in that case, θ will keep oscillating around the minimum of $J(\theta)$. However, in practice, these nearby values are usually close enough to satisfy our precision requirements, so they can still be used.) For these reasons, especially when the training set is large, stochastic gradient descent is often preferred over batch gradient descent.

Most commonly, we already have determined a specific learning rate α (with a description of the dataset beforehand) and then run SGD while gradually decaying the learning rate α toward 0 as the algorithm proceeds. This ensures that the parameters we finally obtain will converge to the minimum, rather than oscillating around it.

• The Normal Equation

notice: this is only applicable for linear regression !

Prerequisites:

一些矩阵导数。

另外要注意等式 (4) 中的 A 必须是非奇异方阵 (non-singular square matrices), 而 $|A|$ 表示矩阵 A 的行列式。那么我们就有下面这些等量关系:

$$\nabla_A \text{tr}(AB) = B^T$$

$$\nabla_{A^T} f(A) = (\nabla_A f(A))^T$$

$$\nabla_A \text{tr}(ABA^T C) = CAB + C^T AB^T$$

$$\nabla_A |A| = |A|(A^{-1})^T$$

Now, given a dataset, we have the *design matrix* x as a $m \times n$ matrix: (实际上, 如果考虑到截距项, 也就是 θ_0 那一项, 就应该是 $m \times (n+1)$ 矩阵), 这个矩阵里面包含了训练样本的输入值作为每一行:

$$(\text{design matrix})X = \begin{pmatrix} -(x^{(1)})^T & - \\ -(x^{(2)})^T & - \\ \vdots & \\ -(x^{(m)})^T & - \end{pmatrix}$$

Then, design y as a m -dimensional vector which contains all the targets in training:

$$y = \begin{pmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(m)} \end{pmatrix}$$

the same for θ :

$$\theta = \begin{pmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_n \end{pmatrix}$$

As $h_\theta(x^i) = (x^i)^T \theta$, we have:

$$X\theta - y = \begin{pmatrix} (x^{(1)})^T \theta \\ \vdots \\ (x^{(m)})^T \theta \end{pmatrix} - \begin{pmatrix} y^{(1)} \\ \vdots \\ y^{(m)} \end{pmatrix} = \begin{pmatrix} h_\theta(x^{(1)}) - y^{(1)} \\ \vdots \\ h_\theta(x^{(m)}) - y^{(m)} \end{pmatrix}$$

$$J(\theta) = \frac{1}{2}(X\theta - y)^T(X\theta - y)$$

Using things in the review part of linear algebra, we could just take the gradient of this expression:

$$\begin{aligned} \nabla_\theta J(\theta) &= \nabla_\theta \left[\frac{1}{2}(X\theta - y)^T(X\theta - y) \right] \\ &= \frac{1}{2} \nabla_\theta [\theta^T X^T X \theta - \theta^T X^T y - y^T X \theta + y^T y] \\ &= \frac{1}{2} \nabla_\theta \text{tr}(\theta^T X^T X \theta - \theta^T X^T y - y^T X \theta + y^T y) \\ &= \frac{1}{2} \nabla_\theta (\text{tr}(\theta^T X^T X \theta) - 2 \text{tr}(y^T X \theta)) \\ &= \frac{1}{2} (X^T X \theta + X^T X \theta - 2X^T y) \\ &= X^T X \theta - X^T y \end{aligned}$$

Let the gradient be 0, and that's the normal function:

$$X^T X \theta = X^T y$$

therefore:

$$\theta = (X^T X)^{-1} X^T y$$

When faced with a regression problem, why might the least-squares cost function J , be a reasonable choice? We'll give a set of probabilistic assumptions, under which least-squares regression is derived as a very natural algorithm.

Assumption:

$$y^{(i)} = \theta^T x^{(i)} + \varepsilon^{(i)}$$

ε^i is an error term that captures either unmodeled effects or random noise.

Let us further assume that the $\varepsilon^{(i)}$ are distributed *IID* (*independently and identically distributed*) according to a Gaussian distribution: $\varepsilon^{(i)} \sim N(0, \sigma^2)$. The density of $\varepsilon^{(i)}$ is given by

$$p(\varepsilon^{(i)}) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(\varepsilon^{(i)})^2}{2\sigma^2}\right)$$

Due to the fact that:

$$\varepsilon^{(i)} = \theta^T x^{(i)} - y^{(i)}$$

This implies that (just substitute the whole $\varepsilon^{(i)}$ into that density function)

$$p(\varepsilon^{(i)} | x^{(i)}; \theta) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right)$$

Additionally, given that:

$$p_Y(y) = p_\varepsilon(\varepsilon) \cdot \left| \frac{dy}{d\varepsilon} \right|$$

我们把 $\varepsilon = y - \theta^T x$ 代入，并对 y 求导计算雅可比行列式的绝对值：

$$\frac{dy}{d\varepsilon} = \frac{dy}{d(y - \theta^T x)} = 1$$

因为导数为 1，直接把第三步的 ε 和 $p(\varepsilon)$ 换成 y 和 $p(y)$ ：

将 $\varepsilon = y - \theta^T x$ 代入第二步的密度函数中：

$$p(y) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y - \theta^T x)^2}{2\sigma^2}\right)$$

换个写法写成条件概率：

$$p(y^{(i)} | x^{(i)}; \theta) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right)$$

这里的记号 $p(y^{(i)} | x^{(i)}; \theta)$ 表示的是这是一个对于给定 $x^{(i)}$ 时 $y^{(i)}$ 的分布，用 θ 代表该分布的参数。注意这里不能用 θ 作为条件（即不能写成 $p(y^{(i)} | x^{(i)}, \theta)$ ），因为 θ 并不是一个随机变量。这个 $y^{(i)}$ 的分布还可以写成

$$y^{(i)} | x^{(i)}; \theta \sim N(\theta^T x^{(i)}, \sigma^2)$$

现在，我们有 m 个样本。因为假设误差 $\varepsilon^{(i)}$ 是 *IID*（独立同分布）的，所以这些样本的联合概率就是每个样本概率的乘积，我们称其为似然函数 *Likelihood*（现在这是关于 θ 的函数！）：

$$L(\theta) = \prod_{i=1}^m p(y^{(i)} | x^{(i)}; \theta)$$

现在，给定了 $y^{(i)}$ 和 $x^{(i)}$ 之间关系的概率模型了，用什么方法来选择咱们对参数 θ 的最佳猜测呢？最大似然法（maximum likelihood）告诉我们要选择能让数据的似然函数尽可能大的 θ 。也就是说，咱们要找的 θ 能够让函数 $L(\theta)$ 取到最大值 (MLE: Maximum Likelihood Estimation)。

取对数可以把连乘变成连加，我们便得到对数似然函数 $\ell(\theta)$ ：

$$\begin{aligned} \ell(\theta) &= \log L(\theta) \\ &= \sum_{i=1}^m \log \left[\frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right) \right] \\ &= \sum_{i=1}^m \left[\log\left(\frac{1}{\sqrt{2\pi}\sigma}\right) - \frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2} \right] \\ &= m \log\left(\frac{1}{\sqrt{2\pi}\sigma}\right) - \frac{1}{2\sigma^2} \sum_{i=1}^m (y^{(i)} - \theta^T x^{(i)})^2 \end{aligned}$$

因此，对 $\ell(\theta)$ 取得最大值也就意味着下面这个子式取到最小值：

$$\frac{1}{2} \sum_{i=1}^m (y^{(i)} - \theta^T x^{(i)})^2$$

到这里我们能发现这个子式实际上就是 $J(\theta)$ ，也就是最原始的最小二乘成本函数 (least-squares cost function)。

• 局部加权线性回归 (Locally weighted linear regression)

在原始版本的线性回归算法中，要对一个查询点 x 进行预测，比如要衡量 $h(x)$ ，要经过下面的步骤：

1. 使用最小二乘法：使用参数 θ 进行拟合，让数据集中的值与拟合算出的值的差值平方最小；

$$\sum_i (y^{(i)} - \theta^T x^{(i)})^2$$

2. 输出 $\theta^T x$

可以看出，最小二乘法中对所有样本一视同仁，也即所有样本对 θ 的影响力度都相同。

在 LWR 局部加权线性回归方法中，步骤如下：

1. 使用参数 θ 进行拟合，让加权距离最小；

$$\sum_i w^{(i)} (y^{(i)} - \theta^T x^{(i)})^2$$

2. 输出 $\theta^T x$ 。

上面式子中的 $w^{(i)}$ 是非负的权值。直观点说就是，如果对应某个 i 的权值 $w^{(i)}$ 特别大，那么在选择拟合参数 θ 的时候，就要尽量让这一点的 $(y^{(i)} - \theta^T x^{(i)})^2$ 最小。而如果权值 $w^{(i)}$ 特别小，那么这一点对应的 $(y^{(i)} - \theta^T x^{(i)})^2$ 就基本在拟合过程中忽略掉了。

从这里我们可以看出，权值越大的样本点的误差会被放大，对于 θ 的影响也会越大

对于权值的选取可以使用下面这个比较标准的公式：

$$w^{(i)} = \exp\left(-\frac{(x^{(i)} - x)^2}{2\tau^2}\right)$$

- ① 当 $x^{(i)}$ 离查询点 x 很近： $(x^{(i)} - x)^2 \rightarrow 0$ ，分子为 0， $\exp(0) = 1$ ，权重最大。

- ② 当 $x^{(i)}$ 离查询点 x 很远： $(x^{(i)} - x)^2 \rightarrow \infty$ ，指数函数趋近于 0。权重几乎为零。

如果 x 是向量，距离就用欧几里得距离平方的泛化形式，那就要对上面的式子进行泛化：

$$w^{(i)} = \exp\left(-\frac{(x^{(i)} - x)^T (x^{(i)} - x)}{2\tau^2}\right)$$

或者我们考虑对不同维度有不同的伸缩效应，于是引出马氏距离 (Mahalanobis Distance)：

$$w^{(i)} = \exp\left(-\frac{(x^{(i)} - x)^T \Sigma^{-1} (x^{(i)} - x)}{2}\right)$$

所以可以看出， θ 的选择过程中，查询点 x 附近的训练样本有更高得多的权值。还要注意，权值 $w^{(i)}$ 的取值方程的形式跟高斯分布的密度函数比较接近的，但权值和高斯分布并没有什么直接联系，它只是借用高斯函数的“钟形”衰减特性来给距离“打分”：随着点 $x^{(i)}$ 到查询点 x 的距离降低，训练样本的权值的也在降低，参数 τ 控制了这个衰减的速度； τ 也叫做带宽参数。

① 无权重的线性回归算法就是一种 [参数] 学习算法：因为有固定的有限个数的参数（也就是 θ_i ），这些参数用来拟合数据。我们对 θ_i 进行了拟合之后，就把它们存了起来，也就不需要再保留训练数据样本来进行更进一步的预测了

② 局部加权线性回归是一个 [非参数] 算法：使用这个算法的时候我们就必须一直保留着整个训练集，因为每来一个新的查询点 x ，权重 $w^{(i)}$ 都会重新计算，导致最优的 θ 也随之改变。“non-parametric”是粗略地指：“模型的复杂度和参数数量，会随着训练集规模 m 的增大而线性增大。”

§ Lec III

Classification

- The framework we've built:
1. Make an assumption about $P(Y|X; \theta)$
 2. Figure out maximum likelihood estimation

Now if we want to apply this framework to other problems, where the value of Y is now either 0 or 1, and that's a classification problem.

It turns out that fit a line (use linear regression) for a classification problem is not a good idea !

1. Logistic Regression

we want $h_\theta(x) \in [0, 1]$

If we choose the function below:

$$h_\theta(x) = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}}$$

$$g(z) = \frac{1}{1 + e^{-z}}$$

This function is called “logistic” or “sigmoid” function.

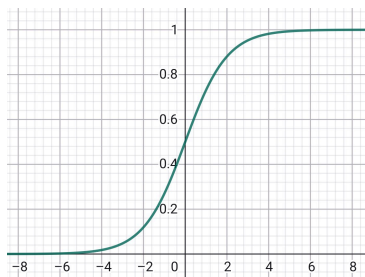


Figure 2: sigmoid / logistic function curve

Intuitively, the function maps real value $(\theta^T x) \in (-\infty, +\infty)$ to $(g(\theta^T x) \in (0, 1))$.

func g 's property:

$$\begin{aligned} g'(z) &= \frac{d}{dz} \left[\frac{1}{1 + e^{-z}} \right] \\ &= \frac{1}{(1 + e^{-z})^2} (e^{-z}) \\ &= \frac{1}{1 + e^{-z}} \cdot \left(1 - \frac{1}{1 + e^{-z}} \right) \\ &= g(z)(1 - g(z)) \end{aligned}$$

Now that we have the logistic regression model, how do we fit an appropriate θ ? We have previously seen that, under a set of assumptions, least squares regression (LSR) can be derived via maximum likelihood estimation (MLE).

So next, let us make a series of statistical assumptions for this classification model, and then use the maximum likelihood method to fit the parameters.

First, define the output probabilities:

$$P(y = 1 \mid x; \theta) = h_\theta(x)$$

$$P(y = 0 \mid x; \theta) = 1 - h_\theta(x)$$

$$y \in \{0, 1\}$$

(note: there we define the direct output prob corresponds to positive example $h_\theta(x)$ just for convention)

A more compact way to write this is (use the fact that y only $\in \{0, 1\}$):

$$p(y \mid x; \theta) = (h_\theta(x))^y (1 - h_\theta(x))^{1-y}$$

Now that the probabilistic model has been established, we need to use *MLE* (Maximum Likelihood Estimation) to find the optimal θ .

Assuming that the m training examples are all generated independently, we can write the likelihood function $L(\theta)$ for the parameters as follows (在已知所有输入特征 X 和参数 θ 的前提下, 观测到这整个标签向量 $\vec{y}$ (即所有 $y^{(i)}$ 到 $y^{(m)}$ 同时出现) 的联合概率):

$$\begin{aligned} L(\theta) &= p(\vec{y}|X; \theta) \\ &= \prod_{i=1}^m p(y^{(i)} | x^{(i)}; \theta) \\ &= \prod_{i=1}^m (h_{\theta}(x^{(i)}))^{y^{(i)}} (1 - h_{\theta}(x^{(i)}))^{1-y^{(i)}} \end{aligned}$$

Similarly, we use log likelihood func $\ell(\theta)$ to figure out the best parameters:

$$\begin{aligned} \ell(\theta) &= \log L(\theta) \\ &= \sum_{i=1}^m [y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)}))] \end{aligned}$$

Now the task is to choose θ to maximize $\ell(\theta)$.

- 怎么让似然函数最大? 就跟之前咱们在线性回归的时候用了求导数的方法类似, 这次是用梯度上升法 (**gradient ascent**) (ascent corresponds to the plus symbol below):

$$\theta := \theta + \alpha \nabla_{\theta} \ell(\theta)$$

(注意更新方程中用的是加号而不是减号, 因为我们现在是在找一个函数的最大值, 而不是找最小值)

还是先从只有一组训练样本 (x, y) 来开始, 然后求导数来推出随机梯度上升规则:

注意: 由于 $g(\theta^T x) = h_{\theta}(x)$ (也就是我们的预估函数)

$$\frac{\partial \ell}{\partial \theta_j} = \left(y \cdot \frac{1}{g} - (1 - y) \cdot \frac{1}{1 - g} \right) \frac{\partial g}{\partial \theta_j}$$

代入性质: $g'(z) = g(z)(1 - g(z))$ 所以:

$$\begin{aligned} \frac{\partial g(\theta^T x)}{\partial \theta_j} &= \frac{\partial g(\theta^T x)}{\partial (\theta^T x)} \cdot \frac{\partial \theta^T x}{\partial \theta_j} \\ &= g(\theta^T x)(1 - g(\theta^T x)) \cdot x_j \end{aligned}$$

代入原式得:

$$\begin{aligned} \frac{\partial \ell}{\partial \theta_j} &= \left(y \cdot \frac{1}{g} - (1 - y) \cdot \frac{1}{1 - g} \right) g(\theta^T x)(1 - g(\theta^T x)) \cdot x_j \\ &= (y(1 - g) - (1 - y)g) \cdot x_j \\ &= (y - g(\theta^T x)) \cdot x_j \end{aligned}$$

$g(\theta^T x)$ 也可以写成 $h_{\theta}(x)$

上面的式子里, 我们用到了之前所提到的 g 的性质: $g'(z) = g(z)(1 - g(z))$. 最后得到了随机梯度上升的参数更新规则:

$$\theta_j := \theta_j + \alpha (y^{(i)} - h_{\theta}(x^{(i)})) x_j^{(i)}$$

(注: 上标的 i 指的是第 i 个样本, 下标 j 则指的是对于向量的第 j 维进行的更新操作)

It turns out that $\ell(\theta)$ is always a concave (凸) function, so we won't get trapped in local op (局部最优). Actually, the only global maximum is also the reason why we choose sigmoid func rather than other funcs that give $(0, 1)$.

For linear regression, the normal equations give a one short way to find the best value of θ . However, there is no known way to have a close form equation for logistic regression. So we only have to use the iterative algorithm like gradient ascent or the Newton method later.

• Newton's method

Gradient descent only allows us to take small steps during update, so we need a lot of iterations to converge. But Newton's method will let us take bigger jumps.

Previously, we've learned Newton's method to find the zeros of a function:

$$\text{want to find } \theta, \quad s.t.: f(\theta) = 0$$

Now we apply it to $\ell'(\theta)$:

$$\text{find } \theta, \quad s.t.: \ell'(\theta) = 0$$

Newton's tangent method iterates as follows:

$$\theta^{(t+1)} := \theta^{(t)} - \frac{f(\theta^{(t)})}{f'(\theta^{(t)})}$$

我们可以把它理解成用一个线性函数来对函数 f 进行逼近，这条直线是 f 的切线，而猜测值是 θ ，解的方法就是找到线性方程等于零的点，把这一个零点作为 θ 设置给下一次猜测，然后以此类推。

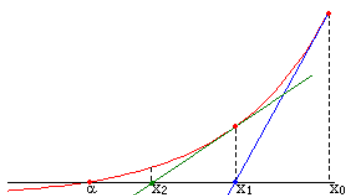


Figure 3: Newton's method to solve the zeros

我们现在要找一阶导数的零点，也就用这样的更新公式：

$$\theta_{\text{new}} := \theta - \frac{\ell'(\theta)}{\ell''(\theta)}$$

In our present case, θ is a vector, so we generalize it (to multi-dimension) to obtain the following update rule (Newton-Raphson method):

$$\theta := \theta - H^{-1} \nabla_{\theta} \ell(\theta)$$

相当于： $\nabla_{\theta} \ell(\theta)$ 就是之前式子里面的一阶导数 $\ell'(\theta)$ ；而 H 就是之前式子里面的二阶导数 $\ell''(\theta)$

$\nabla_{\theta} \ell(\theta)$ 是关于 θ_i 的 $\ell(\theta)$ 的偏导数向量，也就是在之前我们算出关于单个样本的梯度之后累加起来得到所有样本构成的总梯度：

$$\nabla_{\theta} \ell(\theta) = \sum_{i=1}^m (y^{(i)} - h_{\theta}(x^{(i)})) x^{(i)}$$

而 H 是一个 $(n+1) \times (n+1)$ 矩阵（包含截距项）：

$$H_{ij} = \frac{\partial^2 \ell(\theta)}{\partial \theta_i \partial \theta_j}$$

$$H \in \mathbb{R}^{n+1 \times n+1}; \quad \nabla_{\theta} \ell(\theta) \in \mathbb{R}^{n+1}$$

Newton's method has the property of "quadratic convergence", so it converges very fast.

牛顿法通常都能比（批量）梯度下降法收敛得更快，而且达到最小值所需要的迭代次数也低很多。然而，牛顿法中的单次迭代往往要比梯度下降法的单步耗费更多的性能开销(more expensive)，因为要求一个 $n \times n$ 的 Hessian 矩阵的逆(hard in high dimensions)；不过只要这个 n 不是太大，牛顿法通常就还是更快一些。

当用牛顿法来在逻辑回归中求似然函数 $\ell(\theta)$ 的最大值的时候，得到这一结果的方法也叫做 Fisher 评分 (Fisher scoring)。

§ Lec IV

1. Perception Learning Algorithm

Logistic Regression uses the sigmoid function $g(z) = \frac{1}{1+e^{-z}}$ to squeeze the entire real line from $(-\infty, +\infty)$ to $(0, 1)$.

Imagine modifying logistic regression so that it is “forced” to output only 0 or 1. A natural way is to use a threshold function (unit step function):

$$g(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases}$$

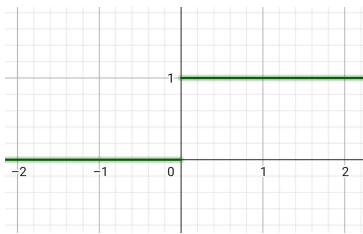


Figure 4: unit step function

Then we still define the hypothesis function $h_\theta(x) = g(\theta^T x)$, but now with g as the threshold function above, and then apply the following update rule (same for both logistic regression and the perceptron):

$$\theta_j := \theta_j + \alpha(y^{(i)} - h_\theta(x^{(i)}))x_j^{(i)}$$

then we obtain the *perceptron learning algorithm*.

Inspect this equation carefully: if the prediction is right, $y^{(i)} - h_\theta(x^{(i)}) = 0$, and there'll be no update. Otherwise $y^{(i)} - h_\theta(x^{(i)}) = \pm 1$.

More close looks at its update algorithm when the prediction is wrong and we end up at $y^{(i)} - h_\theta(x^{(i)}) = \pm 1$:

① If $y^{(i)} = 0$, $h_\theta(x^{(i)}) = 1$, which means we've mistaken a negative example as a positive one. So we take: $\theta_j := \theta_j - \alpha x_j$, to add $-\alpha \vec{x}$ to $\vec{\theta}$, one common way to intending a negative dot product

$\vec{x} \cdot \vec{\theta}$ later. (当 $y=0$, 我们希望 $\vec{\theta}$ 与 $\vec{x}$ 尽量相反) (这里要配一个向量图 1)

② Similarly, when we've mistaken a positive example as negative one, we to add $\alpha \vec{x}$ to $\vec{\theta}$, wishing a positive dot product $\vec{x} \cdot \vec{\theta}$. (当 $y=1$, 我们希望 $\vec{\theta}$ 与 $\vec{x}$ 尽量接近) (这里要配一个向量图 2)

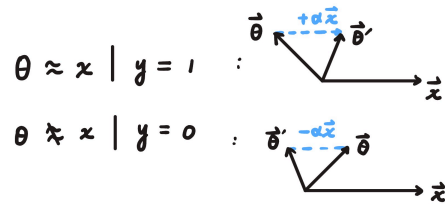


Figure 5: perceptron update rule

(题外话): In the 1960s, this “perceptron” was considered a rough model of how a single neuron in the brain might work. Because of its simplicity, this algorithm serves as a starting point for our later discussion of learning theory in this course. However, it is important to note that although the perceptron learning algorithm may look superficially similar to the other algorithms we have covered, it is actually fundamentally different in kind from logistic regression and least-squares linear regression. In particular, it is very difficult to attach a meaningful probabilistic interpretation to the perceptron’s predictions, nor can the perceptron learning algorithm be derived as a maximum likelihood estimation procedure.

2. The Exponential Family

- 我们先定义一下指数族分布 (exponential family distributions)。如果一个分布能用下面的方式来写出来, 我们就说这类分布属于指数族:

$$p(y; \eta) = b(y) \exp(\eta^T T(y) - a(\eta))$$

上面的式子中:

y —数据(data)

η —此分布的自然参数 (natural parameter, 也叫典范参数 canonical parameter)

$T(y)$ —充分统计量 (sufficient statistic), 通常就是 y 自身

η 与 $T(y)$ 的维度应当匹配 (二者向量点积)

$b(y)$ —basic measure, 是一个标量

$a(\eta)$ —对数分割函数 (log partition function)

$$p(y; \eta) = \frac{b(y)e^{\eta^T T(y)}}{e^{a(\eta)}}$$

$e^{-a(\eta)}$ 这个量本质上扮演了归一化常数 (normalization constant) 的角色, 也就是确保 $p(y; \eta)$ 的总和或者积分等于 1。

当给定 T , a 和 b 时, 就定义了一个用 η 进行参数化的分布族 (family, 或者叫集 set); 通过改变 η , 我们就能得到这个分布族中的不同分布 (也就是说这是一个单参数的分布, 给定 η , 我们就能拿到这个分布的全部概率密度公式等等, 进而之后的更新公式也能被推导出来)。

- Bernoulli Distribution** is in this family

Recall that Bernoulli distribution's PDF is:

$$p(y; \varphi) = \varphi^y (1 - \varphi)^{1-y}$$

Then we rewrite it into the form above:

$$\begin{aligned} p(y; \varphi) &= \exp(y \log \varphi + (1 - y) \log(1 - \varphi)) \\ &= \exp\left(\left(\log\left(\frac{\varphi}{1 - \varphi}\right)\right)y + \log(1 - \varphi)\right) \end{aligned}$$

compare it to this uni-form:

$$p(y; \eta) = \frac{b(y)e^{\eta^T T(y)}}{e^{a(\eta)}}$$

The corresponding parameters for Bernoulli's distribution are:

$$b(y) = 1$$

$$\eta = \log\left(\frac{\varphi}{1 - \varphi}\right)$$

$$T(y) = y$$

$$a(\eta) = -\log(1 - \varphi) = \log(1 + e^\eta)$$

- Gaussian Distribution** is also in this family

assume $\sigma^2 = 1$

$$\begin{aligned} p(y; \mu) &= \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{(y - \mu)^2}{2}\right) \\ &= \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{1}{2}y^2\right) \cdot \exp\left(\mu y - \frac{1}{2}\mu^2\right) \end{aligned}$$

Then we have:

$$b(y) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{y^2}{2}\right)$$

$$\eta = \mu$$

$$T(y) = y$$

$$a(\eta) = \frac{\eta^2}{2} \left(= \frac{\mu^2}{2}\right)$$

- Properties with the exponential family**

- MLE with respect to η is a concave function $\Leftrightarrow$ NLL(negative log likelihood) is convex

$$2. \quad E[y; \eta] = \frac{\partial(a(\eta))}{\partial \eta}$$

$$3. \quad \text{Var}[y; \eta] = \frac{\partial^2(a(\eta))}{\partial \eta^2}$$

通常我们求解某种分布中的均值与方差的时候需要进行积分，但是此时我们只需要进行求导，更易操作

事实上，针对不同类型的数据，我们可以采用 exponential family 中的不同分布类型进行建模：

- ① Real number (实值) — Gaussian
- ② Binary (二分类数据) — Bernoulli
- ③ count (1, 2, 3...整数) — Poisson
- ④ R^+ (正实数) — Gamma, Exponential
- ⑤ 概率分布之上的概率分布 — Beta, Dirichlet (通常出现在贝叶斯机器学习、统计中)

• GLMs

进行泛化，设想一个分类或者回归问题，要预测一些随机变量 y 的值，作为 x 的一个函数。要导出适用于这个问题的广义线性模型，就要对我们的模型、给定 x 下 y 的条件分布来做出以下三个假设：

1. 假设 1:

$$y \mid x; \theta \sim \text{ExponentialFamily}(\eta)$$

即给定 x 和 θ ， y 的分布属于指数分布族，是一个参数为 η 的指数分布。

2. 假设 2:

$$\eta = \theta^T x, \quad \theta \in \mathbb{R}^n, x \in \mathbb{R}^n$$

自然参数 η 和输入值 x 是线性相关的， $\eta = \theta^T x$ ，且如果 η 为有值的向量，则 $\eta_i = \theta_i^T x$

3. 假设 3:

给定 x ，目的是要预测对应这个给定 x 的 $T(y)(=y)$ 的期望值，这就意味着我们的学习假设 h 输出的预测值 $h(x)$ 要满足

$$h(x) = E[y \mid x]$$

例如在逻辑回归中，

$$\begin{aligned} h_{\theta}(x) &= [p(y = 1 \mid x; \theta)] \\ &= [0 \cdot p(y = 0 \mid x; \theta) + 1 \cdot p(y = 1 \mid x; \theta)] \\ &= E[y \mid x; \theta] \end{aligned}$$

注：这里的 $E[y \mid x]$ 应该就是对给定 x 时的 y 值的期望的意思。

上面的几个假设中，第二个可能看上去证明得最差，所以也更适合把这第二个假设看作是一个我们在设计广义线性模型时候的一种“设计选择” (design choice)，而不是一个假设。自然界可能并不真的遵循 $\eta = \theta^T x$ ，但我们把它当成一个设计准则。因为线性是最简单的起点，而且如果这条线不够好，我们可以在 x 上加非线性特征 (比如 $\log x, x^2$ 来强制让它变好)

那么这三个假设/设计，就可以用来推导出一个非常合适的学习算法类别，即广义线性模型 (GLMs)，这个模型有很多特别友好又理想的性质，比如很容易学习。此外，这类模型对一些关于 y 的分布的不同类型建模来说通常效率都很高；例如，我们下面就将要简单介绍一些逻辑回归以及普通最小二乘法这两者如何作为广义线性模型来推出。

整体思想是这样的，一共分为两部分：模型 + 分布。首先我们拿到输入 x ，我们假设模型是一个线性模型，于是这个模型通过可学习的参数 θ 输出 $\theta^T x$ 作为参数 η (即自然参数)， η 被传递给 Exponential Family 作为其核心参数，(我们会在分布这部分选择合适的分布，分布类型的选择取决于我们最后的任务，比如说预

测实值就选 Gaussian，预测值 $\in \{0, 1\}$ 就选 Bernoulli.....进而再选择合适的 $b(y), a(\eta), T(y)$ 最后我们在对应的分布类型上得到最终的预测值 $h(x)$ ，这个预测值被定义为在给定 x 下的条件期望 $E[y | x]$ 。

(上述 mental map 要配一个手绘示意图)

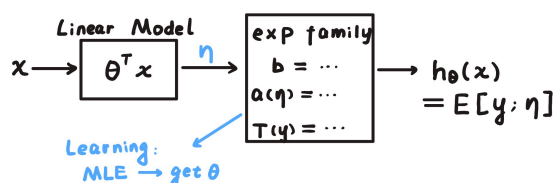


Figure 6: GLM mental map

而这个期望值，恰好等于指数族分布的对数分割函数 $a(\eta)$ 的一阶导数：

$$h(x) = E[y | x] = \frac{\partial a(\eta)}{\partial \eta}$$

这个数学性质（指数族的均值-参数恒等式）是连接线性部分 η 和最终预测值之间的桥梁。

GLM 的流水线就是：输入 x 线性投影得到 η ， η 驱动指数族分布，最后输出的是该分布的期望值。而这个期望值是由 η 经过“正则响应函数”（即 $a(\eta)$ 的导数）映射得到的。

$$\eta = \theta^T x, \quad h(x) = E[y | x] = \frac{\partial a(\eta)}{\partial \eta}$$

在学习的时候，我们做的是 maximum likelihood：

$$\max_{\theta} \log P(y^{(i)}; \theta^T x^{(i)})$$

在训练的时候我们怎样训练得到模型？我们通过梯度下降得到的参数，就

是线性模型里面的参数 θ ，而不是分布里面的 μ, σ^2, η

• Learning update rule

It turns out that no matter what kind of GLM we're doing, or what kind of distribution we choose. The rule is always the same as below:

$$\theta_j := \theta_j + \alpha(y^{(i)} - h_{\theta}(x^{(i)}))x_j^{(i)}$$

(注意：以上这个总是成立的统一更新公式的推导是因为我们使用的 MLE 进而推导出来的)

• Terminology

$\eta \rightarrow$ natural parameter

$\mu = E[y; \eta] = g(\eta) = \frac{\partial}{\partial \eta} a(\eta) \rightarrow g() : \text{canonical response function}$

$\eta = g^{-1}(\mu) \rightarrow g^{-1}() : \text{canonical link function}$

• 3 Parameterizations

①: model parameter : θ

②: natural parameter : η

③: canonical parameter :

φ – Bernoulli

μ, σ^2 – Gaussian

λ – Poisson

Whenever we learn a GLM, we learn θ (that is in the linear model).

$\theta^T x = \eta$ (it's the design choice !)

$g(\eta) = \text{canonical parameter};$

and $g^{-1}(\text{canonical param}) = \eta$

Now recall the logistic regression:

$$h_{\theta}(x) = E[y|x; \theta] = \varphi$$

(we choose the Bernoulli distribution, so the canonical param here is φ)

Also, there is:

$$\varphi = \frac{1}{1 + e^{-\eta}} = \frac{1}{1 + e^{-\theta^T x}}$$

By now, we know that the logistic function is a natural choice when doing binary classification !

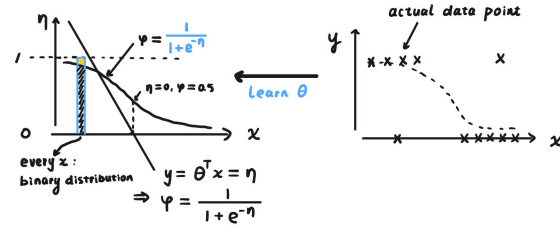


Figure 8: GLM logistic regression(Bernoulli)

Also, recall linear regression: we have input x , and get

$$\theta^T x = \eta$$

Additionally, we use the Guassian as the distribution in this case, so

$$\eta = \mu$$

In our Guassian assumption, we assume that for every x , the corresponding y is in a Guassian distribution of variance 1 (注意：方差不为 1 时方差的大小可以被学习在 θ 中，所以简化起见我们就令方差为 1) and mean of $\theta^T x$ (配一个手绘图)

所以我们相当于是认定事先存在上述手绘图中的这种数据分布规律，然后我们实际拿到的数据是这个分布规律之上产生的，现在我们实际做的就是从一个从右向左的倒推过程，最后要找到合适的 θ 。(再配一个手绘图)

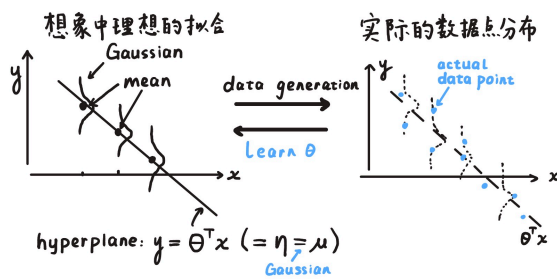


Figure 7: GLM linear regression(Gaussian)

同理对于逻辑回归任务，我们同样也是要进行这样的一个倒推 (再配一张手绘图)

• Softmax regression

Softmax 回归可以被理解为 GLM 家族中的一个例子，但是这里我们将采取非 GLM 的方法进行推证，即采用交叉熵(Cross Entropy)的思想。

Consider a *multi-class* (k -class) classification:

$x^{(i)} \in \mathbb{R}^n$, label $y \in [\{0, 1\}^k]$ (y is a one-hot vector)

Every class has its own set of parameters:

$\theta_{\text{class}} \in \mathbb{R}^n$, and there are k such θ_{class} , $\text{class} \in \{0, \dots, 1, \dots, 0\}$ (one-hot vector)

The whole parameters form a matrix:

$$\begin{matrix} \overbrace{\begin{pmatrix} \theta_1^T \\ \theta_2^T \\ \vdots \\ \theta_k^T \end{pmatrix}}^{n \text{ columns}} \\ \underbrace{\hspace{1.5cm}}_{k \text{ rows}} \end{matrix}$$

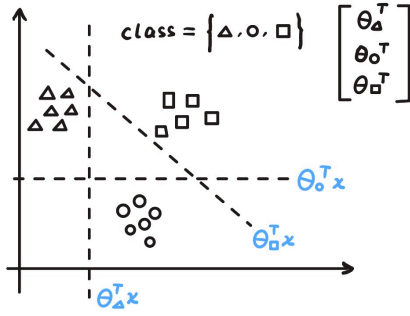


Figure 9: Multi-class classification

Given a x , $\theta^T x \in (-\infty, +\infty)$. Our goal is to get a probability distribution over the classes, so we first take the exponential of each $\theta_i^T x : e^{\theta_i^T x}$ and the value will be positive. Then, normalize the values:

$$\frac{e^{\theta_i^T x}}{\sum_{i=1}^k e^{\theta_i^T x}}$$

So given a x , and we run this whole procedure, we get a *probability output over all the classes* for which class that example is most likely to belong to.

The true y is prob 1 over the true class, and prob 0 over other classes.

Our goal is to minimize the distance between the two prob distributions. The term for that is : “minimize the cross entropy between the two distributions”.

$$\text{Cross Entropy}(p, \hat{p}) = - \sum_{y \in \text{classes}} p(y) \log \hat{p}(y)$$

For this example,

$$\begin{aligned} &= -\log \hat{p}(y_0) \\ &= -\log \frac{e^{\theta_i^T x}}{\sum_{c \in \text{classes}} e^{\theta_c^T x}} \end{aligned}$$

And then we do gradient descent with respect to the parameters.

§ Lec V

Generative Learning Algorithm

By now, the learning algorithms we've learned are called discriminative learning algorithms. (也就是说目前为止，我们讲过的学习算法的模型都是 $p(y | x; \theta)$ ，即给定 x 下 y 的条件分布，以 θ 为参数)

设想一个二分类问题，我们要学习基于一个动物的某个特征来辨别它是大象 ($y = 1$) 还是小狗 ($y = 0$)。给定一个训练集，用逻辑回归或者基础版的感知器算法 (perceptron algorithm) 这样的算法能找到一条直线，作为区分开大象和小狗的边界。接下来，要辨别一个新的动物是大象还是小狗，程序就要检查这个新动物的值落到了划分出来的哪个区域中，然后根据所落到的区域来给出预测。(以上这样的 discriminative algorithm 本质就是在做 MLE)

So rather than looking at both classes simultaneously and searching for a way to separate them, the generative algorithm builds a model of what each of the classes looks like.

At test time, it evaluates a new example against those two models and tries to see which of the two models matches more closely against.

• Discriminative :

Learn $p(y|x)$, i.e. $h_{\theta}(x) = \begin{Bmatrix} 0 \\ 1 \end{Bmatrix}$ directly. (the mapping : $x \rightarrow y$)

• Generative :

Learn $p(x|y)$ (given the class y , what the feature x will be like) &&(and) $p(y)$ (a class prior)

Using the Bayes rule:

$$p(y = 1|x) = \frac{p(x|y = 1)p(y = 1)}{p(x)}$$

$$p(x) = p(x|y = 1)p(y = 1) + p(x|y = 0)p(y = 0)$$

Combining the above two equations gives:

$$p(y = 1|x) = \frac{p(x|y = 1)p(y = 1)}{p(x|y = 1)p(y = 1) + p(x|y = 0)p(y = 0)}$$

1. Gaussian Discriminant Analysis (GDA)

Suppose $x \in \mathbb{R}^n$ (continuous value), and we drop the convention that $x_0 = 1$. (Note that in linear model: $\theta^T x = \theta_0 + \theta_1 x_1 + \dots + \theta_n x_n$, so we design that $x_0 = 1$)

Key Assumption:

$p(x|y)$ is distributed Gaussian

(given the label class, the feature is distributed Gaussian)

In this case, x is a high-dimensional vector, so x should follow a multivariate Gaussian distribution.

$$x \sim \mathcal{N}(\mu, \Sigma)$$

μ is the mean vector ($\in \mathbb{R}^n$)

Σ is the covariance matrix ($\in \mathbb{R}^{n \times n}$)

These two parameters are the requirements for multivariate Gaussian, they control the mean value and the variance.

多元高斯分布的概率密度公式(PDF) :

$$p(x; \mu, \Sigma) = \frac{1}{(2\pi)^{\frac{n}{2}} |\Sigma|^{\frac{1}{2}}} \exp\left(-\frac{1}{2}(x - \mu)^T \Sigma^{-1}(x - \mu)\right)$$

Some pictures for intuition:

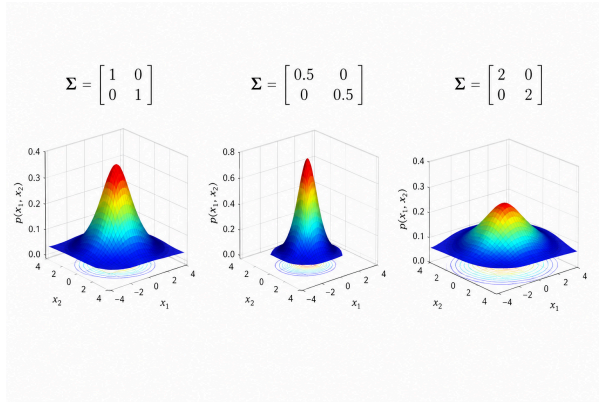


Figure 10: Multivariate Gaussian bumps1

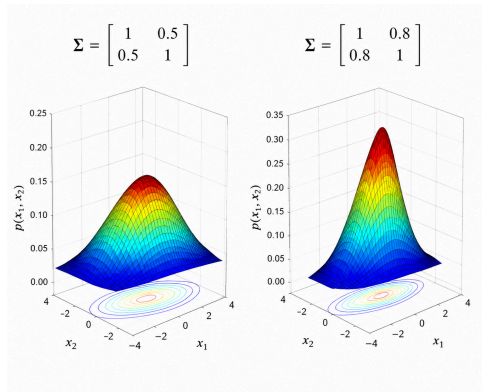


Figure 11: Multivariate Gaussian bumps2

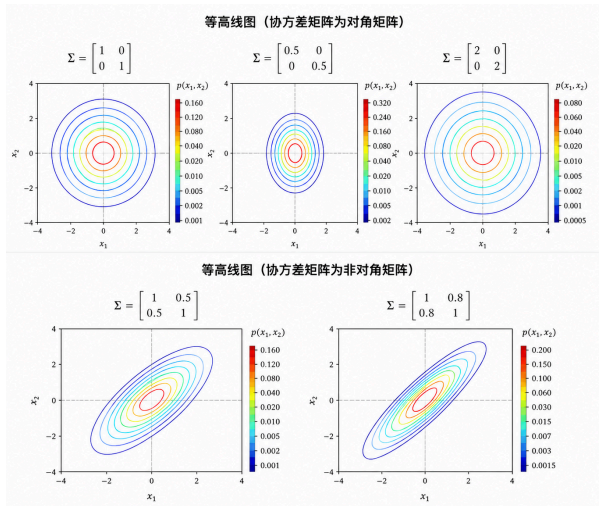


Figure 12: Multivariate Gaussian contours

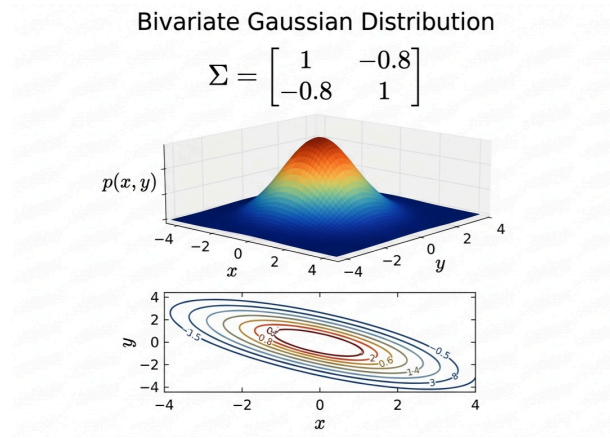


Figure 13: Multivariate Gaussian bumps3

• **GDA model:**

$$p(x|y=0) = \frac{1}{(2\pi)^{\frac{n}{2}} |\Sigma|^{\frac{1}{2}}} \exp\left(-\frac{1}{2}(x - \mu_0)^T \Sigma^{-1}(x - \mu_0)\right)$$

$$p(x|y=1) = \frac{1}{(2\pi)^{\frac{n}{2}} |\Sigma|^{\frac{1}{2}}} \exp\left(-\frac{1}{2}(x - \mu_1)^T \Sigma^{-1}(x - \mu_1)\right)$$

Also, we have to model y (a Bernoulli random variable):

$$p(y) = \varphi^y (1 - \varphi)^{1-y} \quad i.e. (p(y=1) = \varphi)$$

Parameters: $\mu_0, \mu_1 (\in \mathbb{R}^n), \Sigma (\in \mathbb{R}^{n \times n}), \varphi (\in \mathbb{R}(0, 1))$

(usually we use the same Σ rather than Σ_0, Σ_1 , but the means are different)

So if we can fit the 4 parameters above, we can define $p(x|y)$ and $p(y)$, then we can use the Bayes rule to calculate $p(y=1|x), p(y=0|x)$, to predict the label(at test time) :

$$p(y=1|x) = \frac{p(x|y=1)p(y=1)}{p(x|y=1)p(y=1) + p(x|y=0)p(y=0)}$$

- Now we discuss **how to fit the parameters above at training time:**

training set: $\{x^{(i)}, y^{(i)}\}_{i=1}^m$

In order to fit the parameters, we want to maximize the joint likelihood.

Joint likelihood :

$$\begin{aligned}\mathcal{L}(\varphi, \mu_0, \mu_1, \Sigma) &= \prod_{i=1}^m p(x^{(i)}, y^{(i)}; \varphi, \mu_0, \mu_1, \Sigma) \\ &= \prod_{i=1}^m p(x^{(i)}|y^{(i)})p(y^{(i)})\end{aligned}$$

(we want to maximize $p(x^{(i)}, y^{(i)})$)

Generative learning algorithm is to maximize the joint likelihood, whereas for a *discriminative* learning algorithm, we're maximizing the **conditional likelihood**:

$$\mathcal{L}(\theta) = \prod_{i=1}^m p(y^{(i)}|x^{(i)}, \theta)$$

(we're choosing θ to maximize $p(y^{(i)}|x^{(i)})$)

Maximize joint likelihood estimation:

$$\begin{aligned}\max_{\varphi, \mu_0, \mu_1, \Sigma} \log \mathcal{L}(\varphi, \mu_0, \mu_1, \Sigma) \\ = \max_{\varphi, \mu_0, \mu_1, \Sigma} \ell(\varphi, \mu_0, \mu_1, \Sigma)\end{aligned}$$

(proof is)

The MLE solutions are:

$$\varphi = \frac{\sum_{i=1}^m 1\{y^{(i)} = 1\}}{m} \text{ (prob of } y \text{ of label 1),}$$

$$\mu_0 = \frac{\sum_{i=1}^m 1\{y^{(i)} = 0\}x^{(i)}}{\sum_{i=1}^m 1\{y^{(i)} = 0\}} \text{ (所有 } y=0 \text{ 样本的特征均值),}$$

$$\mu_1 = \frac{\sum_{i=1}^m 1\{y^{(i)} = 1\}x^{(i)}}{\sum_{i=1}^m 1\{y^{(i)} = 1\}} \text{ (所有 } y=1 \text{ 样本的特征均值),}$$

$$\Sigma = \frac{1}{m} \sum_{i=1}^m (x^{(i)} - \mu_{y^{(i)}})(x^{(i)} - \mu_{y^{(i)}})^T$$

(可以配一张图直观表示两个分类的均值等取法)

note that:

indicator function (指示函数, 后面会频繁用到)

$$1\{\text{true}\} = 1,$$

$$1\{\text{false}\} = 0.$$

- **Prediction Rule** (predict the most likely class label, 比较后验概率值大小):

$$\arg \max_y p(y|x) = \arg \max_y \frac{p(x|y)p(y)}{p(x)}$$

(return value of arg max is the corresponding value we need to plug in to achieve that biggest possible value, and in this case y is 0 or 1)

Note that the denominator $p(x)$ is just a constant irrelative to y , so the above expression is equal to:

$$\arg \max_y p(x|y)p(y)$$

- **GDA and logistic regression comparison**

- **► logistic regression**

This discriminative learning algorithm is to fit a θ , and to output $\theta^T x$ as the decision boundary. (like a line on the 2D plane)

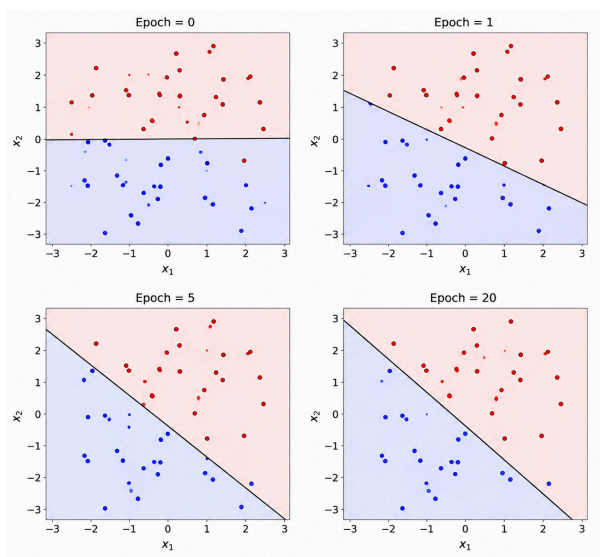


Figure 14: logistic regression iteration

• ▶ GDA

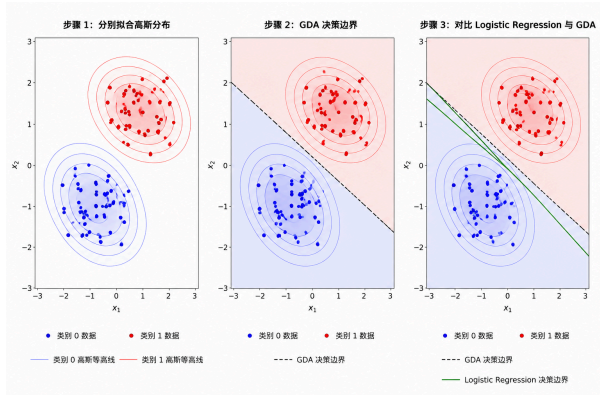


Figure 15: GDA approach

① 分别为正负例拟合高斯分布：使用相同的协方差矩阵，定位两个高斯分布位置，确定参数：

$$\varphi, \mu_0, \mu_1, \Sigma$$

② 我们可以得到决策边界的直线，由于直线处判别为正负例的概率相同：

$$p(y = 1|x) = p(y = 0|x)$$

利用贝叶斯公式：

$$p(x|y = 1) \cdot p(y = 1) = p(x|y = 0) \cdot p(y = 0)$$

利用高斯分布 PDF（概率密度公式），我们把之前在拟合步骤中算好的 μ_0, μ_1, Σ ，分别塞进高斯分布的公式里：

左侧（类别 1）：

$$\frac{1}{(2\pi)^{\frac{n}{2}} |\Sigma|^{\frac{1}{2}}} \exp\left(-\frac{1}{2}(x - \mu_1)^T \Sigma^{-1}(x - \mu_1)\right) \cdot \varphi$$

右侧（类别 0）：

$$\frac{1}{(2\pi)^{\frac{n}{2}} |\Sigma|^{\frac{1}{2}}} \exp\left(-\frac{1}{2}(x - \mu_0)^T \Sigma^{-1}(x - \mu_0)\right) \cdot (1 - \varphi)$$

最后得到决策边界直线的表达式：

$$\theta^T x + \theta_0 = 0$$

①

$$\theta = \Sigma^{-1}(\mu_1 - \mu_0)$$

表明：决策边界直线一定垂直于 μ_0 与 μ_1 的连线方向（在 Σ 定义的度量空间下）。

②

$$\theta_0 = -\frac{1}{2}\mu_1^T \Sigma^{-1} \mu_1 + \frac{1}{2}\mu_0^T \Sigma^{-1} \mu_0 + \log\left(\frac{\varphi}{1 - \varphi}\right)$$

决定直线在空间中的具体位置（偏移量）。

从示例图中可以看出，同一个训练集，使用 GDA 与使用 logistic regression 方法最后得到的决策边界直线不相同

Note:

Why we choose the same covariance matrix Σ ?

It turns out that the decision boundary is usually linear, so the same covariance matrix Σ

leads to linear boundary, and separate Σ_0, Σ_1 will lead to nonlinear boundary.(it;s actually unreasonable)

• Compare GDA and logistic regression

For a fixed set of parameters: $\varphi, \mu_0, \mu_1, \Sigma$, let's plot the predicted prob:

$$p(y=1|x) = \frac{p(x|y=1)p(y=1)}{p(x)}$$

(note:

the red part is parametrized by $\varphi, \mu_0, \mu_1, \Sigma$;

the green part is parametrized by μ_1, Σ (label 1 Gaussian distribution PDF);

the blue part is parametrized only by φ (Bernoulli distribution));

the orange part is parametrized by $\varphi, \mu_0, \mu_1, \Sigma$ (it can be expanded using Baye's rule).

So for every given x , we can compute this ratio and thus get a number for the chance of Y being 1 given X .

Now we can watch this $p(y=1|x)$ function more carefully through an simple example. (下面配上手绘图 (分步骤多步))

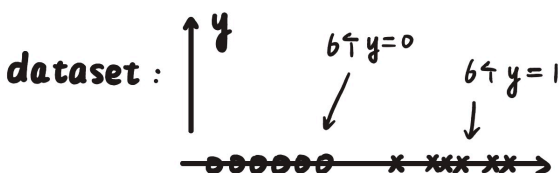


Figure 16: GDA simple example figure 1

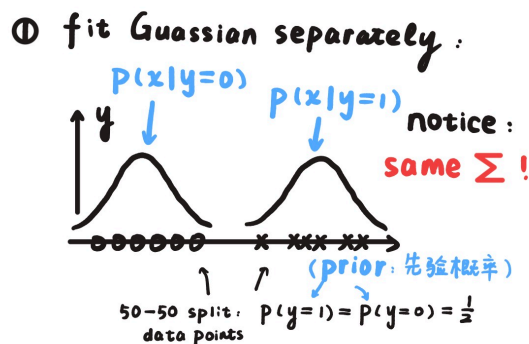


Figure 17: GDA simple example figure 2

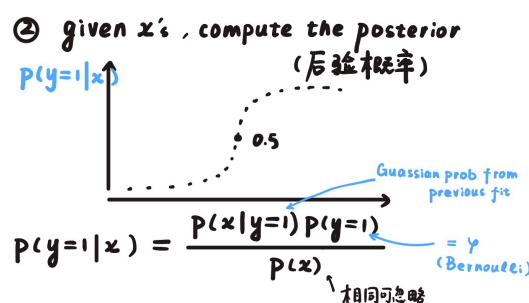


Figure 18: GDA simple example figure 3

可以看出, 如果在训练集中 $\varphi = 0.5$, 则 $p(y=1|x)$ 就是一个标准的 Sigmoid 函数, 事实上 $p(y=0|x)$ 也会是一个标准的 Sigmoid 函数 (作业中会进行严格证明)

所以既然 GDA 与 logistic regression 实质上都是使用的 sigmoid function 来计算最后的预测比率 $p(y=1|x)$, 但是由于参数选择的原因, 我们从之前的 figure 中看出两种算法最后得到的决策边界并不相同。

那么两种算法分别在什么情况下更优?

• In-depth Comparison

(generative)

GDA assumes that:

$$x|y = 0 \sim \mathcal{N}(\mu_0, \Sigma)$$

$$x|y = 1 \sim \mathcal{N}(\mu_1, \Sigma)$$

$$y \sim \text{Bernoulli}(\varphi)$$

(discriminative)

Logistic regression assumes that:

$$p(y = 1|x) = \frac{1}{1 + \theta^T x},$$

with details like $x_0 = 1$

In other words, it assumes that $p(y = 1|x)$ is a logistic function.

根据我们刚才的示意图（之后作业中会进行证明）可以看出，从 GDA 的假设条件出发，我们事实上可以推出 $p(y = 1|x)$ 是一个逻辑函数，但是反过来，从数学上我们可以知道 logistic function 条件并不能推出 GDA 的假设条件，也就是说这是一种充分不必要的关系，GDA 的假设条件更强(stronger set of assumptions)！

（这个地方配一个简单的双箭头互指示意图，但是反推不成立）

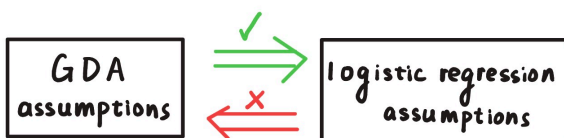


Figure 19: GDA-logistic comparison

在模型中，当我们告诉模型更多的正确信息之后，模型通常会表现更好。因此，如果 GDA 的假设正确，因为它的条件更强，所以这种情

况下使用 GDA 效果更好；但如果假设错误会导致模型表现很差。

• 题外话：

对于任何广义线性模型中的指数家族分布而言，如果加上类似于上述 GDA 假设，都会推出 $p(y = 1|x)$ 是逻辑函数的条件，例如：

$$x|y = 0 \sim \text{Poisson}(\mu_0, \Sigma)$$

$$x|y = 1 \sim \text{Poisson}(\mu_1, \Sigma)$$

$$y \sim \text{Bernoulli}(\varphi)$$

也能得到上述类似的结论。

如果数据集很小，做出更多假设的模型实际上能让模型表现得更好，so it's more computationally efficient 而且更加准确；逻辑回归建立的假设更弱，因此对于偏离的模型假设来说更加鲁棒 (robust)。

然而，如果训练集数据的确是高斯分布的 (non-Gaussian)，而且是有限的大规模数据 (in the limit of large datasets)，那么逻辑回归几乎总是比 GDA 要更好的。因此，在实际中，逻辑回归的使用频率要比 GDA 高得多。

• Naive Bayes

(The E-mail Classification Problem)

① Represent an e-mail as a feature vector x :

Given an e-mail, we'll take this piece of text and represent it as a feature vector. One way is to create an bi-value vector: occur-1, no-0.

$x \in \{0, 1\}^n$ n-dim binary vector, with n: vocab_size

$$x_i = 1 \{\text{word } i \text{ appear in the email}\}$$

(注: 1{}的用法见前文小节:指示函数 (这个地方配一个跳转页面按钮?))

对于 Naive Bayes 算法 (属于生成式算法), 我们的目的同样是要建模 $p(x|y), p(y)$, 但由于 x 是一个 n 维二元向量, 如果直接用多项分布建模 $p(x)$, 那么需要处理 2^n 种情况 (相当于词之间两两搭配得到的所有条件概率), 这会导致参数的个数过多。

所以我们这个地方还要引入额外的假设以解决参数个数过多的问题:

• ▶ Naive Bayes Assumption:

x_i 's are conditionally independent given y

By the chain rule of probability,

$$p(x_1, \dots, x_n | y) = p(x_1|y)p(x_2|x_1, y) \dots p(x_n|x_1, \dots, x_{n-1}, y)$$

So what naive Bayes actually assumes is that the expression above can be simplified as:

$$\begin{aligned} & \stackrel{\text{assume}}{=} p(x_1|y)p(x_2|y) \dots p(x_n|y) \\ & = \prod_{i=1}^n p(x_i|y) \end{aligned}$$

which means: if we've known y 's label, x_i 's existence won't affect x_j 's existence.

However, this is just not a mathematically true assumption, but it has practical sense.

(???correlation with probability graph model???)
是一个有向概率图模型, 其中 y 是父节点, 所有 x_i 是它的子节点 (星形结构) (配一个手绘图)

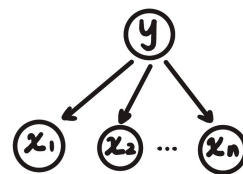


Figure 20: Naive Bayes prob graph

当我们引入上面的条件概率的独立性假设之后, 我们只需要存储 $p(x_i|y)$ 这些 n 个独立的概率, 参数个数从指数级减少至线性级。

• ▶ Parameters of the model

$$\varphi_{j|y=1} = p(x_j = 1|y = 1)$$

$$\varphi_{j|y=0} = p(x_j = 1|y = 0)$$

$$\varphi_y = p(y = 1)$$

由于 x 是一个 n 维二元向量, 所以事实上这里就相当于 $p(x_i|y) \sim \text{Bernoulli}$.

由于 Naive Bayes 与 GDA 同属于生成模型, 我们来看看 Naive Bayes 在参数设置地方与 GDA 之间的关联与区别:

GDA:

$p(x|y)$ is distributed Gaussian

$p(y) = \varphi \quad (y \sim \text{Bernoulli}(\varphi))$

可以看出 Naive Bayes 与 GDA 都是设 y 服从一个伯努利分布, 但不同处在于 GDA 设条件概率服从高斯分布, Naive Bayes 则设条件概率互相全部独立; 作出条件概率这样的假设之后, 求解 MLE 时能简便得解

• ▶ How to fit the parameters

Joint likelihood:(similar to GDA)

$$\begin{aligned}\mathcal{L}(\varphi_y, \varphi_{j|y}) &= \prod_{i=1}^m p(x^{(i)}, y^{(i)}; \varphi_y, \varphi_{j|y}) \\ &= \prod_{i=1}^m p(x^{(i)}|y^{(i)})p(y^{(i)})\end{aligned}$$

(注: m : 训练样本数量 (邮件数量))

MLE conclusion:

$$\begin{aligned}\varphi_y &= \frac{\sum_{i=1}^m 1\{y^{(i)} = 1\}}{m} \\ \varphi_{j|y=1} &= \frac{\sum_{i=1}^m 1\{x_j^{(i)} = 1, y^{(i)} = 1\}}{\sum_{i=1}^m 1\{y^{(i)} = 1\}}\end{aligned}$$

φ_y : $y = 1$ 样本的比例

$\varphi_{j|y=1}$: 找出所有 $y = 1$ 的样本, 统计其中出现词 x_j 的比例

也可以回顾一下 GDA 的 MLE 结论,

$$\begin{aligned}\varphi &= \frac{\sum_{i=1}^m 1\{y^{(i)} = 1\}}{m} \text{ (prob of } y \text{ of label 1),} \\ \mu_0 &= \frac{\sum_{i=1}^m 1\{y^{(i)} = 0\}x^{(i)}}{\sum_{i=1}^m 1\{y^{(i)} = 0\}} \text{ (所有 } y=0 \text{ 样本的特征均值),} \\ \mu_1 &= \frac{\sum_{i=1}^m 1\{y^{(i)} = 1\}x^{(i)}}{\sum_{i=1}^m 1\{y^{(i)} = 1\}} \text{ (所有 } y=1 \text{ 样本的特征均值),} \\ \Sigma &= \frac{1}{m} \sum_{i=1}^m (x^{(i)} - \mu_{y^{(i)}})(x^{(i)} - \mu_{y^{(i)}})^T\end{aligned}$$

可以看出, Naive Bayes 与 GDA 同属于 generative model, 两者参数的更新以及计算十分简便! (使用简单的统计计算, 而非梯度下降等复杂的迭代法!)

Biggest problems:

what if we get zeros in some of the equations?
(the next Laplace moving will solve it!!!)

• ▶ Prediction Rule

Once we've fit the parameters (φ_y and $\varphi_{j|y}$), we're able to do the label predictions.

Given x , 对于二分类问题, 我们比较两类的后验概率: $p(y = 1|x), p(y = 0|x)$ 的大小。用贝叶斯公式代入, 因为分母 $p(x)$ 相同, 可以省略, 所以实际上就是比较:

$$p(x|y = 1)p(y = 1) \stackrel{?}{=} p(x|y = 0)p(y = 0)$$

① Score ($y = 1$):

$$\begin{aligned}\text{Score}(y = 1) &= p(x|y = 1)p(y = 1) \\ &= p(y = 1) \prod_{j=1}^n p(x_j | y = 1)\end{aligned}$$

② Score ($y = 0$):

$$\begin{aligned}\text{Score}(y = 0) &= p(x|y = 0)p(y = 0) \\ &= p(y = 0) \prod_{j=1}^n p(x_j | y = 0)\end{aligned}$$

哪一个后验概率值大, 输出的预测就是对应的 label

§ Lec VI

1. Continuation of Naive Bayes

• Laplace Smoothing

- Let's continue with the previous Naive Bayes.

recap : previous naive Bayes MLE conclusions:

$$\varphi_y = \frac{\sum_{i=1}^m 1\{y^{(i)} = 1\}}{m}$$

$$\varphi_{j|y=1} = \frac{\sum_{i=1}^m 1\{x_j^{(i)} = 1, y^{(i)} = 1\}}{\sum_{i=1}^m 1\{y^{(i)} = 1\}}$$

And at prediction time, we'll calculate:

$$p(y = 1|x) = \frac{p(x|y = 1)p(y = 1)}{p(x)}$$

$$= \frac{p(x|y = 1)p(y = 1)}{p(x|y = 1)p(y = 1) + p(x|y = 0)p(y = 0)}$$

Imagine we have a key word x_{1000} that had never appeared in previous texts. So the prob $p(x_{1000} = 1 | y = 1) = 0$. Similarly, $p(x_{1000} = 1 | y = 0) = 0$. What causes Naive Bayes to break down is that : if we use these as estimates of the parameters :

$$\varphi_{1000|y=1} = 0$$

$$\varphi_{1000|y=0} = 0$$

Then in Naive Bayes:

$$p(y = 1|x) = \frac{\text{p(x|y=1)}p(y = 1)}{\text{p(x|y=1)}p(y = 1) + \text{p(x|y=0)}p(y = 0)}$$

Because:

$$p(x|y = 1) = \prod_{i=1}^n p(x_i | y),$$

$$p(x_{1000} | y = 1) = 0$$

So the red part = 0. Similarly, we have the blue part = 0 because $p(x_{1000} | y = 0) = 0$

That means if we encounter the key word x_{1000} , our classifier will estimate the prob as:

$$p(y = 1 | x) = \frac{0}{0 + 0}$$

Additionally, it's a bad idea to estimate the probability of something as 0 just because we've not seen it once yet, in statistic sense.

- In Laplace smoothing, generally:

Assume we have a random variable :

$$x \in \{1, \dots, k\}$$

So our estimates of the *prior* parameters(using MLE) are (these will serve as φ_y in our Naive Bayes model) :

$$\text{Estimate : } p(x = j) = \frac{\sum_{i=1}^m 1\{x^{(i)} = j\}}{m}$$

However, to avoid prob 0, we do Laplace smoothing:

$$p(x = j) = \frac{\sum_j = 1^m 1\{x^{(i)} = j\} + 1}{m + k}$$

Note that it is still a valid probability (the prob sum = 1):

$$\sum_{j=1}^k p(x = j) = \frac{m + k}{m + k} = 1$$

回到我们的朴素贝叶斯分类器问题上，使用了拉普拉斯平滑之后，对参数的估计就写成了下面的形式：

$$\varphi_{j|y=1} = \frac{\sum_{i=1}^m 1\{x_j^{(i)} = 1, y^{(i)} = 1\} + 1}{\sum_{i=1}^m 1\{y^{(i)} = 1\} + 2},$$

$$\varphi_{j|y=0} = \frac{\sum_{i=1}^m 1\{x_j^{(i)} = 1, y^{(i)} = 0\} + 1}{\sum_{i=1}^m 1\{y^{(i)} = 0\} + 2}$$

• Multivariate Bernoulli representation

Recall our text representation for Naive Bayes:

$$x = \begin{bmatrix} x_1 \\ \vdots \\ x_i \\ \vdots \\ x_n \end{bmatrix}$$

As x_i is bi-valued (0 or 1), it drops the information that maybe certain words occur in the e-mail text more than once !

• Multinomial Event Model

Now there's another way to represent a piece of the i th text: Essentially, we encode a n_i -dim vector by sequentially replacing each word with its corresponding vocabulary index, where n_i equals the i th text's length.

e.g:

$$x = \begin{bmatrix} 1200 \\ \vdots \\ 6300 \\ \vdots \\ 400 \end{bmatrix} \in \mathbb{R}^{n_i}$$

n_i = length of text i

$x_j \in \{1, \dots, n\}$ (n = vocab size)

Then we're gonna build a generative model:

$$p(x, y) = p(x|y)p(y)$$

$$\stackrel{\text{assume}}{=} \prod_{j=1}^n p(x_j | y) p(y)$$

现在可能会有些疑惑，这不正是我们之前在朴素贝叶斯中使用到的方法吗？（朴素贝叶斯中我们利用条件概率链式拆分，最后引入假设条件概率独立性最后就拆分成上述这个连乘式）

现在开始找不同：注意刚才的 n_i 与 x_j 的定义与朴素贝叶斯中的定义大有不同！朴素贝叶斯中 n_i = 词汇表长度，而 x_j 是各个单词的索引而并非“非 0 则 1”，所以现在这里是一个多项式概率而不是二元或伯努利概率。

这里我们要使用到的参数与之前相同：

$$\varphi_y = p(y = 1)$$

$$\varphi_{k|y=0} = p(x_j = k | y = 0)$$

注意第二个概率等式：右侧含义是“如果 y 的标签为 0，第 j 个词是 k 的概率”。但是，左侧并没有出现 j ，因为我们做出假设这个词 k 在每个位置出现的概率相同（与位置 j 无关）。

现在，我们来讨论如何从训练集通过 MLE 拟合得到上述两个参数。

如果给定一个训练集 $\{(x^{(i)}, y^{(i)}); i = 1, \dots, m\}$ ，其中 $x^{(i)} = (x_1^{(i)}, x_2^{(i)}, \dots, x_{n_i}^{(i)})$ （这里的 n_i = 第 i 个训练样本中的单词数目， $x^{(i)}$ 集合就表示第 i 个训练样本中的所有单词），那么这个数据的似然函数如下所示：

$$L(\varphi, \varphi_{k|y=0}, \varphi_{k|y=1}) = \prod_{i=1}^m p(x^{(i)}, y^{(i)})$$

$$= \prod_{i=1}^m \left(\prod_{j=1}^{n_i} p(x_j^{(i)} | y; \varphi_{k|y=0}, \varphi_{k|y=1}) \right) p(y^{(i)}; \varphi_y)$$

对于这个含有双重连乘符的公式理解：

① 第一层（外层 $\prod_{i=1}^m$ ）：遍历数据集里的每一个训练样本（总共有 m 个），最后就是要连乘所有的 $p(x^{(i)}, y^{(i)})$

$$\prod_{i=1}^m p(x^{(i)}, y^{(i)})$$

② 第二层（中间层 $\prod_{j=1}^{n_i}$ ）：针对当前第 i 个训练样本，先把当前这个概率拆分为 $p(x^{(i)} | y^{(i)})$ ，再把 $p(x^{(i)} | y^{(i)})$ 拆成连乘式，表示遍历里面的每一个单词

$$p(x^{(i)} | y^{(i)})p(y^{(i)}) = \left(\prod_{j=1}^{n_i} p(x_j^{(i)} | y^{(i)}) \right) p(y^{(i)})$$

对上述似然函数使用 MLE 可以得到对参数的最大似然估计结果：

$$\begin{aligned}\varphi_{k|y=1} &= \frac{\sum_{i=1}^m 1\{y^{(i)} = 1\} \left(\sum_{j=1}^{n_i} 1\{x_j^{(i)} = k\} \right)}{\sum_{i=1}^m (1\{y^{(i)} = 1\}) n_i}, \\ \varphi_{k|y=0} &= \frac{\sum_{i=1}^m 1\{y^{(i)} = 0\} \left(\sum_{j=1}^{n_i} 1\{x_j^{(i)} = k\} \right)}{\sum_{i=1}^m (1\{y^{(i)} = 0\}) n_i}, \\ \varphi_y &= \frac{\sum_{i=1}^m 1\{y^{(i)} = 1\}}{m}\end{aligned}$$

比如说简单解读理解一下 $\varphi_{k|y=0}$ 的计算过程：首先选择所有 $y=0$ 的样本，查看里面所有单词，再看里面单词 k 数量的占比是多少，比值就是我们对于单词 k 出现在 $y=0$ 样本中的任意位置的概率大小估计。

如果要使用拉普拉斯平滑来估计 $\varphi_{k|y=0}$ 和 $\varphi_{k|y=1}$ ，就在分子上加 1，分母上加 $|V|$ ：

$$\begin{aligned}\varphi_{k|y=1} &= \frac{\sum_{i=1}^m 1\{y^{(i)} = 1\} \left(\sum_{j=1}^{n_i} 1\{x_j^{(i)} = k\} \right) + 1}{\sum_{i=1}^m (1\{y^{(i)} = 1\}) n_i + |V|}, \\ \varphi_{k|y=0} &= \frac{\sum_{i=1}^m 1\{y^{(i)} = 0\} \left(\sum_{j=1}^{n_i} 1\{x_j^{(i)} = k\} \right) + 1}{\sum_{i=1}^m (1\{y^{(i)} = 0\}) n_i + |V|}\end{aligned}$$

注意： $|V|$ 的由来：

在多项式模型中，所有可能的单词一共有 $|V|$ 个(vocab size)。对于 $y=1$ 的所有样本，我们必须保证：

$$\sum_{k=1}^{|V|} \varphi_{k|y=1} = 1$$

也即在平滑之后，概率仍将满足归一化条件，因此分母将加上 $|V|$ (词汇表大小)

补充：如果处理到的词不在词汇表中？

法一：直接丢弃

法二：将所有稀有词映射到一个特殊标记 UNK

Advantages of Naive Bayes:

computationally efficient (don't require iterative algorithm to update parameters); and quick to implement

2. Support Vector Machine (SVM)

(help to find **non-linear** decision boundaries)

• Lead-in

Imagine we have this dataset (配一个非线性边界分类的数据点图)：

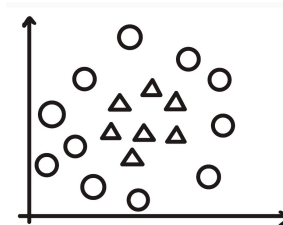


Figure 21: non-linear classification

Ordinary Logistic regression only gives out linear decision boundaries because it fits the function: $\theta^T x$ — a hyperplane.

However, if we change our feature vector x from

$$x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

to

$$x = \begin{bmatrix} x_1 \\ x_2 \\ x_1^2 + x_2^2 \end{bmatrix}$$

then the decision boundary would be:

$$\theta^T x = \theta_1 x_1 + \theta_2 x_2 + \theta_3 (x_1^2 + x_2^2) = 0$$

Thus it's a non-linear boundary !

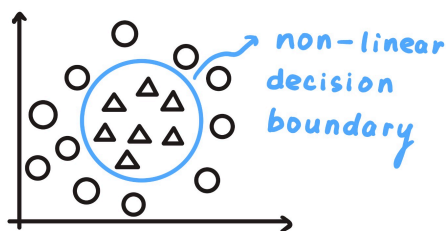


Figure 22: non-linear decision boundary

However, the choosing of the features can be hard because we don't know what set of feature could get us a right decision boundary.

What SVM does is that it's able to derive an algorithm that takes input features $x_1, x_2 \dots$ and maps them to a higher dimensional set of features. And then it applies a linear classifier to learn non-linear decision boundaries (similar to logistic regression).

• Fundamental intuitions about functional and geometric margins

• functional margin

The functional margin of a classifier measures how confidently and accurately classify an example.

e.g: binary classification + logistic regression

$$h_\theta(x) = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}}$$

Thus the classifier will predict 1 if $\theta^T x \geq 0$ ($h_\theta(x) \geq 0.5$), otherwise it'll predict 0 .

Therefore, if $y^{(i)} = 1$, we hope that $\theta^T x \gg 0$; and if $y^{(i)} = 0$, we hope that $\theta^T x \ll 0$.

这个地方我们已然发现之前定义中标签 y 取值的不便性：我们无法用 $y \cdot (\theta^T x)$ 来统一化我们最后最大化/最小化的函数目标，以作为我们的评判标准。（因为当 $y = 0$ 时上述式子必然为 0!）所以我们会引出后面的 Notation changes 。

• Geometric margin

Assume the dataset is linearly separable. （所有样本都能被正确分类）（配手绘图，两种分割线的比较）

What SVM does in the low-dim space is an optimal margin classifier, which aims to find a separation line to maximize the geometric margin.

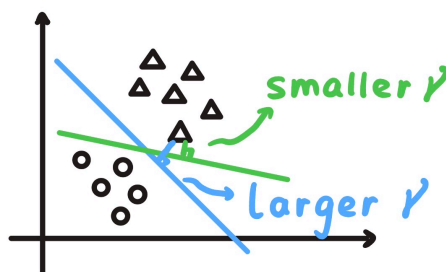


Figure 23: maximizing geometric margin

- **Notation changes in SVMs :**

由于我们在 functional margin intuition 处看到之前使用的 $y \in \{0, 1\}$ 这种标签的不便性, 我们采用以下的标签定义:

① labels $y \in \{-1, +1\}$

② have an output value $h \in \{-1, +1\}$
(instead of outputting an hypothesis probability in logistic regression, SVMs will output $\{-1, +1\}$ in cases below)

$$g(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ -1 & \text{otherwise.} \end{cases}$$

That implies an hard output transition from -1 to $+1$. (这个在后面的 Optimal margin Classifier 中可以找到对应的原因, 读者可以先暂置这一点, 等会儿看完这一节后回头反思这个地方这样定义的原因)

- **Parameters**

1. Previously, in logistic regression: (parameter: θ)

$$h_{\theta}(x) = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}}, \quad x \in \mathbb{R}^{n+1}, x_0 = 1$$

2. Here in SVM: (parameters: w, b)

$$h_{w,b}(x) = g(w^T x + b), \quad x \in \mathbb{R}^n, b \in \mathbb{R}$$

(no longer the constraint that $x_0 = 1$)

One way to intuitively understand the form transition of the parameters is:

$$\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_n \end{bmatrix}, \quad b = \theta_0, \quad w = \begin{bmatrix} \theta_1 \\ \vdots \\ \theta_n \end{bmatrix}$$

- **Formal Definitions**

Now we come back to the definition of *functional margin*, and applies it to SVM.

1. Functional Margin

1) **Define** : functional margin of hyperplane defined by (w, b) wrt(with respect to) $(x^{(i)}, y^{(i)})$:

$$\hat{\gamma}^{(i)} = y^{(i)}(w^T x + b)$$

$w^T x + b$ actually defines a hyperplane separating out positive and negative examples.

Here we want our classifier to achieve a large functional margin $\hat{\gamma}^{(i)}$.

① If $y^{(i)} = 1$, we want $w^T x + b \gg 0$;

② If $y^{(i)} = -1$, we want $w^T x + b \ll 0$.

Combining the two statements above, we basically want to maximize $\gamma^{(i)}$ in both cases. (want $\gamma^{(i)} \gg 0$)

If $\hat{\gamma}^{(i)} > 0$, that means $h(x^{(i)}) = y^{(i)}$. (the algorithm gets the right label)

2) **Define** : functional margin wrt(with respect to) training set :

$$\hat{\gamma} = \min_{i=1, \dots, m} \hat{\gamma}^{(i)}$$

In the previous definition we actually define functional margins with respect to a *single training example*. (how are we doing well on that training example ?) And now this definition actually ask : how well are you doing *on the worst example* in the training set ?

(note that here we assume that the training set is linearly separable)

We find that it's simple to cheat on the functional margin value if we just multiply our parameters w, b by a factor of k . So a common way to avoid this cheating is to normalize the

length of all the parameters !

e.g: we impose a constraint:

$$\|w\| = 1$$

or do the replacement:

$$(w, b) \rightarrow \left(\frac{w}{\|w\|}, \frac{b}{\|w\|} \right)$$

2. Geometric margin

• geometric margin wrt a single example

If we have a linear classifier : $w^T x + b = 0$

If we have a positive example $(x^{(i)}, y^{(i)})$ (data point), and our classifier classifies this example correctly. Now we define the geometric margin of this training example is equal to the distance(Euclidean distance) between the data point and the decision boundary.

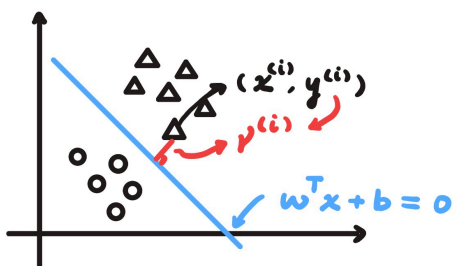


Figure 24: geometric margin illustration

1) **Define** : Geometric margin of hyperplane (w, b) wrt $(x^{(i)}, y^{(i)})$:

$$\gamma^{(i)} = \frac{y^{(i)}(w^T x^{(i)} + b)}{\|w\|}$$

Relation between geometric margin and functional margin:

$$\text{geometric} = \frac{\text{functional}}{\|w\|}$$

2) **Define**: Geometric margin wrt the training set :

$$\gamma = \min_{i=1, \dots, m} \gamma^{(i)}$$

Note: functional: $\hat{\gamma}$; geometric: γ

• Optimal Margin Classifier

Goal : Choose w, b to maximize γ (geometric margin).

$$\begin{aligned} & \max_{\gamma, w, b} \gamma \\ \text{s.t.} \quad & \frac{y^{(i)}(w^T x^{(i)} + b)}{\|w\|} \geq \gamma \quad i = 1, \dots, m \end{aligned}$$

Thus we want to maximize γ but still keeping every geometric margin bigger than γ .

It turns out that it's a *non-convex* optimization problem so it's difficult to solve without gradient descent. (to find out w, b)

As $\gamma = \frac{\hat{\gamma}}{\|w\|}$, our **goal** is :

$$\max_{w, b} \frac{\hat{\gamma}}{\|w\|}$$

($\hat{\gamma}$ is the smallest functional margin in the training dataset)

现在，我们考虑任意地对 w, b 进行相同倍数 k 的缩放，最后得到的决策边界（超平面） $w^T x + b = 0$ 会保持不变，那么我们最后得到的几何间隔 γ 大小也就会保持不变，但是函数间隔 $\hat{\gamma}$ 的大小将会被缩放相同的倍数 k 。

既然缩放 w, b 不会改变最后的目标函数 γ 的大小，只会改变 $\hat{\gamma}$ 的大小，那么我们不妨假设

$$\hat{\gamma} = 1$$

这样一来，我们最后要最大化的目标函数就是

$$\gamma = \frac{1}{\|w\|}$$

现在看出，我们的目标转变为

$$\min_{w,b} \|w\|$$

也即等价于（为了求导方便，但是最优解位置不变）

$$\min_{w,b} \frac{1}{2} \|w\|^2$$

现在来看下应该施加怎样的约束条件：从最开始的

$$\frac{y^{(i)}(w^T x^{(i)} + b)}{\|w\|} \geq \gamma \quad i = 1, \dots, m$$

我们简单进行变形：

$$y^{(i)}(w^T x^{(i)} + b) \geq \gamma \|w\| \quad i = 1, \dots, m$$

由于

$$\gamma \|w\| = \frac{\hat{\gamma}}{\|w\|} \cdot \|w\| = \hat{\gamma} = 1$$

所以我们要最小化的凸二次目标函数对应的约束条件就是：

$$y^{(i)}(w^T x^{(i)} + b) \geq 1 \quad i = 1, \dots, m$$

Therefore, we can actually reformulate the previous problem into an equivalence:

$$\begin{aligned} & \min_{w,b} \frac{\|w\|^2}{2} \\ \text{s.t. } & y^{(i)}(w^T x^{(i)} + b) \geq 1 \quad i = 1, \dots, m \end{aligned}$$

Now this is a convex optimization problem.

The assumption is that the dataset is linearly separable.（每一个样本必须可以被正确分类！）

Then, optimal margin classifier serves as the basic building block of SVM.

• Kernels

The kernels will allow us to choose an infinite large set of features, which will be discussed in lectures later.

§ Lec VII

Kernels & Soft Margin SVM

• Recap :

In last lecture, we knew optimal margin classifier's goal is:

$$\max_{\gamma, w, b} \gamma$$

$$s.t. \quad \frac{y^{(i)}(w^T x^{(i)} + b)}{\|w\|} \geq \gamma \quad i = 1, \dots, m$$

where the constraint means: “every example has geometric margin greater than or equal to γ ”, but we want to set γ as big as possible.

So essentially we have to play with parameters w, b to *maximize the worst geometric margin in the dataset*. (because that's basically the threshold of γ)

And with our previously derived properties, we change our final goal to:

$$\min_{w, b} \frac{\|w\|^2}{2}$$

$$s.t. \quad y^{(i)}(w^T x^{(i)} + b) \geq 1 \quad i = 1, \dots, m$$

• Additional Assumption : representation theorem

In order to derive SVM, we're gonna make an additional restriction:

Assumption: Suppose w can be represented as a linear combination of the training examples :

$$w = \sum_{i=1}^m \alpha_i x^{(i)}$$

(this introduction of representation theorem can be proved in a long procedure...)

Let's rewrite the assumption adding $y^{(i)}$. In this case, $y^{(i)} = \pm 1$, so this makes sense :

$$w = \sum_{i=1}^m \alpha_i y^{(i)} x^{(i)}$$

Then let's see some intuitions about the correctness of this theorem.

Intuition #1:

In logistic regression, we run this iterative gradient descent to update the parameter θ :

$$\theta = 0 \quad (\text{initialization})$$

$$\theta := \theta - \alpha (h_{\theta}(x^{(i)}) - y^{(i)}) x^{(i)} \quad (\text{update})$$

In updating, we can see θ will always end up as a linear combination of the training examples $x^{(i)}$.

The same happens in batch gradient descent.

Intuition #2:

It turns out that vector w is always at 90 degrees to the decision boundary, and the decision boundary separates where we predict positive from where we predict negative.

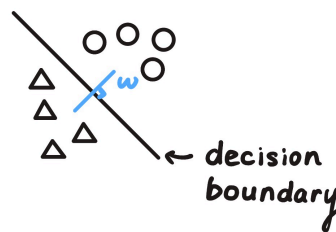


Figure 25: w is orthogonal to the boundary

Using linear algebra, we can show that w lies in the span of the training samples. w pins the direction of the decision boundary. (b only

changes the relative position) (两张手绘图：两个样本的；以及三维空间中特殊 $x_3 = 0$ 的)

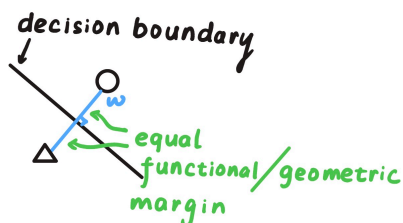


Figure 26: orthogonality e.g 1

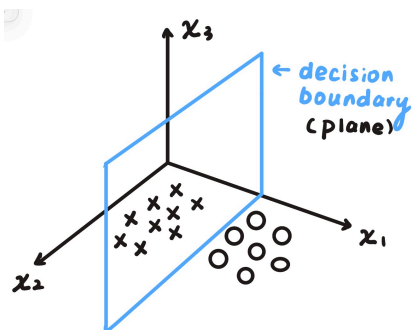


Figure 27: orthogonality e.g 2

1) Substituting into w , the optimization goal can be reformulated as :

$$\begin{aligned}\min_{w,b} \frac{\|w\|^2}{2} &= \min_{w,b} \frac{1}{2} \left(\sum_{i=1}^m \alpha_i y^{(i)} x^{(i)} \right)^T \left(\sum_{j=1}^m \alpha_j y^{(j)} x^{(j)} \right) \\ &= \min_{w,b} \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m (\alpha_i \alpha_j y^{(i)} y^{(j)} x^{(i)T} x^{(j)}) \\ &= \min_{w,b} \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m (\alpha_i \alpha_j y^{(i)} y^{(j)} \langle x^{(i)}, x^{(j)} \rangle)\end{aligned}$$

(note that “ $\langle x, z \rangle$ ” means inner product)

2) Also, the restriction can be reformulated as:

$$\begin{aligned}y^{(i)} (w^T x^{(i)} + b) &\geq 1 \\ \Leftrightarrow y^{(i)} \left(\sum_{j=1}^m \alpha_j y^{(j)} x^{(j)} \right)^T x^{(i)} + b &\geq 1 \\ \Leftrightarrow y^{(i)} \left(\sum_{j=1}^m \alpha_j y^{(j)} \langle x^{(j)}, x^{(i)} \rangle + b \right) &\geq 1\end{aligned}$$

The only place that the feature vectors appears is in the inner product. So if we can compute this efficiently, we can handle with manipulating even infinite dimensional feature vectors.

• Goal & Optimization Reformulation

previously, we have :

$$\begin{aligned}\min_{w,b} \frac{\|w\|^2}{2} \\ \text{s.t. } y^{(i)} (w^T x^{(i)} + b) &\geq 1 \quad i = 1, \dots, m\end{aligned}$$

- And we can simplify this optimization further to an “**Dual optimization problem**” : (using convex optimization theory or we can see it as an pure algebra method (cancel out b))

$$\begin{aligned}\max \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m y^{(i)} y^{(j)} \alpha_i \alpha_j \langle x^{(i)}, x^{(j)} \rangle \\ \text{s.t. } \alpha_i \geq 0, \\ \sum_{i=1}^m y^{(i)} \alpha_i = 0.\end{aligned}$$

Then,

- 1) we solve α_i 's
- 2) To make prediction:
compute :

$$\begin{aligned} h_{w,b}(x) &= g(w^T x + b) \\ &= g\left(\left(\sum_{i=1}^m \alpha_i y^{(i)} x^{(i)}\right)^T x + b\right) \\ &= g\left(\sum_{i=1}^m \alpha_i y^{(i)} \left((x^{(i)})^T x\right) + b\right) \\ &= g\left(\sum_{i=1}^m \alpha_i y^{(i)} \langle x^{(i)}, x \rangle + b\right) \end{aligned}$$

• Kernel trick

1) Write the whole algorithm in terms of $\langle x^{(i)}, x^{(j)} \rangle$ (inner product of two different training examples, or *i.e.* $\langle x, z \rangle$ to simplify the notation)

2) Let there be some mapping from the original input features $X \rightarrow \Phi(X)$ (high dimensional set of features)

e.g.

$$\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \rightarrow \Phi(x) = \begin{bmatrix} x_1 \\ x_2 \\ x_1 x_2 \\ x_1^2 x_2 \\ \vdots \end{bmatrix}$$

Thus $\Phi(x)$ could be infinite dimensional.

3) Find a way to compute :

$$K(x, z) = \Phi(x)^T \Phi(z)$$

This is the kernel function, and there are tricks to efficiently compute the dot product even when $\Phi(x)$ and $\Phi(z)$ are incredibly high dimensional.

4) Replace $\langle x, z \rangle$ in the algorithm with $K(x, z)$

Because if we could do this, we are actually running the whole learning algorithm on the high dimensional set of features.

Running algorithm on high dimension could be very computationally expensive, so the essence of kernel trick is that because we've written the whole algorithm in an inner product form, so we can always just compute the kernels, no need to explicitly compute $\Phi(x)$.

• ▶ A simple example of kernels

1) Suppose :

$$x = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} \in \mathbb{R}^n \xrightarrow{\text{map}} \Phi(x) = \begin{bmatrix} x_1 x_1 \\ x_1 x_2 \\ x_1 x_3 \\ x_2 x_1 \\ x_2 x_2 \\ x_2 x_3 \\ x_3 x_1 \\ x_3 x_2 \\ x_3 x_3 \end{bmatrix} \in \mathbb{R}^{n^2}$$

$$\Phi(z) = \begin{bmatrix} z_1 z_1 \\ z_1 z_2 \\ z_1 z_3 \\ z_2 z_1 \\ z_2 z_2 \\ z_2 z_3 \\ z_3 z_1 \\ z_3 z_2 \\ z_3 z_3 \end{bmatrix} \in \mathbb{R}^{n^2}$$

Because $\Phi(x)$ is n^2 -dimensional, we need $O(n^2)$ time to compute $\Phi(x)$ or compute $\Phi(x)^T \Phi(z)$ explicitly.

Now if we use the kernel trick, we'll do it in a better way.

$$K(x, z) = \Phi(x)^T \Phi(z) \stackrel{\text{proved}}{=} (x^T z)^2$$

Why efficient ? Remember $x, z \in \mathbb{R}^n$, so $x^T z$ only need $O(n)$ time to compute. Then we just have to square this scalar.

So we just have to do this proving:

$$x^T z = \sum_{i=1}^n x_i z_i$$

As

$$\begin{aligned} (x^T z)^2 &= \left(\sum_{i=1}^n x_i z_i \right) \left(\sum_{j=1}^n x_j z_j \right) \\ &= \sum_{i=1}^n \sum_{j=1}^n x_i z_i x_j z_j \\ &= \sum_{i=1}^n \sum_{j=1}^n (x_i x_j) (z_i z_j) \end{aligned}$$

Now if we take a look at the previous $\Phi(x)^T \Phi(z)$, we'll see they are exactly the same !
(此处可以配一张向量点乘配对手绘图)

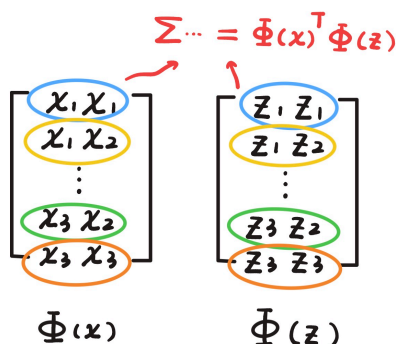


Figure 28: dot product illustration

Therefore :

$$\Phi(x)^T \Phi(z) \stackrel{\text{proved}}{=} (x^T z)^2$$

2) Now if do some little changes to the kernel function:

$$K(x, z) = (x^T z + c)^2, \quad c \in \mathbb{R} \text{ (a constant)}$$

That's equal to modifying the features as follows:

$$\Phi(x) = \begin{bmatrix} x_1 x_1 \\ x_1 x_2 \\ x_1 x_3 \\ x_2 x_1 \\ x_2 x_2 \\ x_2 x_3 \\ x_3 x_1 \\ x_3 x_2 \\ x_3 x_3 \\ \sqrt{2c} x_1 \\ \sqrt{2c} x_2 \\ \sqrt{2c} x_3 \end{bmatrix}$$

3) If got changed to :

$$K(x, z) = (x^T z + c)^d, \quad c, d \in \mathbb{R}$$

This responds to :

$\Phi(x)$ has all $\binom{n+d}{d}$ features of monomials up to order d .

Summary:

SVM = Optimal margin classifier + kernel trick

好的可视化视频:

<https://www.youtube.com/watch?v=OdINM96sHio>

So SVM actually find a linear decision boundary (using optimal margin classifier) in a high-dimensional space ! And when we look at the original feature space we'll find a non-linear decision boundary.

- ▶ **How to make Kernels ?** (一个有效的 Kernel 应该满足怎样的性质?)

Guiding principle :

“If x, z are ‘similar’, $K(x, z) = \Phi(x)^T \Phi(z)$ is ‘large’ (the inner product of two similar vectors should be large). And the other way round.” 有了这个指导性的直觉，我们相应的 kernel function 也应该满足当 x, z 接近的时候值大，当不接近的时候值小。

As the condition is that

$$K(x, z) = \Phi(x)^T \Phi(z)$$

This puts some constraints on our kernel functions that we could choose:

①

$$K(x, x) = \Phi(x)^T \Phi(x) \geq 0$$

Let $\{x^{(1)}, \dots, x^{(d)}\}$ be d points.

Let $K \in \mathbb{R}^{d \times d}$ (“kernel matrix”)

$$K_{ij} = K(x^{(i)}, x^{(j)}) = \Phi(x^{(i)})^T \Phi(x^{(j)})$$

Therefore, given any vector z ,

$$\begin{aligned} z^T K z &= \sum_i \sum_j z_i K_{ij} z_j \\ &= \sum_i \sum_j z_i \left(\Phi(x^{(i)})^T \Phi(x^{(j)}) \right) z_j \\ &= \sum_i \sum_j z_i \left(\sum_k (\Phi(x^{(i)}))_k (\Phi(x^{(j)}))_k \right) z_j \\ &= \sum_k \sum_i \sum_j z_i (\Phi(x^{(i)}))_k (\Phi(x^{(j)}))_k z_j \\ &= \sum_k \left(\sum_i z_i (\Phi(x^{(i)}))_k \right)^2 \\ &\geq 0 \end{aligned}$$

So K (the kernel matrix) is positive semi-definite ! (more generally, it’s a sufficient condition for our K to be a valid kernel function) (事实上，马上我们就会证明这个“半正定”的条件就是一个判定有效核的充分条件)

Mercer’s Theorem :

K is a valid kernel function

i.e $\exists \Phi$ s.t. $K(x, z) = \Phi(x)^T \Phi(z)$

if and only if

For any d points $\{x^{(i)}, \dots, x^{(d)}\}$, the corresponding kernel matrix K is **positive semi-definite**.

Widely used kernels:

- ▶ **Linear kernel**

$$K(x, z) = x^T z$$

$$\Phi(x) = x$$

(no high dimensional feature mapping)

- ▶ **Gaussian kernel**

$$K(x, z) = \exp\left(-\frac{\|x - z\|^2}{2\sigma^2}\right)$$

$$\Phi(x) \in \mathbb{R}^\infty$$

- ▶ **Polynomial kernel**

$$K(x, z) = (x^T z)^d$$

$$\Phi(x) \in \mathbb{R}^{\binom{n+d}{d}}$$

我们不妨可以借助 Mercer's Theorem 来证明一下高斯核是一个有效的核：

首先，直觉上，当 x, z 接近的时候 $K(x, z)$ 值较大，满足我们的直觉性原则。

现在来证明 $\forall z; z^T K z \geq 0$, K 是高斯核对应的矩阵。

由于：

$$K_{ij} = \exp\left(-\frac{\|x^{(i)} - x^{(j)}\|^2}{2\sigma^2}\right)$$

(其中 $\{x^{(1)}, \dots, x^{(i)}, \dots, x^{(j)}, \dots\}$ 是我们已有的样本点)；所以原式为：

$$\begin{aligned} z^T K z &= \sum_i \sum_j z_i K_{ij} z_j \\ &= \sum_i \sum_j z_i \exp\left(-\frac{\|x^{(i)} - x^{(j)}\|^2}{2\sigma^2}\right) z_j \end{aligned}$$

法一：将 $K_{ij} = \exp\left(-\frac{\|x^{(i)} - x^{(j)}\|^2}{2\sigma^2}\right)$ 泰勒展开

由于：

$$\begin{aligned} K_{ij} &= \exp\left(-\frac{\|x^{(i)} - x^{(j)}\|^2}{2\sigma^2}\right) \\ &= \exp\left(-\frac{1}{2\sigma^2}(x^{(i)T}x^{(i)} - 2x^{(i)T}x^{(j)} + x^{(j)T}x^{(j)})\right) \\ &= \exp\left(-\frac{x^{(i)T}x^{(i)}}{2\sigma^2}\right) \cdot \exp\left(\frac{x^{(i)T}x^{(j)}}{\sigma^2}\right) \cdot \exp\left(-\frac{x^{(j)T}x^{(j)}}{2\sigma^2}\right) \end{aligned}$$

看得出来第 1、3 项都只与样本点 $x^{(i)}, x^{(j)}$ 的取值有关，且恒 > 0 ，所以我们可以把这两块分别设为 $f(x^{(i)}), f(x^{(j)})$ ，然后继续：

$$z^T K z = \sum_i \sum_j z_i \left(f(x^{(i)}) \exp\left(\frac{x^{(i)T}x^{(j)}}{\sigma^2}\right) f(x^{(j)}) \right) z_j$$

由于 $f(x^{(i)}), f(x^{(j)})$ 分别只与 i, j 有关，我们把它合并进相应的系数里，并令：

$$\tilde{z}_i = z_i f(x^{(i)}); \quad \tilde{z}_j = z_j f(x^{(j)})$$

那么原式变成：

$$z^T K z = \sum_i \sum_j \tilde{z}_i \exp\left(\frac{x^{(i)T}x^{(j)}}{\sigma^2}\right) \tilde{z}_j$$

对指数部分作 Taylor expansion (展成无穷级数)：

$$\begin{aligned} z^T K z &= \sum_i \sum_j \tilde{z}_i \tilde{z}_j \sum_{k=0}^{\infty} \frac{1}{k!} \left(\frac{x^{(i)T}x^{(j)}}{\sigma^2}\right)^k \\ &= \sum_i \sum_j \tilde{z}_i \tilde{z}_j \left[\sum_{k=0}^{\infty} \frac{1}{k! \cdot \sigma^{2k}} (x^{(i)T}x^{(j)})^k \right] \end{aligned}$$

由于全是正数，绝对收敛，可以任意交换求和顺序：

$$z^T K z = \sum_{k=0}^{\infty} \frac{1}{k! \cdot \sigma^{2k}} \left[\sum_i \sum_j \tilde{z}_i \tilde{z}_j (x^{(i)T}x^{(j)})^k \right]$$

我们惊奇地发现内层的 $(x^{(i)T}x^{(j)})^k$ 就是多项式核函数！于是代入多项式核函数 (k 次式的) 对应的 Φ_k ：

$$(x^{(i)T}x^{(j)})^k = \Phi_k(x^{(i)})^T \Phi_k(x^{(j)})$$

所以原式变成：

$$z^T K z = \sum_{k=0}^{\infty} \frac{1}{k! \cdot \sigma^{2k}} \left[\sum_i \sum_j \tilde{z}_i \tilde{z}_j \Phi_k(x^{(i)})^T \Phi_k(x^{(j)}) \right]$$

我们可以分别合并关于 i, j 的求和：

$$z^T K z = \sum_{k=0}^{\infty} \frac{1}{k! \cdot \sigma^{2k}} \left[\left(\sum_i \tilde{z}_i \Phi_k(x^{(i)}) \right)^T \sum_j \tilde{z}_j \Phi_k(x^{(j)}) \right]$$

注意： $\sum_i \tilde{z}_i \Phi_k(x^{(i)})$ 与 $\sum_j \tilde{z}_j \Phi_k(x^{(j)})$ 二者其实一样！仅仅是哑变量 i, j 的区别！

所以，右侧这个式子就是一个向量的模，也便 ≥ 0 。

所以此时

$$\begin{aligned} z^T K z &= \sum_{k=0}^{\infty} \frac{1}{k! \cdot \sigma^{2k}} \left[\left\| \sum_i \tilde{z}_i \Phi_k(x^{(i)}) \right\|^2 \right] \\ &\geq 0 \end{aligned}$$

利用 Mercer's Theorem, 则高斯核是一个有效的核。

法二：把里面的高斯函数那一块写成无穷积分形式（此处略）

Note that the *kernel trick* can be married with many other learning algorithms (e.g. PCA), but the most successful and widely used one is on SVM.

• Fix the assumption of linearly separable data

When we map our data set to a high-dimensional space, the dataset does become more separable. But if the data is noisy, we don't want to try too hard to separate every example, as it may lead to really complicated decision boundary. (这个地方配一个手绘示意图!)

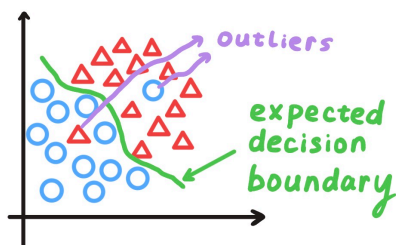


Figure 29: not linearly separable data boundary

• ℓ_1 norm soft margin SVM

1) Previous basic algorithm:

$$\min \frac{1}{2} \|w\|^2$$

$$s.t. \quad y^{(i)}(w^T x^{(i)} + b) \geq 1, \quad i = 1, \dots, m$$

This constraint is actually saying: “we need every example's functional margin ≥ 1 ”.

2) Now we're gonna loosen the restrictions :

$$\min \frac{1}{2} \|w\|^2 + c \sum_{i=1}^m \xi_i$$

$$s.t. \quad y^{(i)}(w^T x^{(i)} + b) \geq 1 - \xi_i, \quad i = 1, \dots, m$$

$$\xi_i \geq 0$$

As long as the functional margin ≥ 0 , we'd assume that this example is classified rightly.

Compared with SVM, SVM is asking for it to not just classify correctly, but classify correctly with the functional margin ≥ 1 .

' $\xi_i \geq 0$ ' is loosening the constraint (we allow some points to have functional margin ≤ 1), but we don't want ξ_i to be too large, so we add ξ_i in our minimizing goal.

Another reason we want to use ℓ_1 norm soft margin SVM is that if we just have one outlier, we don't want it to change the previous fine decision boundary.

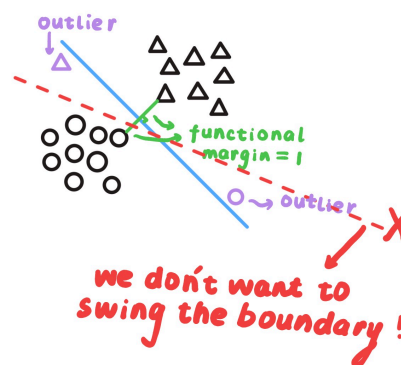


Figure 30: soft margin SVM boundary

That way the SVM is more robust outliers.

3) Then we're gonna go through derivation (represent w as a function of the α 's)

It turns out the problem then simplifies to the following:

$$\begin{aligned} \max \quad & \sum_{i=1}^m \alpha_i - \sum_{i=1}^m \sum_{j=1}^m y^{(i)} y^{(j)} \alpha_i \alpha_j < x^{(i)}, x^{(j)} > \\ \text{s.t.} \quad & \sum_{i=1}^m y^{(i)} \alpha_i = 0 \\ & 0 \leq \alpha_i \leq c, \quad i = 1, \dots, m \end{aligned}$$

(Dual form with the optimization problem)

Compared with our previously derived dual form, here the soft margin SVM just has an additional condition that $\alpha_i \leq c$

- 一些对于具有核函数方法的 SVM 的用途的举例：

e.g.¹ :

Protein sequence classifier:

蛋白质是氨基酸序列，如果将所有 21 种氨基酸各自编码，那么蛋白质就是一串编码序列。

这个时候我们应该思考，拿到一串编码序列作为输入后，如何将这个输入 x 投影至 $\Phi(x)$ ？我们应该构造怎样的特征空间？

$$\Phi(x) = ?$$

一种构造特征向量的方法是，列出所有 4 种氨基酸的组合：（假设氨基酸 26 种用 $A \sim Z$ 来编码表示）

$$\begin{bmatrix} A & A & A & A \\ A & A & A & B \\ A & A & A & C \\ \vdots & & & \\ Z & Z & Z & Z \end{bmatrix}$$

然后，根据这些序列在氨基酸中出现的次数来构建 $\Phi(x)$ （这里相当于是创新性构造出核函数）

$$\begin{bmatrix} A & A & A & A \\ A & A & A & B \\ A & A & A & C \\ \vdots & & & \\ Z & Z & Z & Z \end{bmatrix} \rightarrow \begin{bmatrix} \\ \\ \\ \\ \end{bmatrix} = \Phi(x)$$

$$\Phi(x) \in \mathbb{R}^{(21^4)}$$

Therefore, SVM allows us to invent kernel functions to measure the similarity.

（这里可以回顾一下 kernel trick 的实质：如果我们手动去构建那个 21^4 维的向量，然后计算这样两个向量 x, z 之间的相似度（此处相似度就是求内积），那么计算量会多到爆炸！核函数其实也就是给我提供了一种不需要显式计算这种高维向量内积的捷径）

在上述例子中，如果我们先手动去构造输入 x, z 的对应 21^4 维向量，再作点积，那么计算量过大。但如果换一种思路，我们直接去计数 x, z 里面的“相同 4 元序列”出现的次数， x, z 相同序列出现次数再取对应乘积和，那么问题就得到简化。

e.g:

设字母表 $\Sigma = \{A, B, C, D\}$ ，取 2-mer 片段（只考虑二元组合），特征空间维度 $D = 4^2 = 16$

输入序列：

• $X = \{ABABD\}$, $Y = \{ABACD\}$

按字典序排列 16 维特征空间：

$$\begin{bmatrix} AA \\ AB \\ AC \\ \vdots \\ DC \\ DD \end{bmatrix}$$

(i) 法一：显式特征映射（笨办法）

第一步：构建 16 维频次向量 $\Phi(x)$

$$\Phi(X) = (0, 2, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0)^T$$

$$\Phi(Y) = (0, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0)^T$$

第二步：计算向量内积 $\Phi(X)^T \Phi(Y)$

$$\text{点积结果} = 2 + 1 + 0 + 0 + 0 = 3$$

(2). 法二：使用核技巧

无需显示构造 16 维向量，直接计算交集片段的频次乘积和：

$$\begin{aligned} K(X, Y) &= \sum_m \text{count}_X(m) \times \text{count}_Y(m) \\ &= (2 \times 1)_{AB} + (1 \times 1)_{BA} + (1 \times 0)_{BD} + (0 \times 1)_{AC} + (0 \times 1)_{CD} \\ &= 2 + 1 + 0 + 0 + 0 \\ &= 3 \end{aligned}$$

结论：显式映射内积 $\Phi(X)^T \Phi(Y) = 3$ 与核函数 $K(X, Y) = 3$ 在数学上完全等价。核技巧将复杂度从高维特征空间 $O(|\Sigma|^k)$ 降至仅与实际出现的片段数相关 $O(n)$ 。

§ Lec VIII

Learning Algorithm Application

Outline:

- Bias / Variance
- Regularization
- Train / Dev / Test splits
- Model selection & Cross-validation

1. Bias & Variance

Given a dataset, we'd want to fit the data "just right".

(underfit(high *bias*) → just right → overfit(high *variance*))

high bias : the algorithm has very strong preconceptions that the data could be fit by the (linear) function

high variance : if we do the experiment again, we may end up with very different prediction

2. Regularization

1) linear regression:

Optimization Goal:

$$\min_{\theta} \frac{1}{2} \sum_{i=1}^m \|y^{(i)} - \theta^T x^{(i)}\|^2 + \lambda \|\theta\|^2$$

Here we add the red part as a **regularization term**.

Use this dataset as a simple example:



Figure 31: dataset

① If we set $\lambda = 0$, we are actually fitting a 5-degree polynomial curve, and that's overfitting:

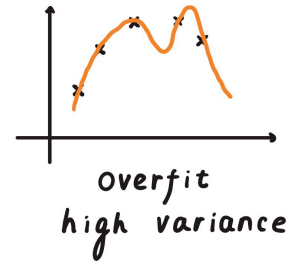


Figure 32: overfit

② If we set λ too large, we are actually underfitting the curve (when λ grows too big, $h_{\theta}(x) \approx 0$)

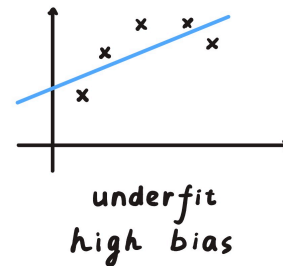


Figure 33: underfit

So with the penalty in the minimization goal, we'll end up preventing the parameters θ from being too big. And that will probably give out θ being "just right":

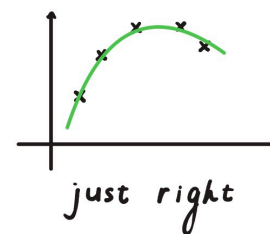


Figure 34: "just right"

2) In logistic regression, we have a cost function:

$$\arg \max_{\theta} \sum_{i=1}^n \log p(y^{(i)} | x^{(i)}; \theta) - \lambda \|\theta\|^2$$

Here this is a maximization goal, so we minus a regularization term.

An simple illustration is shown below:

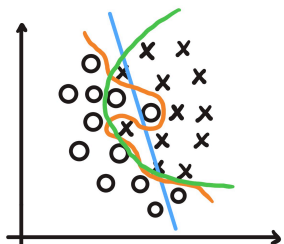


Figure 35: logistic regression

One rule of thumb for logistic regression:

If we don't use regularization, it's fine when the number of examples is at least on the order of the number of parameters we want to fit.

Q : Why SVM doesn't overfit even when it works in an infinite-dim space ?

A : As SVM's optimization goal is :

$$\min \|w\|^2$$

It turns out that this has a similar effect as $\min \lambda \|\theta\|^2$, so by forcing the parameters to be small is difficult for SVM to overfit the data too much.

3) Text Classification:

If we have just 100 training examples, but the feature vector $\in \mathbb{R}^{10000}$, it will probably overfit the data.

If we use a logistic regression with regularization, it'll probably be a good algorithm. (but we need gradient descent to solve local value parameters)

Q : Why don't we regularize per parameters ?

$$\lambda \|\theta\|^2 \quad \text{instead of} \quad \sum_j \lambda_j (\theta_j)^2 \quad ?$$

A : Because if we regularize per parameters, we'll end up with the λ 's. And we don't have good weights to choose those λ .

Note that we also pre-processing the features by scaling each dimension to the same interval $(-1, +1) / (0, 1)$. (normalization)

And it turns out this pre-processing will make gradient descent faster.

4) 上述正则化方法的数学由来以及 MAP

Given a training set S :

$$S = \{(x^{(i)}, y^{(i)})\}_{i=1}^m$$

we want to find the most likely value of θ

$$P(\theta | S) = \frac{P(S | \theta)P(\theta)}{P(S)}$$

So if we want to fit the most likely value of θ given the training data, we're doing this optimization: (note that $P(S)$ is a constant)

$$\arg \max_{\theta} P(\theta | S) = \arg \max_{\theta} P(S | \theta)P(\theta)$$

根据概率论，整个数据集的似然 $P(S | \theta)$ 可以写成所有样本的联合分布：

$$P(S | \theta) = P((x^{(1)}, y^{(1)}), \dots, (x^{(m)}, y^{(m)}) | \theta)$$

假设样本之间相互独立 (i.i.d.)，联合分布等于边缘分布的乘积：

$$P(S | \theta) = \prod_{i=1}^m P(x^{(i)}, y^{(i)} | \theta)$$

把这个式子再进一步用条件概率公式拆开

$$P(S | \theta) = \prod_{i=1}^m P(y^{(i)} | x^{(i)}, \theta) \cdot P(x^{(i)} | \theta)$$

由于逻辑回归（或者任何广义线性模型）是判别式模型，用来建模“在给定 x 的情况下， y 的条件概率”，即 $P(y | x, \theta)$ ，它并不关心 x 本身是怎么来的，所以输入数据 $x^{(i)}$ 的分布 $P(x^{(i)} | \theta)$ 是不依赖于模型参数 θ 的。

因此，

$$p(x^{(i)} | \theta) = p(x^{(i)})$$

这些都是常数，所以最后得：

$$P(S | \theta) \propto \prod_{i=1}^m P(y^{(i)} | x^{(i)}, \theta)$$

所以我们最后要优化的式子就简化为：

$$\arg \max_{\theta} \prod_{i=1}^m P(y^{(i)} | x^{(i)}, \theta) P(\theta)$$

如果假设 θ 满足多元高斯分布 ($\theta \in \mathbb{R}^n$)

$$\text{assume : } \theta \sim \mathcal{N}(0, \tau^2 I)$$

（注意上式协方差矩阵表明各个维度独立但是方差相等）写出概率密度函数，也即：

$$\begin{aligned} P(\theta) &= \frac{1}{(2\pi)^{\frac{n}{2}} |\tau^2 I|^{\frac{1}{2}}} \exp\left(-\frac{1}{2} \theta^T (\tau^2 I)^{-1} \theta\right) \\ &= \frac{1}{(2\pi)^{\frac{n}{2}} |\tau^2 I|^{\frac{1}{2}}} \exp\left(-\frac{1}{2\tau^2} \theta^T \theta\right) \end{aligned}$$

So this is the prior distribution for θ , and if we plug this prob distribution into the ultimate optimization goal, and take log to maximize it, then we have:

$$\begin{aligned} &\arg \max_{\theta} \log \left(\prod_{i=1}^m P(y^{(i)} | x^{(i)}, \theta) P(\theta) \right) \\ &= \arg \max_{\theta} \left[\sum_{i=1}^m \log(P(y^{(i)} | x^{(i)}, \theta)) + \log(P(\theta)) \right] \end{aligned}$$

上式记作 $\ell(\theta)$

代入之前我们假设而得到的：

$$\begin{aligned} \log(P(\theta)) &= \log \left[\frac{1}{(2\pi)^{\frac{n}{2}} |\tau^2 I|^{\frac{1}{2}}} \exp\left(-\frac{1}{2\tau^2} \theta^T \theta\right) \right] \\ &\propto -\frac{1}{2\tau^2} \theta^T \theta \end{aligned}$$

所以我们最终的优化目标就是：

$$\arg \max_{\theta} \left[\sum_{i=1}^m \log(P(y^{(i)} | x^{(i)}, \theta)) - \frac{1}{2\tau^2} \|\theta\|^2 \right]$$

注意前面红色这一块

$$\arg \max_{\theta} \left[\sum_{i=1}^m \log(P(y^{(i)} | x^{(i)}, \theta)) \right]$$

其实就是判别式模型（包括所有广义线性模型（如：线性回归、逻辑回归...））在做的最大似然估计（MLE），也就是我们之前没有正则化的时候的最后的最大化目标函数；

而我们加上了后面这一块：

$$\arg \max_{\theta} \left(-\frac{1}{2\tau^2} \|\theta\|^2 \right)$$

这正是我们之前对于 linear regression, logistic regression 采用的正则化方法！

所以小结一下：上面我们通过加减 $\lambda \|\theta\|^2$ 来正则化的方法的数学由来其实就是我们引入的这个关于参数 θ 的多元高斯分布假设：

$$\text{assume : } \theta \sim \mathcal{N}(0, \tau^2 I)$$

以上方法称为 MAP（最大后验估计），也就是在尽量拟合数据的同时，还要让参数 θ 符合我们预设的规律（由于 θ 满足高斯分布所以不会太大）；而我们之前一直采用的 MLE（最大似然估计）则是认为参数 θ 是一个未知但固定的值，只让模型在训练集上进行充分拟合。

MLE、MAP 两种方法其实涉及统计学两大学派之间的区别：

① **Frequentist:**

认为存在一个真实的 θ 使得我们当前这个数据最可能，我们要去估计这个值，所以使用 MLE

$$P(S | \theta) \rightarrow \text{MLE}$$

② **Bayesian:**

认为 θ 是未知的，但在看到任何数据之前，我们已经对数据集生成机制有先验信念，而这些先验信念被编码在概率分布中

$$P(\theta) \rightarrow \text{prior distribution}$$

在上述例子中，我们使用的是 Gaussian prior，这其实挺合理，首先世界上大多数事物都是高斯分布的，而且在我们不知道 θ 的情况下，我们把它的均值设为 0 也是正常的

$$\max_{\theta} P(\theta | S) \rightarrow \text{MAP maximum}$$

3. Train / Dev / Test splits

Generally, we can draw the error curve below.

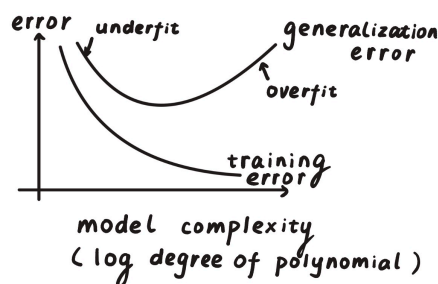


Figure 36: error curve

Now we're gonna introduce an mechanic algorithm to find the balanced point where both training error and generalization error are small enough.

Given a dataset, we basically splits it into several subsets : train, dev, test sets.

$$S \rightarrow S_{\text{train}}, S_{\text{dev}}, S_{\text{test}}$$

• The Whole Procedure

- ① Train a sequence of models (options for the degree of polynomial) on S_{train} , and get some hypothesis h_i
- ② Measure error on S_{dev} , and pick the one with lowest error on S_{dev}
- ③ Evaluate the algorithm on a separate test set S_{test} and report that error. (to publishing an unbiased test result)

We shouldn't use the test set to make any decision, because that won't be an unbiased report ! This can only be used for reporting or tracking performance.

- **Dataset Splits**

- **Historical rule of thumb**

$$S \rightarrow \begin{cases} \text{Train} : 70\% \\ \text{Test} : 30\% \end{cases}$$

or

$$S \rightarrow \begin{cases} \text{Train} : 60\% \\ \text{Dev} : 20\% \\ \text{Test} : 20\% \end{cases}$$

This rule applies to cases when we *don't have a massive dataset*.

- **Massive dataset**

When we have a massive dataset, the percentage of data we send to dev and test are shrinking !

If we're just measuring the small differences between algorithm a and b, we need large Dev and Test datasets.

But if we're just comparing different algorithms, than we don't need large Dev and Test datasets to distinguish the differences.

Note that what we do to Train and Dev datasets is called **hold-out cross validation**.

"development set" = "cross validation set"

- **Small datasets**

If we have a small dataset to be split for Train and Dev sets, but we don't want to waste any data to not being trained for our model.

Here is an algorithm used for this case:

K-fold cross-validation (k-fold CV)

e.g:

$$S = \{(x^{(i)}, y^{(i)})\}_{i=1}^{100}$$

We use $k = 5$ for illustration ($k = 10$ is typical).

So we divide our dataset into 5 different subsets (in this example each subset will have 20 examples).

Algorithm :

```
For  $i = 1, \dots, k$  : {
  Train: i.e fit parameters on  $k - 1$  pieces;
  then test on the remaining one piece
}
Average (errors from these  $k$  classifiers)
```

The if we try to find the degree of the polynomial, we just wrap the existing procedure with an outer loop :

For $d = 1, \dots, s$ (degree of polynomial)

So for each attempted degree, we repeat the whole procedure above. Then we compared the average errors from each degree of polynomial and find the best degree for the polynomial.

An optional final step (after we've decided which degree to use):

Refit the model on all 100% of the data

Compared to simple cross-validation, it makes more efficient use of data because we're holding out only 10% of the data on each iteration. But the disadvantage is that this is computationally very expensive. So it's applicable to small dataset.

There is an extreme version of k -fold CV :
Leave-one-out cross-validation

Which basically means that we're holding out just one example each time for test. (only applicable to very small dataset (maybe $m \leq 100$ (the number of examples)))

Q : In k-fold CV's average procedure, do we estimate the variance of those k estimates?
A : Actually we'd think these k estimates are highly correlated because they always have $\frac{k-2}{k-1}$ of same training data.

4. Feature Selection

Algorithm: **Forward Search**

Start with $\mathcal{F} = \emptyset$ (an empty set of features)
Repeat: {
 1) Try adding each feature i to $\mathcal{F}$, and see which single feature most improves the dev set performance.
 2) Add that feature to $\mathcal{F}$
}

That's basically keeping adding features greedily until when more features now hurts performance.

§ Lec IX

Learning Theory

outline:

- Setup / Assumptions
- Bias / Variance
- Approx / Estimate
- Empirical Risk Minimizer
- Uniform convergence
- VC dimension

1. Assumptions

1) There exists a data distribution D that:

$$(x, y) \sim D$$

This assumption must hold in both supervised learning and unsupervised learning.

And that means data in the training and test set should all come from the same distribution D .

2) All the examples are sampled *independently*.

When we take a closer look, when those samples (random variable) go through a deterministic function, that will get a random variable θ^* (h^*) as well.

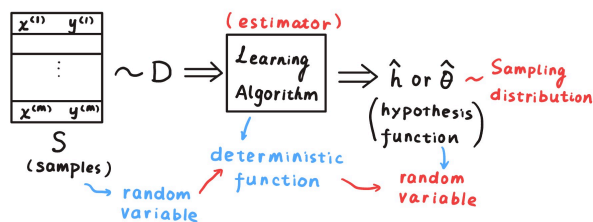


Figure 37: learning procedure

But in this process, we believe that there exists a “true parameter” θ^* (or h^*). Note that here θ^* (or h^*) is not a random variable, it's just an unknown constant.

2. Bias & Variance

In linear regression, we have this data view:

Data View :

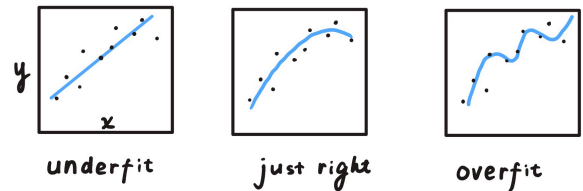


Figure 38: data view

Now we take a parameter view:

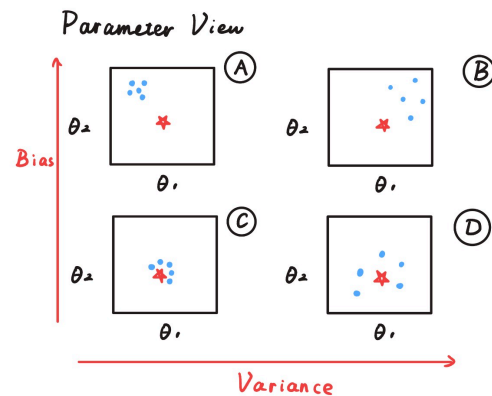


Figure 39: parameter view

Here we select four learning algorithms A,B,C,D. Each time the data is fed into an algorithm, and we obtain an estimate output $\hat{h}$ ($\hat{\theta}$) by that algorithm. Finally, we get multiple estimates of $\hat{h}$ ($\hat{\theta}$) from each of the four algorithms. (blue points in the figure above) And we pin the true parameter as the red point.

So the blue points are basically the samples from the distribution of $\hat{h}$ ($\hat{\theta}$), and the number of blue points is the time that we do the sampling.

And here it's easy to see the concept of bias&variance:

- ① Bias : whether the sampling distribution is centered around the true parameter
- ② Variance : the dispersity of the sampling distribution

In fact, Bias and Variance is the properties of 1st and 2nd moments of the sampling distribution.

As we increase the size of data, the variance of $\hat{\theta}$ would become small.

$$m \rightarrow \infty, \quad \text{Var}[\hat{\theta}] \rightarrow 0$$

The rate at which the variance tends to zero as $m \rightarrow \infty$ is called the statistical efficiency, and it measures the ability to extract information from the data.

• Consistent Algorithm:

$$\text{as } m \rightarrow \infty, \quad \hat{\theta} \rightarrow \theta^*$$

which means :

$$E[\hat{\theta}] = \theta^*, \quad \text{for all } m$$

For high-biased algorithms, no matter how much data we feed into the algorithm, the distribution of $\hat{h}$ would never center around θ^* .

For high-variance algorithms, they are highly easily distracted by the noise from the data.

As we can see, bias and variance are independent to each other. And bias and variance are properties of an algorithm given the fixed number of m .

3. Approx / Estimate

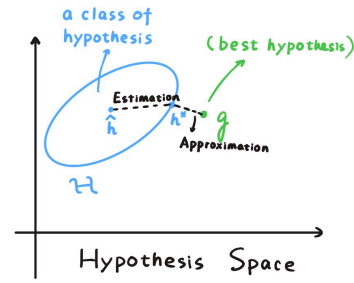


Figure 40: hypothesis space

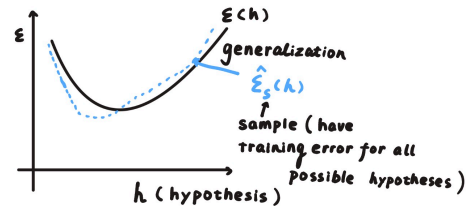


Figure 41: error plot

g — Best possible hypothesis

$\mathcal{H}$ — a class of hypothesis (e.g hypo of logistic regression)

h^* — best hypothesis in class $\mathcal{H}$ (learned from infinite data)

$\hat{h}$ — the hypothesis learned from current finite data

$\varepsilon(h)$ — risk / generalization error (an infinite process to measure model h 's error rate because D could be sampled infinitely)

$$\varepsilon(h) = E_{(x,y) \sim D}[1\{h(x) \neq y\}]$$

$\hat{\varepsilon}_S(h)$ — empirical risk (a finite process, model h 's error on current data sample of size m)

$$\hat{\varepsilon}_S(h) = \frac{1}{m} \sum_{i=1}^m 1\{h(x^{(i)} \neq y^{(i)})\}$$

$\varepsilon(g)$ — Bayes error / **Irreducible error**: if we take the best possible hypothesis, the rate that we make errors

$\varepsilon(h^*) - \varepsilon(g)$ — **Approximation error**: the price we have to pay for limiting ourselves in a certain class $\mathcal{H}$

$\varepsilon(\hat{h}) - \varepsilon(h^*)$ — **Estimation error**

Thus, we have

$$\varepsilon(\hat{h}) = \text{Estimation error} + \text{Approximation error} \\ + \text{Irreducible error}$$

Let's try to understand this equation:

First, there's Bayes error that we cannot reduce by infinite data or algorithm;

Second, we'll choose our frame of learning algorithm (the class of all possible models) and that'll give approximation error;

Finally, we only have finite data, which leads to Estimation error.

Then we break up this equation further:

$$\varepsilon(\hat{h}) = \text{Estimation error} + \text{Approximation error} \\ + \text{Irreducible error} \\ = (\text{Estimation Variance} + \text{Estimation Bias}) \\ + \text{Approximation error} + \text{Irreducible error} \\ = \text{Variance} + \text{Bias} + \text{Irreducible error}$$

Bias basically captures why is $\hat{h}$ far away from g .

Note : What's $\varepsilon(\hat{h})$?

$\hat{h}$ is the hypothesis we've learned from the current finite data, so $\varepsilon(\hat{h})$ is $\hat{h}$'s error rate on D. (generalization error of $\hat{h}$)

• Fight High Variance

1) Increase m

2) Regularization

(It can reduce variance, however, it may lead to the increase of bias)

• Fight High Bias

1) Make $\mathcal{H}$ bigger.

(Regularization actually shrinks $\mathcal{H}$, which commonly pays the price of bias to reduce variance. Bigger $\mathcal{H}$ could actually make variance bigger)

4. Empirical Risk Minimizer (ERM)

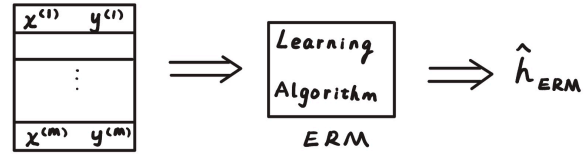


Figure 42: ERM

In ERM, we take the naive method to minimize the empirical risk of our learned hypothesis $\hat{h}$, which is, reduce error on the current finite training set, *i.e.*, $\min \varepsilon(\hat{h})$ to derive $\hat{h}$ from $\mathcal{H}$.

$$\hat{h}_{\text{ERM}} = \arg \min_{h \in \mathcal{H}} \frac{1}{m} \sum_{i=1}^m 1\{h(x^{(i)}) \neq y^{(i)}\}$$

If we limit ourselves to using ERM, we could come up with many other results.

5. Uniform Convergence

1) Two central questions :

① In ERM, we try to minimize the training loss (error on the finite dataset), then what effect does it have on the generalization error ?

Basically, what's the relationship between $\varepsilon(\hat{h})$ and $\varepsilon(h)$?

② Compare $\varepsilon(\hat{h})$ with $\varepsilon(h^*)$?

2) Tools:

① Union bound :

If we have k different events: $A_1, \dots, A_k$ (they need not be independent), then:

$$P(A_1 \cup A_2 \cup \dots \cup A_k) \leq P(A_1) + \dots + P(A_k)$$

② Hoeffding's inequality:

Let $Z_1, Z_2, \dots, Z_m \sim \text{Bernoulli}(\varphi)$,

$$\hat{\varphi} = \frac{1}{m} \sum_{i=1}^m Z_i$$

Let $\gamma > 0$ (margin), then:

$$P(|\hat{\varphi} - \varphi| > \gamma) \leq 2 \exp(-2\gamma^2 m)$$

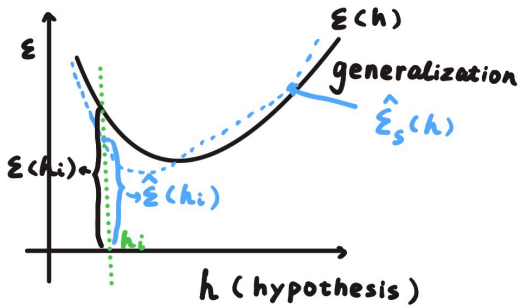


Figure 43: $\varepsilon(\hat{h}), \varepsilon(h)$

3) question 1: $\varepsilon(\hat{h})$ with $\varepsilon(h)$

Let's start with some hypothesis h_i , then we get $\varepsilon(h_i), \hat{\varepsilon}(h_i)$ from the curve plot. 由于:

$$E[\hat{\varepsilon}(h_i)] = \varepsilon(h_i)$$

where the expectation regards to the data sample. 由于

$$Z_j = 1\{h_i(x^{(j)}) \neq y^{(j)}\}$$

是一个伯努利变量, 并且

$$\varphi = E_{x^{(j)}, y^{(j)} \in D} [1\{h_i(x^{(j)}) \neq y^{(j)}\}] = \varepsilon(h_i)$$

so we apply the Hoeffding's inequality, then we have:

$$P\left(\left|\frac{1}{m} \sum_{j=1}^m Z_j - \varepsilon(h_i)\right| > \gamma\right) \leq 2 \exp(-2\gamma^2 m)$$

$$P(|\hat{\varepsilon}(h_i) - \varepsilon(h_i)| > \gamma) \leq 2 \exp(-2\gamma^2 m)$$

So this expression means that if we increase the size m , then $\hat{\varepsilon}(h_i)$ will be more centered around $\varepsilon(h_i)$. (在这里我们推出: 对于一个给定的 h_i , 它的 generalization error 与 empirical error 确实是紧密接近的, 所以我们用 ERM 最小化 empirical error 确实可以最小化 generalization error)

However, this method has some logical flaw because we start with some hypothesis h_i and

take the average of all the possible data that we could sample in the above. But practically, we actually start with some data, and run the ERM to find the best h for that particular data. That means h and the data we have are not really independent. (实际情况与上述我们推导的逻辑相反, 我们在学习时不是先固定模型 (也就是假设) 再去数据上拟合, 而是从抽样的数据出发训练得到在这个数据上表现最好的模型)

To fix this logical flaw, we're gonna extend this result to all h . And this is basically called **uniform convergence** as we'll see generally how the risk curve converges uniformly to the generalization risk curve. (不能只证明 “对于一个固定的 h 它的 generalization error 与 empirical risk 接近”, 而是要推广证明: $\mathcal{H}$ 里面所有的假设 h , 它们的 generalization error 与 empirical risk 都同时很接近! 这就是一致收敛!)

case 1) : Finite hypothesis classes

Assume: class $\mathcal{H}$ has a finite number of classes:

$$|\mathcal{H}| = k$$

then we get:

$$P(\exists h \in \mathcal{H} \text{ s.t. } |\hat{\varepsilon}_S(h) - \varepsilon(h)| > \gamma) \leq k \cdot 2 \exp(-2\gamma^2 m)$$

then we flip it over: (所有 k 个 h 都为稳定接近的概率)

$$P(\forall h \in \mathcal{H} \text{ s.t. } |\hat{\varepsilon}_S(h) - \varepsilon(h)| \leq \gamma) > 1 - k \cdot 2 \exp(-2\gamma^2 m)$$

$$\text{Let } \delta = k \cdot 2 \exp(-2\gamma^2 m)$$

δ — prob of error (the difference is bigger than some margin γ)

γ — margin of error

m — sample size

For the three variables above, we can actually fix the two of them and solve for the third.

§ Lec X

Decision Tree & Emsemble methods

outline:

- Decision tree
- (general) emsemble methods
- Bagging
- Random forests
- Boosting

1. Decision Tree

Previous lectures have covered linear models, and now we'll go into our first non-linear model — decision tree.

If we have a dataset like below, it's hard to find a linear decision boundary:

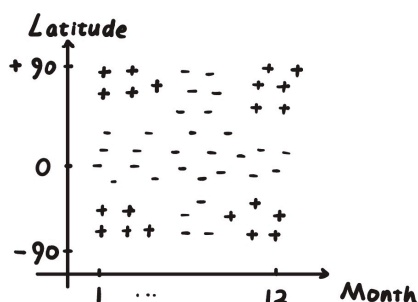


Figure 45: dataset 1

What decision tree does is **greedy, top-down(from root to leaf), recursive partitioning of the dataset space**.

For the dataset above, decision tree might decide like this:

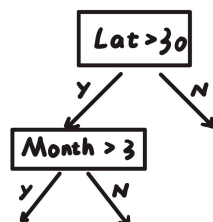


Figure 46: dataset 1 decision process

Then the dataset can have splits :

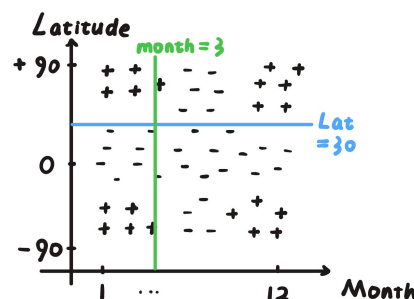


Figure 47: dataset 1 split

If we have a parent region R_p , and we're looking for a split function S_p :

$$S_p(j, t)$$

where j is the feature number and t is the threshold being used.

$$S_p(j, t) = (\{X | X_j < t, X \in R_p\}, \{X | X_j > t, X \in R_p\})$$

The output of this split function is a tuple that contains two subregions and we called R_1, R_2

• How to choose splits ?

① Define loss on a region R : $L(R)$

Given C classes total, define $\hat{P}_c$ to be the proportion of examples in R that are of class c .

Then we can define loss as 'misclassification':

$$L_{\text{misclass}} = 1 - \max_c \hat{P}_c$$

(because we choose a greedy split each time.
(也就是少数服从多数的原则，叶节点区域最后的分类就是里面最多数节点所属的类别))

② Therefore we want to choose a split that maximize this loss difference :

$$\max_{j,t} L(R_p) - \left(\frac{|R_1|}{|R_p|} L(R_1) + \frac{|R_2|}{|R_p|} L(R_2) \right)$$

$L(R_p)$ is the parent loss, and the latter term is children loss (weighted sum). Since parent loss matters nothing here because it's already defined, so basically we're trying to

$$\max_{j,t} - \left(\frac{|R_1|}{|R_p|} L(R_1) + \frac{|R_2|}{|R_p|} L(R_2) \right)$$

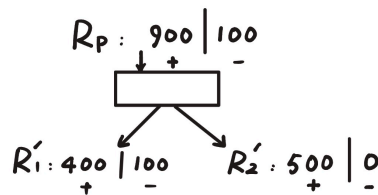


Figure 50: dataset 2 split ②

Split ② is better than split ① because we intuitively split more positive examples out.

However, if we only look at misclassification loss, these two are the same :

$$\frac{|R_1|}{|R_p|} L(R_1) + \frac{|R_2|}{|R_p|} L(R_2) = \frac{800}{1000} \times \frac{100}{800} + 0 = 0.1$$

$$\frac{|R'_1|}{|R_p|} L(R'_1) + \frac{|R'_2|}{|R_p|} L(R'_2) = \frac{500}{1000} \times \frac{100}{500} = 0.1$$

which shows that we can't differentiate the performance by just comparing their misclassification loss.

• Misclassification loss has issues

Let's introduce a simple example to discuss why misclassification is not suitable for this case:

Assume we have this dataset of 900 positive examples + 100 negative examples



Figure 48: dataset 2

After the first split, as misclassification loss forces to classify the 900 right, and to classify the other 100 wrong. Assume the first split leads to this: (手绘图 2)

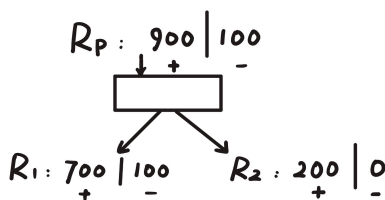


Figure 49: dataset 2 split ①

Let's compare it to another split:

• Instead, we can define this cross-entropy loss:

$$L_{\text{cross}} = - \sum_c \hat{P}_c \log_2 \hat{P}_c$$

(concept borrowed from the information theory: the number of bits we need to tell an example what class it belongs to)

For cross-entropy loss, we can draw this strictly-concave curve:



Figure 51: cross-entropy loss curve1

Assume we have two children regions R_1, R_2 on the curve :

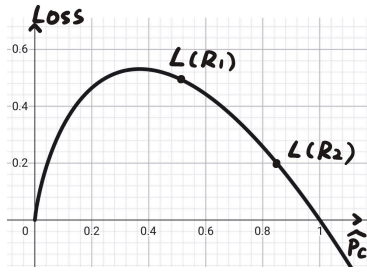


Figure 52: cross-entropy loss curve2

If the number of examples in R_1, R_2 is the same, then the change in loss equals to :

$$\max_{j,t} L(R_p) - \frac{L(R_1) + L(R_2)}{2}$$

We take the midpoint of R_1, R_2 on the curve plot:

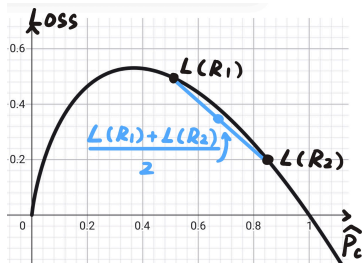


Figure 53: cross-entropy loss curve midpoint

And since *there's the same number of example in R_1, R_2* , we have :

$$\hat{p}_c(R_p) = \frac{\hat{p}_c(R_1) + \hat{p}_c(R_2)}{2}$$

Note : We can verify this property through the simple split example above :

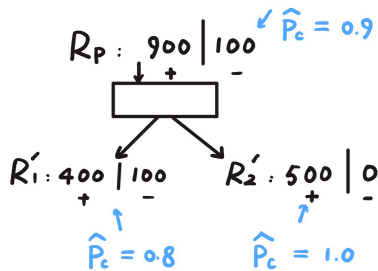


Figure 54: cross-entropy loss midpoint verification

Therefore, we can easily find the change in loss on the curve :

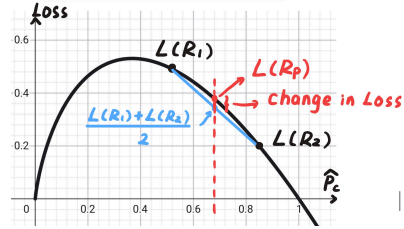


Figure 55: loss change on cross-entropy loss curve

Just because the loss function is strictly concave, so whether or not the number of examples in R_1, R_2 is evenly divided, we always have

$$L(R_p) - \frac{L(R_1) + L(R_2)}{2} > 0$$

That's how we get a reduction in loss.

- Now we can recall the **loss function of misclassification** :

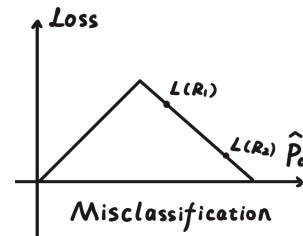


Figure 56: misclassification loss curve

In this case we can't get any reduction in loss !

- Furthermore, we could use **Gini loss** :

$$L_{\text{Gini}} = \sum_c \hat{p}_c(1 - \hat{p}_c)$$



Figure 57: misclassification loss curve

This is also strictly convex curve like the cross-entropy loss, so similarly we are guaranteed to obtain a loss reduction.

$$L_{\text{squared}} = \frac{\sum_{i \in R_m} (y_i - \hat{y}_m)^2}{|R_m|}$$

(Actually, under squared error loss, the optimal constant prediction is the mean value.)

• Decision Tree for Regression : Regerssion Tree

In the former decision tree, we adopt the “majority vote” method that classify the leave region to the class the same as the major class in this region.

Now assume we have a dataset like this, and we do the splits as well:

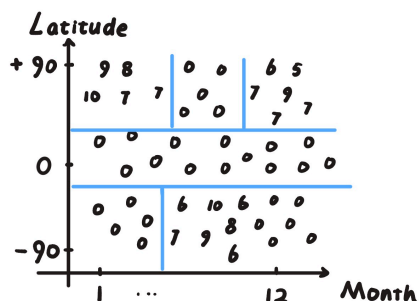


Figure 58: regression tree dataset split

In regression task, we have to predict continuous values. So what we do when we get to one of our leaves(the smallest subregion) is that instead of just predicting a majority class, **we predict the mean of the values left.**

For region R_m , we predict the average mean value :

$$\hat{y}_m = \frac{\sum_{i \in R_m} y_i}{|R_m|}$$

Then the loss function we use is:

• Decision Tree for Categorical variables

Regression tree can also work for catefory variables. It'll attempt to split the category set into 2 subsets. So if there are Q categories, there are $(2^{Q-1} - 1)$ possible splits.

在实践中，对于分类任务（使用基尼或交叉熵），可以证明：最优的二分方式一定可以通过按正例比例（或者回归任务中的均值大小）排序后，在相邻位置之间切分得到。这样的话，我们只需要先排序，在进行一遍线性搜索便可以找到最佳的二分法。

• Regularization

If we allow our decision tree to grow without ever stopping, it'd end up assigning each data point a subregion, which leads to overfitting.

Due to its overfitting, we can see that decision tree is a sort of high-variance models because it's sensitive to noise in data. Now we should regularize these high-variance DTs.

heuristic methods:

- ① Stop splitting a certain leaf when its size hits a threshold. (have a minimum leaf size)
- ② Enforce a maximum depth of the tree.
- ③ Enforce a maximum number of nodes.
- ④ Enforce a minimum decrease in loss, abandon not obvious splits. (usually not good)
- ⑤ Let the decision tree grow fully first, then do **post-pruning**.

(In some cases, we find that a combination of several 'questions' can greatly improve the decision tree(not just a single question, questions may correlate), so enforcing a growing size could be risky in this sense. So we first let the tree grow fully, then in pruning, we take a validation set and evaluate the misclassification rate on the validation set for each leaf we might remove.)

Post-pruning : 自底向上考虑剪枝：对于每一个内部节点，考虑是否把它下面的整个子树剪掉，变成一个叶节点。

用验证集评估：对于每个候选的剪枝节点，计算剪枝后在验证集上的误分类率（或损失）。如果剪枝后验证误差不升反降，就剪掉

• Runtime

Assume : n examples, f features, d depth

At test time, run time is $O(d)$ (这与树的深度相等，因为我们在树的每一层只会判断一次), and basically, we have $d < \log_2 n$ (最优情况下如果决策树在每一层左右分支的数据量都差不多的话，那么决策树会是一棵平衡二叉树，此时深度最小)

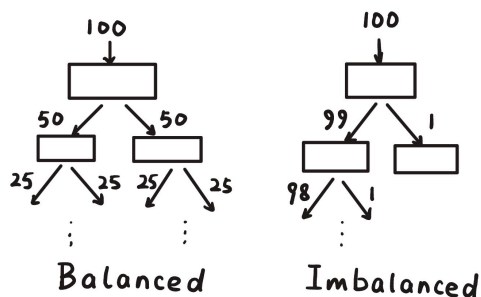


Figure 59: decision tree depth (balanced vs. imbalanced)

At train time, each point is part of $O(d)$ nodes. (在训练构建树的过程中，每一个样本都会从根节点一路被划分到某个叶子节点，所以每一层会出现一次，即 d 次) And cost of point at each node is proportional to the features —

$O(f)$. (决策树在每一层决定“当前节点到底用哪个特征来切分”时，需要遍历所有特征（共 f 个）) So the total cost is $O(nfd)$. Note that we have the data matrix $\in R^{n \times f}$, so the cost of train time is quite low.

• Downsides of Decision Trees

① No additive structure :

Assume we have a dataset like this (a binary classification task):

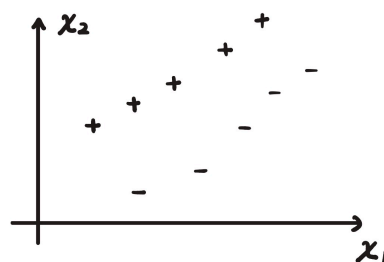


Figure 60: dataset 3

If we use linear or logistic regression, they can easily give decision boundary like the black dashed line. However, using decision tree, it'd end up with those complex blue lines for splits and that's just rough approximation.

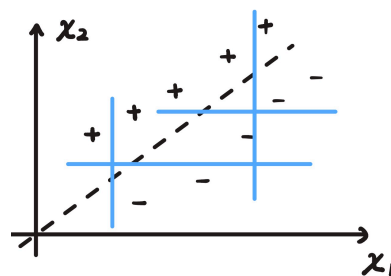


Figure 61: no additive

So we intuitively understand that the decision tree is hard to handle cases where the features are interacting additively with each other. (事实上就是因为决策树的每一次分裂决策边界只能平行于坐标轴，而无法直接画出一条斜线，所以不能像线性回归一样画出斜线 $\theta^T x$)

• Recap of decision tree

Upsides:

- ① Easy to explain
- ② Interpretable (不是黑箱)
- ③ Able to deal with categorical variables (不需要对分类变量做独热编码就能直接处理)
- ④ Fast

Downsides:

- ① High variance (对数据噪声敏感, 且容易过拟合)
- ② Bad at additive cases
- ③ Because of the first two downsides, they generally have fairly low predictive accuracy.

However, we can improve decision trees a lot through ensembling.

2. Ensembling

1) 先来看核心要用得到的数学知识:

Take X_i 's, which are random variables, that are independent and identically distributed (*i.i.d.*)

Assume $\text{Var}[X_i] = \sigma^2$, we can have:

$$\text{Var}[\bar{x}] = \text{Var}\left[\frac{1}{n} \sum_i X_i\right] = \frac{1}{n} \sigma^2$$

If we drop the independent assumption, now X_i 's are just *i.d.*

Suggest X_i 's are correlated by ρ , now:

$$\text{Var}[\bar{x}] = \rho \sigma^2 + \frac{1-\rho}{n} \sigma^2$$

(we can intuitively understand it when $\rho = 0$ or $\rho = 1$)

2) 现在我们转换至决策树的视角:

X_i = 第 i 个模型在某个测试点上的预测值

n = 集成中的模型数量

$\bar{X}$ = 集成后的预测 (n 个模型预测的平均)

ρ = 模型之间预测的相关系数

$\text{Var}[\bar{X}]$ = 集成后预测的方差

上一讲中我们推导过:

总误差 = Bias + Variance + Irreducible error

而决策树是一个 高 Variance 低 Bias 的模型, 所以使用集成方法我们目标主要在于降低 Variance.

根据上面的数学公式, 一方面要运用很多种模型, 让 n (训练得到的模型数量) 变大; 另一个就是 X_i 之间要去相关化这样让 ρ 变小, 也就是要降低模型之间的相关性, 这样才能降低最终的方差。

• Ways to Ensemble

1) Use different algorithms

(we can take multiple algorithms like neural networks, random forest, SVM ... and take the average of them, but it's not a time-efficient method)

2) Use different training sets

(collecting new data will cost a lot)

3) **Bagging** (random forests : bagging variant for decision trees)

4) **Boosting** (Adaboost, xgboost)

• Bagging — Bootstrap Aggregation

Bootstrap: A method used in statistics to measure the uncertainty of the estimate

Assume :

Have a true population P , so the training set S is sampled from P . ($S \sim P$)

Ideally, we just draw sets $S_1, S_2, \dots$ and train the model separately on these different sets. But we don't have the time to do that.

What Bootstrapping does is that we **assume** $S = P$. So we can generate new samples from S ! We can do sampling N times from S **with replacement** to get :

Bootstrap samples Z from S

Or we can do it in a mathematical way to say:

$$Z \sim \hat{P}_S$$

$\hat{P}_S$ is the empirical distribution defined by S

Then we can take the models and train on the different bootstrap samples. Finally we look at the variability in the predictions that the models end up making based on these different bootstrap samples and that'll give a measure of uncertainty.

• Bagging procedure:

Suggest we have bootstrap samples: $Z_1, \dots, Z_M$
And we'll train model G_m on Z_m . Then we define a meta model :

$$G_{\text{bag}}(x) = \frac{\sum_{m=1}^M G_m(x)}{M} \quad \text{random forests = Bagging + Decision Tree + additional randomization}$$

So the whole procedure is : take bootstrap samples, train separate models on the samples, and aggregate them all together.

• Why does this work ? (Bias-Variance Analysis)

Recall this formula from earlier :

$$\begin{aligned} \text{Var}[\bar{x}] &= \rho\sigma^2 + \frac{1-\rho}{n}\sigma^2 \quad (\text{here } n = M) \\ &= \rho\sigma^2 + \frac{1-\rho}{M}\sigma^2 \end{aligned}$$

So what bootstrapping does is driving down ρ (Bootstrap 抽样让每个模型看到不同的训练集, 从而使模型之间的预测去相关), and when we take many bootstrap samples, we're increasing M to bring down the variance.

Another advantage is that higher M only decrease the variance, so it'll improve the performance without leading to overfitting. (过拟合一般是因为方差过大)

However, one problem is that when we use bootstrapping, we're potentially increasing the bias of our models. That's because of random subsampling. (训练数据越少, 模型的偏差通常越大) (However, this is not significant compared with the gains obtained from training.)

• Decision Trees + Bagging — (random forest)

Recall that decision trees are of high variance and of low bias. That makes them ideal fits for Bagging !

Random forest is sort of a version of decision trees + Bagging. And random forest actually introduces more randomization into each individual decision tree.

Random Forests 在两个方面的随机化 :

- ① 样本随机化 (就是 Bagging 本身具有的)
- ② 特征随机化 (特有的): For each split of the random forest, we only consider a fraction of the total features. (每次分裂时我们只在一个小的特征子集里面来考虑最优的那个分类特征)

(Note that this is for decreasing ρ , for decorrelating the models. So we can decrease variance significantly. 因为如果数据中有一个非常强的特征, 那么所有树可能会在很多层都选用这个特征来分类, 导致所有树的结构相似, ρ 仍然很大)

• Boosting

我们可以来对比一下两种做法的思路区别：

① Bagging : 并行训练多个独立模型，取平均。主要降 Bias
② Boosting : 串行训练多个基模型，每个新模型都试图修正前面模型的错误。主要降 Variance

Boosting 的 Additive 本质 : In boosting, we'll train one model and add that prediction into the ensemble. (Boosting 最后的模型是之前多个基模型的加权和)

An illustration example :

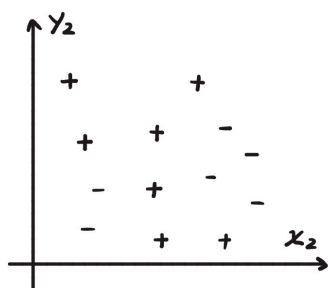


Figure 62: dataset 4

Say we have a **size 1 decision tree (decision stumps)**. By limiting depth to 1, we're actually decreasing the variance while allow for high bias. That makes them suitable for boosting method.

Assume we've got this decision boundary:

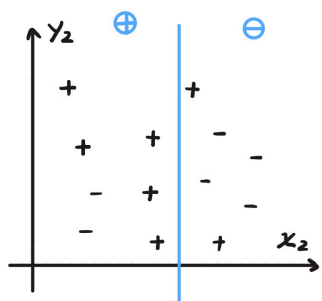


Figure 63: dataset4 : boosting decision boundary1

Then we'll identify the mistakes we've made in the graph :

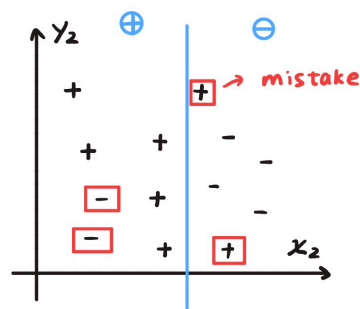


Figure 64: dataset4 : boosting decision boundary2 with identified mistakes

What boosting actually does is to **increase the weights of these misclassified examples**. And for the next decision stump to be trained, we'll train it on this modified training set. And that'll probably give the new green line :

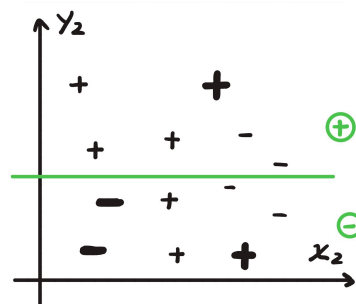


Figure 65: dataset4 : boosting decision boundary 2

So we do this step recursively, keep reweighting the misclassified examples to give out a new boundary.

We determine that for classifier G_m , a weight α_m is assigned to it. Better classifier will be assigned more weight, and the weight is proportional to how many examples the classifier get wrong or right.

e.g: In Adaboost:

$$\alpha_m = \log\left(\frac{1 - \text{error}_m}{\text{error}_m}\right)$$

Finally, the total classifier is :

$$G(x) = \sum_m \alpha_m G_m$$

Each G_m is trained on a reweighted training set.

由于每一个分类器相当于是一个阶跃函数，所以多个阶跃函数的线性组合最后会得到非线性的决策边界。边界应当是一条阶梯状曲线。

§ Lec XI

Neural Networks

Deep Learning Lec 1

Reasons for DL's success :

- new computational methods
- available data
- algorithms

• Logistic Regression

1) e.g¹

Classification goal 1 : Find cats in the image :

$$\begin{cases} 0 & \text{absence} \\ 1 & \text{presence} \end{cases}$$

Given a image, we'll flatten it to a vector (by RGB channels), then we take this vector and push it into the operation $wx + b$. And then we apply the sigmoid function to it. (也就是说, 逻辑回归可以看作一个只有一个神经元的神经网络!)

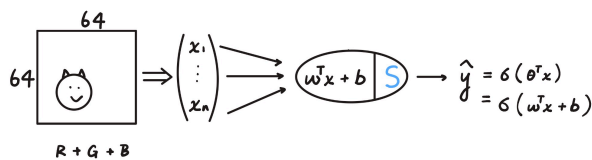


Figure 66: neural network e.g¹

For the image of pixels $64 \times 64 \times 3$ (3 is the RGB channels) :

$$x \in \mathbb{R}^{12288 \times 1}, w \in \mathbb{R}^{1 \times 12288}$$

For the training process :

- ① Initialize the parameters w, b (weights & biases)
- ② Find the optimal $w, b \rightarrow$ we have to define the loss function (from MLE method !)

$$\text{Loss} = -y \log \hat{y} + (1 - y) \log(1 - \hat{y})$$

We'll use gradient descend to find the optimal solution.

- ③ Use $\hat{y} = \sigma(wx + b)$

In this simple example, the parameters we need are :

$$12288 \text{ weights} + 1 \text{ bias}$$

So the size of parameters depends on the size of input.

In this binary classification task, we need dataset of $\{0, 1\}$.

• ▶ Two important equations:

①

$$\text{neuron} = \text{linear} + \text{activation}$$

In the example above:

$$\text{neuron} = "(wx + b)" + \text{Sigmoid}$$

②

$$\text{model} = \text{architecture} + \text{parameters}$$

2) $e.g^2$

Classification goal 2 : Find cats, lions, iguanas in the image

Here we have 3 classes, one to implement this is to split 3 sets of weights and biases to give out 3 sets of outputs.

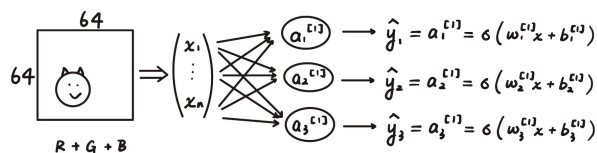


Figure 67: neural network $e.g^2$

Note that [1] represents “layer” (neurons in the same layer won’t communicate) ; index 1 represents the neuron’s index inside a certain layer.

As we have $\hat{y}_1, \hat{y}_2, \hat{y}_3$, thus the output will be a 3-dimensional vector.

Correspondingly, we need dataset like :

$$\begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$$

with each position representing different animals.

And the neurons’ responsibilities will evolve according to how we label our dataset. (比如说标注数据集的时候第二维表示狮子存在与否，那么相应训练出来的第二个神经元就会负责识别图片里面的狮子)

Robustness : 以上的这种做法具有鲁棒性因为这一层三个神经元之间没有相互交流，所以我们可以独立地训练这三个神经元。标注数据中比如说代表狮子的存在与否如果从第二维挪到第三维，那么相应的神经元也将第二个变为第三个（也就是第三个神经元会潜在地自动承担起识别狮子的任务!）

3) $e.g^3$

Classification goal 3: We add a constraint that there’s only one kind of animal in the image, and we want to classify that.

Note that here we introduce a notation $Z_1^{[1]}$ for the linear part of the first neuron; and $Z_2^{[1]}$, $Z_3^{[1]}$ for the second and third neurons.

So now there’re two parts of a neuron : compute Z , and then compute $\sigma(Z)$

Now for this example, we’re gonna remove all the activation function and only compute Z ’s first.

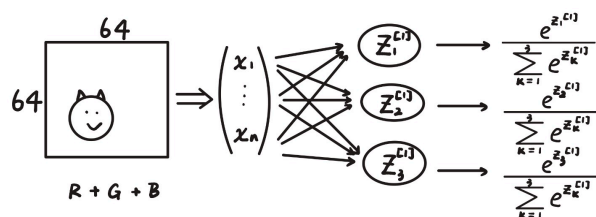


Figure 68: softmax classification neural network

As the sum = 1, the three output probabilities are dependent to each other. So we take the one-hot labeling for our data :

$$\begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

And this is called **softmax multi-class network**.

Note that here we shouldn’t adopt the simple loss function $\text{Loss} = -y \log \hat{y} + (1 - y) \log(1 - \hat{y})$ like before, 因为 Softmax 强制三个输出之和为 1，但如果对每个输出独立使用二元交叉熵，损失函数会鼓励每个输出独立地接近目标，但归一化约束使它们无法同时独立满足，导致梯度冲突。所以我们应该换一个损失函数！

如果我们尝试使用这种 loss function:

$$\text{Loss}_{3N} = - \sum_{k=1}^3 [y_k \log \hat{y}_k + (1 - y_k) \log(1 - \hat{y}_k)]$$

假如我们现在对第二个神经元的权重 $w^{[2]}$ 求导, 结果会非常复杂, 因为每个 $\hat{y}_k$ 都依赖于所有 Z_j , 而每个 Z_j 又对应自己的一套参数。所以对 $w^{[2]}$ 求导时, 不仅第二项有贡献, 第一项和第三项也会通过分母中的 e^{Z_2} 产生影响。链式法则展开会牵扯到所有参数, 导致梯度表达式极其冗长。

So what we use in Softmax regression is the **Softmax cross-entropy loss function** :

$$\text{Loss} = - \sum_{k=1}^m y_k \log \hat{y}_k$$

m is the number of classes

Furthermore, if we want to modify the current neural network to predict the age of the cat in the image, we can just change the sigmoid function in the network. (因为 sigmoid 函数把输出限制在(0,1)之间, 但我们要输出的是年龄而不是概率)

We could just replace sigmoid with a linear function, or the **ReLU** function (rectified linear units) :

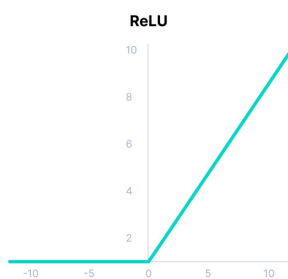


Figure 69: ReLU activation

Also we will change the loss function to suit this regression task. (maybe $\|y - \hat{y}\|^2$ ($\mathcal{L}_2$ norm)) And the loss function of regression is easier to optimize than the loss function of the classification (softmax...).

• Neural Networks

• ▶ For this neural network below :

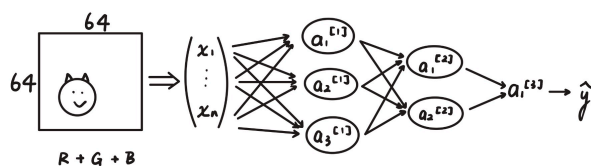


Figure 70: simple neural network architecture

We know that the number of the outputs of the neural network should correspond to the classification task. (one output — regression; multiple outputs — classification)

Now let's consider the number of parameters in this neural network :

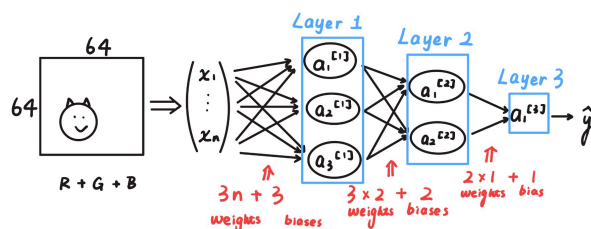


Figure 71: parameters in the neural network

And we define some vocabulary here:

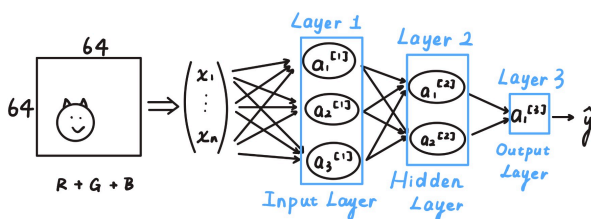


Figure 72: 3 layers of the neural network

“Hidden layer” means that the input and output are all hidden from this layer. And we don't know what they try to figure out from the image.

What's interesting about the neural networks is that we'll find that the fundamental function of

the first several layers is detecting some edges. Then hidden layers receive the abstract of the input and they'll detect like ears, mouth of the cat. And the last layers will construct a face and decide whether it is a cat.

- Another example : house price prediction

If letting human construct a neural network, we may construct the information stream according to human knowledge like this :

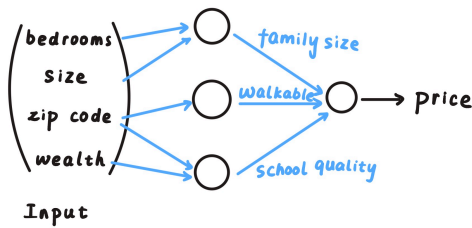


Figure 73: neural network with precedent knowledge

However, we always allow for a full connection between the neighboring layers, so that'll be a blackbox model, and it's called end-to-end learning. (we don't constraint networkd in the middle)

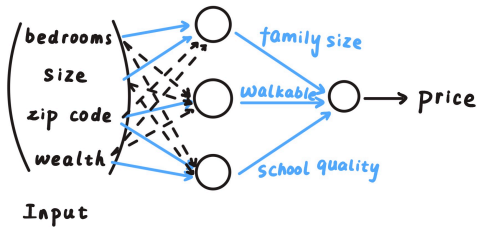


Figure 74: black-box neural network

• Propagation Equation

We try to write the propagation equation from the input layer to the output layer. (here it's a 3-layer neural network)

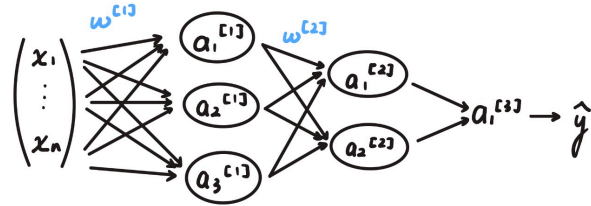


Figure 75: simple neural network architecture

$$Z^{[1]} = w^{[1]}x + b^{[1]}$$

$$a^{[1]} = \sigma(Z^{[1]})$$

$$Z^{[2]} = w^{[2]}a^{[1]} + b^{[2]}$$

$$a^{[2]} = \sigma(Z^{[2]})$$

$$Z^{[3]} = w^{[3]}a^{[2]} + b^{[3]}$$

$$a^{[3]} = \sigma(Z^{[3]})$$

Then we can derive all the shapes of the parameters and the intermediate variables.

$$Z^{[1]} \in \mathbb{R}^{3 \times 1} \rightarrow w^{[1]} \in \mathbb{R}^{3 \times n} \rightarrow b^{[1]} \in \mathbb{R}^{3 \times 1} \rightarrow a^{[1]} \in \mathbb{R}^{3 \times 1}$$

$$Z^{[2]} \in \mathbb{R}^{2 \times 1} \rightarrow w^{[2]} \in \mathbb{R}^{2 \times 3} \rightarrow b^{[2]} \in \mathbb{R}^{2 \times 1} \rightarrow a^{[2]} \in \mathbb{R}^{2 \times 1}$$

$$Z^{[3]} \in \mathbb{R}^{1 \times 1} \rightarrow w^{[3]} \in \mathbb{R}^{1 \times 2} \rightarrow b^{[3]} \in \mathbb{R}^{1 \times 1} \rightarrow a^{[3]} \in \mathbb{R}^{1 \times 1}$$

So we can intuitively see that the dimension of Z corresponds to the number of neurons in that layer, and the size of w corresponds to the number of edges connecting two layers.

• A batch of m examples

Now the shape of the input is:

$$X = \begin{pmatrix} x^{(1)} & x^{(2)} & \dots & x^{(m)} \end{pmatrix}$$

Figure 76: shape of batched input X

So the size of the intermediate variables gets changed to:

$$Z^{[1]} \in \mathbb{R}^{3 \times m} \text{ (each column vector is a previous } Z^{[1]})$$

$$Z^{[2]} \in \mathbb{R}^{2 \times m}, \quad Z^{[3]} \in \mathbb{R}^{1 \times m}$$

$$Z^{[3]} = \begin{pmatrix} | & | & \dots & | \\ z^{[3](1)} & z^{[3](2)} & \dots & z^{[3](m)} \\ | & | & \dots & | \end{pmatrix}$$

Figure 77: shape of linear part Z

However, the size of the parameters w, b remains the same as before !

Note that here b remains the same, because we do **broadcasting** to expand $b^{[1]}$ from $\mathbb{R}^{3 \times 1}$ to $\mathbb{R}^{3 \times m}$!!! (copy the column vector m times to get a matrix)

• Optimizing

Goal : Optimize $w^{[1]}, w^{[2]}, w^{[3]}, b^{[1]}, b^{[2]}, b^{[3]}$

• Define : loss / cost function

(loss : only one example in the batch;
cost : multiple examples in the batch)

$$\mathcal{J}(\hat{y}, y) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}^{(i)}$$

We normalize it with $\frac{1}{m}$ because we're going for batch gradient descent. (compute loss function for the whole batch, then calculate the cost function that'll be derived and give us the direction of the gradients)

So $\mathcal{L}^{(i)}$ is the loss function corresponding to only one input. So it'll be a *logistic loss function* :

$$\mathcal{L}^{(i)} = -[y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})]$$

• Backpropagation

Why backward ?

Because we want to apply the iterative updating algorithm to the parameter w , and we start from the layer that's closest to the loss function :

$$w^{(l)} := w^{(l)} - \alpha \frac{\partial \mathcal{J}}{\partial w^{(l)}}$$

$$b^{(l)} := b^{(l)} - \alpha \frac{\partial \mathcal{J}}{\partial b^{(l)}}$$

for every layer ($\forall l = 1, 2, 3$)

Because $\mathcal{J}$ depends on $\hat{y}$, and $\hat{y}$ depends on $Z^{[3]}$, and also $Z^{[3]}$ depends on $w^{[3]}, a^{[2]}$, so we do this chain rule :

$$\frac{\partial \mathcal{J}}{\partial w^{[3]}} = \frac{\partial \mathcal{J}}{\partial a^{[3]}} \cdot \frac{\partial a^{[3]}}{\partial Z^{[3]}} \cdot \frac{\partial Z^{[3]}}{\partial w^{[3]}}$$

And when we move the layer before :

$$\frac{\partial \mathcal{J}}{\partial w^{[2]}} = \frac{\partial \mathcal{J}}{\partial Z^{[3]}} \cdot \frac{\partial Z^{[3]}}{\partial a^{[2]}} \cdot \frac{\partial a^{[2]}}{\partial w^{[2]}}$$

Then in this expression, we'll plug in the $\frac{\partial \mathcal{J}}{\partial Z^{[3]}}$ that was derived before.

(这个地方要思考下怎样找到合适的路径!) 比如说就不能去求 $\frac{\partial w^{[2]}}{\partial a^{[1]}}$!)

§ Lec XII

1. Backpropagation

and

cost function :

$$\mathcal{J}(\hat{y}, y) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}^{(i)}(\hat{y}, y)$$

with

$$\mathcal{L}^{(i)} = -[y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})]$$

Each layer l of the neural network has its own set of parameters $w^{[l]}, b^{[l]}$, and we have this update rule :

$$w^{[l]} := w^{[l]} - \alpha \frac{\partial \mathcal{J}}{\partial w^{[l]}}$$

We also take this simple neural network as an illustration example :

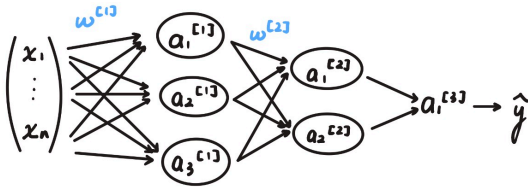


Figure 78: simple neural network

1) Now we want to compute $\frac{\partial \mathcal{J}}{\partial w^{[3]}}$ first because $w^{[3]}$ is closest to the final cost.

As $\mathcal{J}$ is a sum expression, we'd simple compute $\frac{\partial \mathcal{L}}{\partial w^{[3]}}$, because derivation is linear.

$$\frac{\partial \mathcal{L}}{\partial w^{[3]}} = - \left[y^{(i)} \frac{\partial}{\partial w^{[3]}} (\log(\sigma(w^{[3]}a^{[2]} + b^{[3]}))) + (1 - y^{(i)}) \log(1 - \sigma(w^{[3]}a^{[2]} + b^{[3]})) \right]$$

(note that $\hat{y} = \sigma(w^{[3]}a^{[2]} + b^{[3]})$)

As we have these properties :

$$\frac{\partial \log(\sigma(f))}{\partial w} = \frac{1}{\sigma(f)} \frac{\partial(\sigma(f))}{\partial w}$$

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

Therefore :

$$\begin{aligned} \frac{\partial}{\partial w^{[3]}} (\log(\sigma(w^{[3]}a^{[2]} + b^{[3]}))) &= \frac{1}{\sigma(...)} \cdot \frac{\partial(\sigma(...))}{\partial w^{[3]}} \\ &= \frac{1}{\sigma(...)} \cdot (\sigma(...)(1 - \sigma(...))) \cdot a^{[2]T} \\ &= (1 - \sigma(w^{[3]}a^{[2]} + b^{[3]})) \cdot a^{[2]T} \end{aligned}$$

and similarly is the latter part :

$$\begin{aligned} \frac{\partial}{\partial w^{[3]}} (\log(1 - \sigma(...))) &= \frac{1}{1 - \sigma(...)} \cdot \left(-\frac{\partial \sigma(...)}{\partial w^{[3]}} \right) \\ &= \frac{1}{1 - \sigma(...)} \cdot -\sigma(...)(1 - \sigma(...)) \cdot a^{[2]T} \\ &= -\sigma(w^{[3]}a^{[2]} + b^{[3]}) \cdot a^{[2]T} \end{aligned}$$

Finally we have :

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial w^{[3]}} &= -[y^{(i)}(1 - \sigma(...))a^{[2]T} - (1 - y^{(i)})\sigma(...)a^{[2]T}] \\ &= -(y^{(i)} - \sigma(w^{[3]}a^{[2]} + b^{[3]}))a^{[2]T} \\ &= -(y^{(i)} - a^{[3]})a^{[2]T} \end{aligned}$$

Then we can derive the partial derivative of $\mathcal{J}$ with respect to $w^{[3]}$:

$$\frac{\partial \mathcal{J}}{\partial w^{[3]}} = \frac{1}{m} \sum_{i=1}^m (-(y^{(i)} - a^{[3]})a^{[2]T})$$

If we take the partial derivative of $\mathcal{J}$ with respect to $b^{[3]}$, the difficulty will be the same.

2) Now we'd think about how does the derivative backpropagate back to $w^{[2]}$.

$$\frac{\partial \mathcal{L}}{\partial w^{[2]}} = \frac{\partial \mathcal{L}}{\partial a^{[3]}} \cdot \frac{\partial a^{[3]}}{\partial Z^{[3]}} \cdot \frac{\partial Z^{[3]}}{\partial a^{[2]}} \cdot \frac{\partial a^{[2]}}{\partial Z^{[2]}} \cdot \frac{\partial Z^{[2]}}{\partial w^{[2]}}$$

(we need variables that directly connect to each other to pass down the chain rule)

Note that $\frac{\partial \mathcal{L}}{\partial w^{[3]}}$ corresponds to the first two terms (red parts) in $\frac{\partial \mathcal{L}}{\partial w^{[2]}}$. So we just plug in and get :

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial w^{[3]}} &= \frac{\partial \mathcal{L}}{\partial a^{[3]}} \cdot \frac{\partial a^{[3]}}{\partial Z^{[3]}} \cdot \frac{\partial Z^{[3]}}{\partial w^{[3]}} \\ &= \frac{\partial \mathcal{L}}{\partial a^{[3]}} \cdot \frac{\partial a^{[3]}}{\partial Z^{[3]}} \cdot a^{[2]T} \end{aligned}$$

又因为我们已知

$$\frac{\partial \mathcal{L}}{\partial w^{[3]}} = -(y^{(i)} - a^{[3]})a^{[2]T}$$

所以红色部分就等于 $-(y^{(i)} - a^{[3]})$

代入 $\frac{\partial \mathcal{L}}{\partial w^{[2]}}$ 后得：

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial w^{[2]}} &= -(y^{(i)} - a^{[3]}) \cdot \frac{\partial Z^{[3]}}{\partial a^{[2]}} \cdot \frac{\partial a^{[2]}}{\partial Z^{[2]}} \cdot \frac{\partial Z^{[2]}}{\partial w^{[2]}} \\ &= (a^{[3]} - y^{(i)}) \cdot w^{[3]T} \cdot a^{[2]}(1 - a^{[2]}) \cdot a^{[1]T} \end{aligned}$$

Let's analyze the shapes of different part in this derivative :

$$(a^{[3]} - y^{(i)}) \in \mathbb{R}^{1 \times 1} : \text{scalar}$$

$$w^{[3]T} \in \mathbb{R}^{2 \times 1}$$

$$a^{[2]}(1 - a^{[2]}) \in \mathbb{R}^{2 \times 1} : \text{element-wise product}$$

$$a^{[1]T} \in \mathbb{R}^{1 \times 3}$$

If we do the product very rigorously, then we order the terms in this way :

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial w^{[2]}} &= w^{[3]T} * a^{[2]}(1 - a^{[2]}) \cdot (a^{[3]} - y^{(i)}) \cdot a^{[1]T} \\ &\Rightarrow \mathbb{R}^{2 \times 1} * \mathbb{R}^{2 \times 1} \cdot \mathbb{R}^{1 \times 1} \cdot \mathbb{R}^{1 \times 3} \\ &\Rightarrow \mathbb{R}^{2 \times 3} \end{aligned}$$

Speaking of cache, in forward propagation, we're gonna store almost all the values we get and use them in backpropagation.

2. Improving NNs

• Use different activation function

① Sigmoid

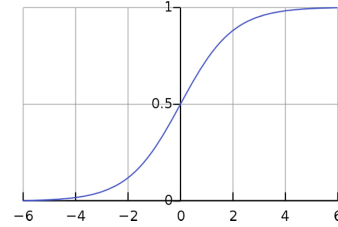


Figure 79: $\sigma(x)$

$$\sigma(z) = \left(\frac{1}{1 + e^{-z}} \right)$$

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

advantage : we can squeeze $(-\infty, +\infty)$ to $(0, 1)$ and output a number as a probability

disadvantage : if z is too high or too low, the gradient is very close to 0, then in backpropagation it's hard to do update

② ReLU

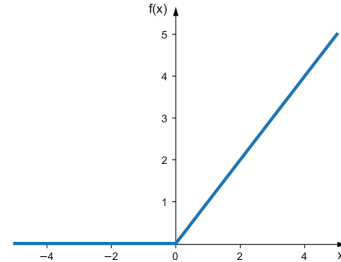


Figure 80: $\text{ReLU}(x)$

$$\text{ReLU}(z) = \begin{cases} 0 & \text{if } z \leq 0 \\ z & \text{if } z > 0 \end{cases}$$

$$\text{ReLU}'(z) = 1\{z > 0\}$$

In ReLU, there's no problem of gradient vanish, because the gradient is always 1 in the positive region.

③ tanh

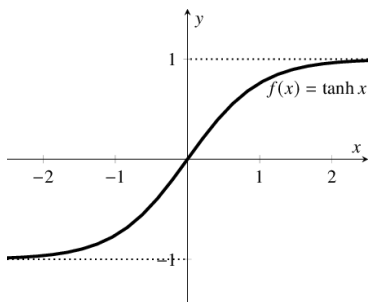


Figure 81: $\tanh(x)$

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

$$\tanh'(z) = 1 - (\tanh(z))^2$$

Similar to sigmoid.

Why activation function ?

e.g If there's no activation function, so that $a^{[l]} = Z^{[l]}$:

$$\begin{aligned} \hat{y} &= a^{[3]} = Z^{[3]} = w^{[3]}a^{[2]} + b^{[3]} = w^{[3]}Z^{[2]} + b^{[3]} \\ &= w^{[3]}(w^{[2]}Z^{[1]} + b^{[2]}) + b^{[3]} \\ &= w^{[3]}(w^{[2]}(w^{[1]}x + b^{[1]}) + b^{[2]}) + b^{[3]} \\ &= Wx + B \end{aligned}$$

So without activation the whole layers of neural network will be simplified as a single linear layer. (equal to a linear regression)

• Initialization techniques

• ▶ Input Initialization

Normalize the input to avoid saturation of the network (like z is too high or low in sigmoid) :

Suggest we have input $x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$, then the distribution may like this :

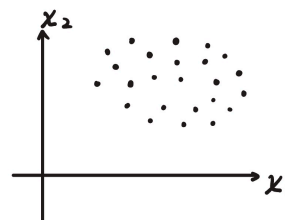


Figure 82: raw input x distribution

The problem is when we do $w^{[1]}x + b^{[1]}$ to compute $Z^{[1]}$, if x is too big then big $Z^{[1]}$ may lead to saturate activation.

So the technique is to normalize x :

$$\begin{bmatrix} \mu_1 \\ \mu_2 \end{bmatrix} = \mu = \frac{1}{m} \sum_{i=1}^m x^{(i)}$$

$$\sigma^2 = \frac{1}{m} \sum_{i=1}^m (x^{(i)} - \mu)^2$$

Then we normalize x :

$$x := \frac{x - \mu}{\sigma}$$

That'll change x 's distribution to this :

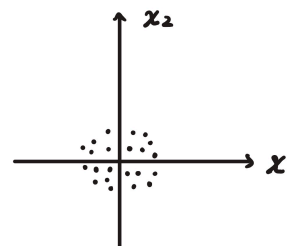


Figure 83: normalized input x

(note that when we do testing, we use the μ, σ from above to normalize the test set ! (should not compute the mean and variance of the test set !))

And the loss function may change from the left one to the right one :

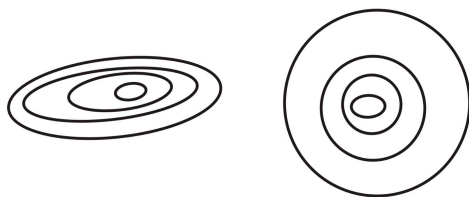


Figure 84: normalized $\rightarrow$ loss plot comparison

As we look at the gradient decent route, the right one is more efficient.

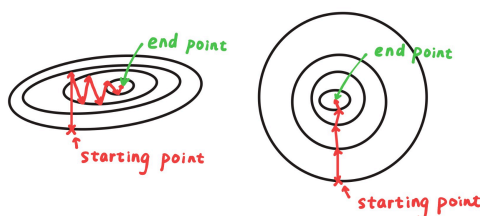


Figure 85: normalized $\rightarrow$ optimizing route comparison

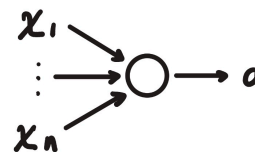


Figure 86: single neuron

It has multiple inputs and outputs and an activation a .

e.g:

$$a = \sigma(Z);$$

$$Z = w_1 x_1 + \dots + w_n x_n$$

假设：输入 x_i 独立同分布，均值为 0，方差为 $\text{Var}(x)$ ；权重 w_i 独立同分布，均值为 0，方差为 $\text{Var}(w)$

我们为了避免 Z 进入激活函数的饱和区，应当控制 Z 在每一层传播的方差应当保持不变，即要使得 $\text{Var}(x) \approx \text{Var}(z)$

$$\begin{aligned} \text{Var}(Z) &= \text{Var}\left(\sum_{i=1}^n w_i x_i\right) = \sum_{i=1}^n \text{Var}(w_i x_i) \\ &= n \cdot \text{Var}(w) \cdot \text{Var}(x) \\ &\approx \text{Var}(x) \end{aligned}$$

所以我们需要使 $\text{Var}(w) \approx \frac{1}{n}$

• • Weights Initialization

1) Problem : Vanishing | Exploding gradients

① Now we consider a simple neural network, in which we let activation = I , $b = 0$ (no activation, no bias).

$$\hat{y} = w^{[l]} w^{[l-1]} \dots w^{[1]} x$$

If $w = \begin{bmatrix} s & 0 \\ 0 & s \end{bmatrix}$, then if $s < 1$, the gradient would vanish when doing the multiplication; and if $s > 1$, the gradient would explode.

One way to solve this is to initialize w into the right range of value. In the case above, we want $s \approx 1$.

② Example with 1 neuron :

2) Weights Initialization Techniques :

① Sigmoid activations

```
w^(l) = np.random.randn(shape) *
np.sqrt(1 / n^(l-1))
```

We're looking at how many inputs come into our layer l , and initialize the weights of this layer proportionally to the number of inputs that are coming in. This initialization is proved to behave well for **sigmoid** activations.

这个与上面的那个简单例子相同，所以得出的结论一样，就是要让 $\text{Var}(w) \approx \frac{1}{n^{l-1}}$ (n^{l-1} 是第 l 层的输入个数)

② ReLU activations

Interestingly, if we use **ReLU**, it's better to change the numerator from 1 to 2 (ReLU 会把一半的神经元置零 (负输入的输出为 0)。这意味着实际有效的输入个数只有一半。为了补偿这种“减半”效应，需要把方差放大一倍)：

```
w^(l) = np.random.randn(shape) *  
np.sqrt(2 / n^(l-1))
```

The reason why we use random initialization is that if there's no randomness, we'll end up with a problem called symmetry where every neuron is going to learn kind of the same thing.

③ Xavier Initialization

这种初始化同时考虑前向传播和反向传播的方差稳定性：

前向传播要求： $\text{Var}(w^l) \approx \frac{1}{n^{l-1}}$ ；

反向传播要求： $\text{Var}(w^l) \approx \frac{1}{n^l}$ ；

两者折中，取调和平均，即得：

$$\text{Var}(w^l) = \frac{2}{n^{l-1} + n^l}$$

也即得：

$$w^l \sim \mathcal{N}\left(0, \frac{2}{n^{l-1} + n^l}\right)$$

这种初始化适用于 tanh / sigmoid 作激活函数的时候

• Optimization

• ▶ Mini-batch gradient descent

Suggest we have a dataset like :

$$X = (x^{(1)}, \dots, x^{(m)})$$

$$Y = (y^{(1)}, \dots, y^{(m)})$$

Then we split it into batches (a total of T batches) :

$$X = (x^{\{1\}}, \dots, x^{\{T\}})$$

$$Y = (y^{\{1\}}, \dots, y^{\{T\}})$$

Now the Mini-batch gradient descent algorithm works like this :

For iteration $t = 1, \dots$:

Select a batch of data $(x^{\{t\}}, y^{\{t\}})$:

Forward propagate this batch

Backpropagate this batch

Update all w^l, b^l for all layers

Forward propagate : we send all the data in this batch into the network and compute the loss function over the entire batch.

If we look at the graphs for cost function :

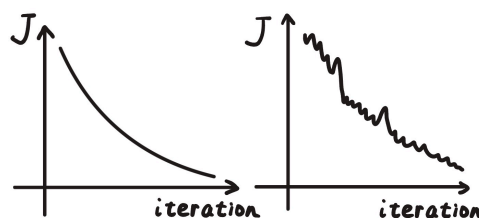


Figure 87: cost function comparison

On the left we use batch gradient descent, and the cost function would smoothly descend as iteration increases.

On the right we use mini-batch gradient descent and because the gradient is approximated and doesn't necessarily go straight to the lower point of the cost function, we have noises on the cost function, but the cost is decreasing as a trend.

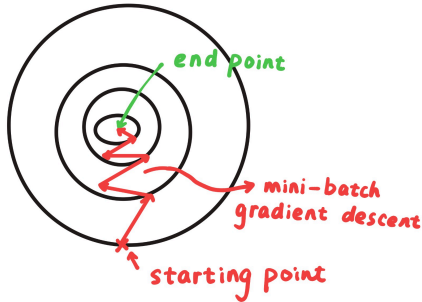


Figure 88: mini-batch GD route

Though mini-batch gradient descent needs more iterations, but each iteration of it is much easier to compute, so it's more efficient than batch gradient descent.

• Momentum Algorithm

Intuition : Let's look at this loss contour plot :

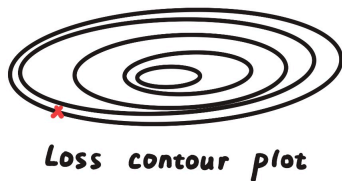


Figure 89: loss contour plot

The ordinary gradient descent's route is always orthogonal to the contour curve, so the route may like this :

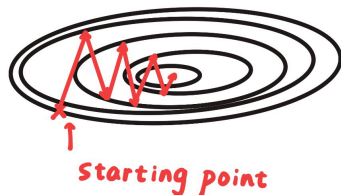


Figure 90: ordinary GD optimization route

So how to improve efficiency ? We want to move with larger updates horizontally, and move with smaller updates vertically.

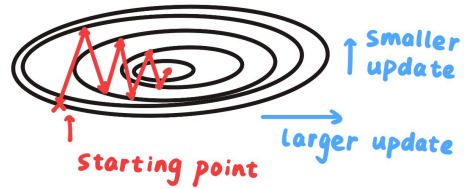


Figure 91: shape of linear part Z

So we're gonna use a technique called momentum, which is going to look at the past gradients, and tried to consider these past updates to find the correct direction.

When we look at the past updates, we'd take the average of the past horizontal and vertical updates. Here, because we're essentially keep moving rightward, so there'll be little change to horizontal update, but since there's ups and downs vertically, we'd take smaller vertical updates.

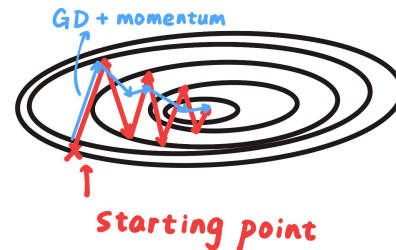


Figure 92: shape of linear part Z

That'll lead us faster to optimal point.

momentum : “have weight”, so cannot change direction very noisily.

初始化 : $v = 0$,

$$v := \beta v + (1 - \beta) \frac{\partial \mathcal{L}}{\partial w}$$

$$w := w - \alpha v$$

这样 w 的更新直接使用使用的是 v , 这样的话 v 既有当前的梯度值, 也有历史的方向值, 这样与“惯性”含义有些类似。

§ Lec XIII

Model Debugging and Error Analysis

outline :

- Diagnostics for debugging learning algorithms
- Error analysis and ablative analysis
- Premature (statistical) optimization

1. Debugging Learning Algorithms

• Motivating Example :

If we're constructing an anti-spam classifier, and we carefully choose a small set of 100 words to use as features.

Now we use logistic regression with regularization (Bayesian logistic regression), implemented with gradient ascent but gets a really high test error.

(Bayesian Logistic Regression) :

$$\max_{\theta} \sum_{i=1}^m \log p(y^{(i)} | x^{(i)}, \theta) - \lambda \|\theta\|^2$$

Common approaches :

- Try getting more training examples
- Try a smaller / larger set of features
- Try changing the features : Email header vs. email body features
- Run gradient descent for more iterations
- Try Newton's method
- Use a different value for λ
- Try using an SVM / neural network

An effective way we often use is **bias-variance diagnostic**. (high bias $\rightarrow$ underfit; high variance $\rightarrow$ overfit)

• Bias - Variance Diagnostic :

- ① Variance : Training error will be much lower than test error
- ② Bias : Training error will also be high

1) Typical learning curve for high variance :

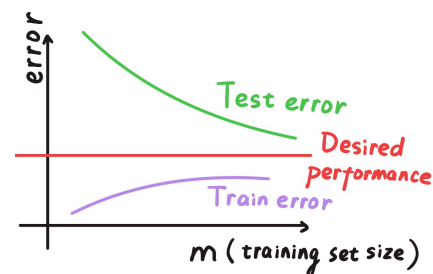


Figure 93: learning curve for high variance

This corresponds to overfitting :

- ① Test error still decreasing as m increases. Suggest larger training set will help.
- ② **Large gap between training and test error.** (this is more important !)

2) Typical Learning curve for high Bias :

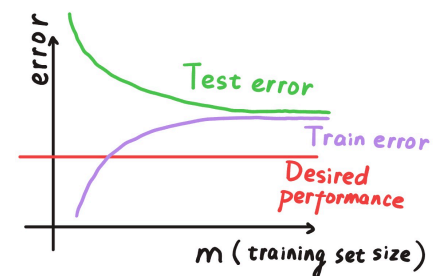


Figure 94: learning curve for high bias

- ① Even training error is unacceptably high
- ② **Small gap** between training and test error

3) Now we can recall the fixes we use before :

- Try getting more training examples — Fixes high variance
- Try a smaller set of features — Fixes high variance
- Try a larger set of features — Fixes high bias
- Try email header features — Fixes high bias
- Use a different value for λ — bias-variance trade off (正则化越强→增大偏差减小方差 !)

• **Another example :**

Logistic regression gets 2% error on both spam and non-spam (Unacceptable high error on non-spam)

SVM using a linear kernel gets 10% error on spam, and 0.01% error on non-spam. (Acceptable performance)

But we still want to use LR because of computational efficiency.

Questions:

① **Is the algorithm (gradient ascent for logistic regression) converging ?**

Usually we have this curve :

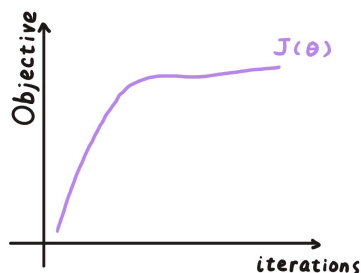


Figure 95: objective optimization function

It's often very hard to tell if an algorithm has converged yet by looking at the objective.

② **Are you optimizing the right function ?**

e.g. (maybe we'd care more about non-spam than spam so we should weight them more)

$$a(\theta) = \sum_i w^{(i)} 1\{h_\theta(x^{(i)}) = y^{(i)}\}$$

③ **Is logistic regression the right model ? If yes, is λ correct ?**

$$\max_{\theta} J(\theta) = \sum_{i=1}^m \log p(y^{(i)}|x^{(i)}, \theta) - \lambda \|\theta\|^2$$

④ **Is SVM the right model ? If yes, is C correct ?**

$$\min_{w,b} \|w\|^2 + C \sum_{i=1}^m \xi_i$$

$$s.t. y^{(i)}(w^T x^{(i)} - b) \geq 1 - \xi_i$$

We can summarize the example above like this :

We've got θ_{SVM} and θ_{BLR} (BLR = Bayesian Logistic Regression). For this function that we really care :

$$a(\theta) = \sum_i w^{(i)} 1\{h_\theta(x^{(i)}) = y^{(i)}\}$$

We now have

$$a(\theta_{\text{SVM}}) > a(\theta_{\text{BLR}})$$

Remember our θ_{BLR} comes from maximizing $J(\theta)$ in BLR.

• **Optimization Algorithm Diagnostics**

Then the diagnostic can be :

$$J(\theta_{\text{SVM}}) \stackrel{?}{>} J(\theta_{\text{BLR}})$$

case 1 :

$$a(\theta_{\text{SVM}}) > a(\theta_{\text{BLR}})$$

$$J(\theta_{\text{SVM}}) > J(\theta_{\text{BLR}})$$

But θ_{BLR} was trying to maximize $J(\theta)$. So we can infer that θ_{BLR} fails to maximize $J(\theta)$ and the problem is with the convergence of the algorithm. — **Problem : optimization algorithm**

case 2 :

$$a(\theta_{\text{SVM}}) > a(\theta_{\text{BLR}})$$

$$J(\theta_{\text{SVM}}) \leq J(\theta_{\text{BLR}})$$

BLR was successful in maximizing $J(\theta)$, but SVM does better at the weighted accuracy $a(\theta)$. This means that $J(\theta)$ is the wrong objective function to maximize if we care about $a(\theta)$. — **Problem : Objective function of the maximization problem.** (case 2 对应的修复方法: 修改当前模型的目标函数 (比如调整 λ, C 值); 或者直接换一个模型 (这样天然就会修改目标函数))

Again we come back to these approaches:

- Run gradient descent for more iterations — **Fixes optimization algorithm**
- Try Newton's method — **Fixes optimization algorithm**
- Use a different value for λ — **Fixes optimization objective** (note that this approach is more often used in bias-variance trade-off)
- Try using an SVM / neural network — **Fixes optimization objective or change to a different model**

The diagnostics above can be used for debugging RL algorithms.

2. Error Analysis

Many applications combine many different learning components into a “pipeline”.

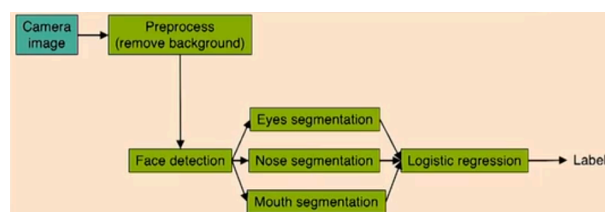


Figure 96: system pipeline

Now we want to analyze how much error is attribute to each of the component so we can decide which component to work on next.

Plug in ground-truth for each component, and see how accuracy changes. Then we find in which part lies the most room for improvement.

Error helps to figure out the *difference between the current point and our goal (perfect performance)*.

3. Ablative Analysis

Remove components from the system one at a time, to see how it breaks.

Then we can find out which part accounts for most of the improvement.

It tries to explain *difference between our current performance and performance much worse*.

§ Lec XIV

Unsupervised Learning

1. K-Means Clustering

First, we're given an unlabeled dataset. And we want an algorithm to try to find maybe the two clusters here.

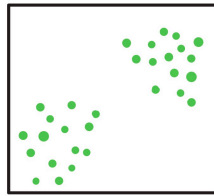


Figure 97: system pipeline

First step : Pick two crosses — cluster centroids. And we go through all the training examples and for each of them we colored them as two categories depending on their distance to the centroids.

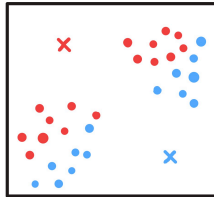


Figure 98: system pipeline

Second step : Find the means respectively for all the blue and red dots and move our centroids to the mean points.

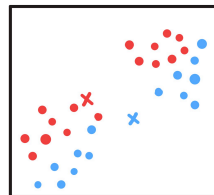


Figure 99: system pipeline

Iteratively, we color the points depending their distances to the new centroids. And again we move centroids to the new mean points.

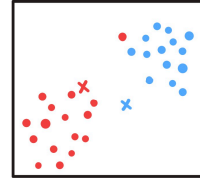


Figure 100: system pipeline

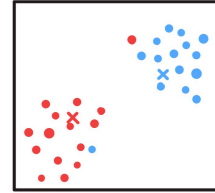


Figure 101: system pipeline

To some point, this iteration will stop and the algorithm will converge.

We can describe K-means clustering in a mathematical way.

Given a dataset : $x^{(1)}, \dots, x^{(m)}$ (unlabeled)

① **Initialize cluster centroids** (usually we don't set them randomly, we just pick k examples in the training set as our initial centroids) :

$$\mu_1, \dots, \mu_k \in \mathbb{R}^n$$

② **Repeat until convergence** :

(a) Set $c^{(i)} = \arg \min_j \|x^{(i)} - \mu_j\|_2$ (color the points) (we use ℓ_2 norm with or without the square)

(b) For $j = 1, \dots, h()$

$$\mu_j := \frac{\sum_{i=1}^m 1\{c^{(i)} = j\} x^{(i)}}{\sum_{i=1}^m 1\{c^{(i)} = j\}}$$

This algorithm is proved to converge.
If we write that as a **cost function** :

$$\mathcal{J}(c, \mu) = \sum_{i=1}^m \|x^{(i)} - \mu_{c(i)}\|^2$$

It's a function of c (assignments) and μ (centroids).

It turns out that with every iteration, k-means will drive this cost function down. And with the right term > 0 , so this algorithm must converge. Notice that sometimes k-means could be stuck in local minima.

• Density Estimation

Motivating example : aircraft engine anomaly detection problem

Given a vibration and heat value combination, we'd decide whether it is an unusual one.

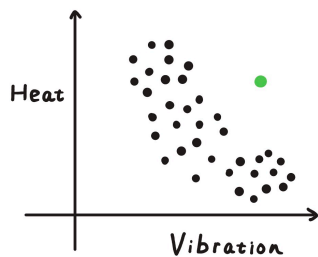


Figure 102: system pipeline

One way to implement is to model $p(x)$. (直接学一个函数 $p(x)$ 来告诉我们“特征组合 x 出现的密度有多高”。When $p(x) < \varepsilon$, that means some anomaly.

What's interesting about the green dot in the graph is that neither its vibration nor heat feature is out of range. However if we look at the combination of these two feature we shall find out the anomaly.

If we only look at the data points, it's a “L” shape distribution and we don't have a distribution to directly model this complex distribution.

Now we can see it as a **mixture of Gaussian distribution**. With the two ellipses as contour lines of two Gaussian distribution (注意二维高斯分布的密度函数的等高线就是一个椭圆), we can now assume that the data points in the figure are sampled from these two Gaussian distributions (两个椭圆的拼接).

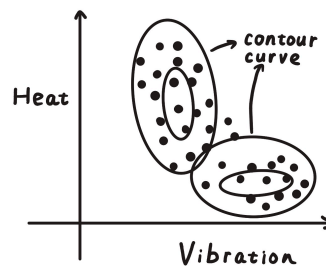


Figure 103: system pipeline

• Gaussian Mixture Model (GMM)

Suppose now we have this simple one-dimensional data points :

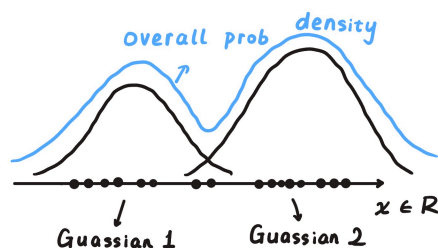


Figure 104: system pipeline

We assume these data come from two Gaussian distributions, but we don't know which data points belong to which distribution (unlabeled data).

EM (expectation maximum) algorithm can help us fit a model despite not knowing which Gaussian each example that comes from.

• • **Gaussian Mixture Model (GMM)** :

我们需要引入一个变量 $z^{(i)}$ 来表示数据点 $x^{(i)}$ 来自哪个高斯分布。

Suppose there's a latent random variable z , and $x^{(i)}, z^{(i)}$ have this joint distribution :

$$P(x^{(i)}, z^{(i)}) = P(x^{(i)} | z^{(i)})P(z^{(i)})$$

where $z^{(i)}$ is multinomial with some set of parameters ϕ .

$$z \sim \text{Multinomial}(\phi)$$

$z^{(i)} \in \{1, \dots, k\}$ 总共 k 种取值可能

$$\phi \geq 0, \sum_{j=1}^k \phi_j = 1$$

the parameter ϕ_j gives $p(z^{(i)} = j)$

在我们上面的例子中，我们假设一共有 k 个簇，然后在这些簇里面才会产生数据 $x^{(i)}$ ，我们假设这 k 个簇里面的数据都分别是一个高斯分布：

$$x^{(i)} | z^{(i)} = j \sim \mathcal{N}(\mu_j, \Sigma_j)$$

因此数据点的产生是这样的：

- ① 先按照概率（此时概率为 ϕ_j ）抽到一个 $z^{(i)}$ ，其值为 j ；
- ② 在第 j 个簇的高斯分布中再抽取得到 $x^{(i)}$

最后我们独立地重复以上步骤 m 次得到数据集 $\{x^{(1)}, \dots, x^{(m)}\}$

现在我们通过 $P(x^{(i)}, z^{(i)})$ 来间接得到 $P(x)$ 的表达式，有：

$$\begin{aligned} P(x^{(i)}, z^{(i)} = j) &= P(x^{(i)} | z^{(i)})P(z^{(i)} = j) \\ &= P(x^{(i)} | z^{(i)} = j) \phi_j \\ &= \phi_j \cdot \mathcal{N}(x^{(i)}; \mu_j, \Sigma_j) \end{aligned}$$

在实际问题中，我们只观测到 $x^{(i)}$ ，看不到 $z^{(i)}$ 。所以我们需要对 $z^{(i)}$ 求和，得到 $x^{(i)}$ 的边缘分布，这也就是我们最后要对原始数据建模的概率分布：

$$\begin{aligned} P(x^{(i)}) &= \sum_{j=1}^k P(x^{(i)}, z^{(i)} = j) \\ \Leftrightarrow P(x^{(i)}) &= \sum_{j=1}^k \phi_j \cdot \mathcal{N}(x^{(i)}; \mu_j, \Sigma_j) \end{aligned}$$

这样，我们相当于就是把 $P(x^{(i)})$ 这个最终要建模的概率分布参数化用参数 ϕ_j, μ_j, Σ_j 来表示了。

所以我们只要估计出参数的最优值，那么这个最终建模概率分布就可以计算了。

The parameters that we have to estimate are :

$$\theta = \{\phi_j, \mu_j, \Sigma_j\}$$

给定数据点，要估计参数值，我们本能地想到可以用最大似然估计，于是我们写出对应的似然函数与对数似然函数：

$$\begin{aligned} \text{want to max : } \mathcal{L}(\theta) &= \prod_{i=1}^m p(x^{(i)}; \theta) \\ \Leftrightarrow \max \mathcal{L}(\theta) : \log(\mathcal{L}(\theta)) &= \sum_{i=1}^m \log(p(x^{(i)}; \theta)) \end{aligned}$$

注意：最大似然估计实质就是把 $P(x)$ 这个要最终建模的概率分布“参数化”为一个参数 θ 控制的函数族 $P(x; \theta)$ ，然后对这个最大化时候的参数取值就是最后得到的参数的估计值。

So we want to maximize :

$$\begin{aligned} \mathcal{L}(\phi, \mu, \Sigma) &= \sum_{i=1}^m \log P(x^{(i)}; \phi, \mu, \Sigma) \\ &= \sum_{i=1}^m \log \left(\sum_{j=1}^k \phi_j \cdot \mathcal{N}(x^{(i)}; \mu_j, \Sigma_j) \right) \end{aligned}$$

如果我们对上面这个对数似然函数直接做最大化来求解参数（令导数为 0）会发现无法用闭式解来找到这些参数的最大似然估计。

所以后续我们引入 EM 算法来解决这个参数最优化求解问题。

• $w_j^{(i)}$'s Change

在上面最开始的 MLE 中我们发现无法直接用闭式解求得最大似然估计，现在我们来看下是什么原因导致没法直接求解？怎样才能求解？

我们来看有一种情况：

如果我们已知 $z^{(i)}$ 的值（也就是我们不仅观测到 $x^{(i)}$ ，而且还知道这个属于哪个分量 $z^{(i)}$ ），那么我们就可以把似然函数拆写成下面的形式：

$$\begin{aligned}\ell(\phi, \mu, \Sigma) &= \sum_{i=1}^m \log \sum_{z^{(i)}=1}^k P(x^{(i)}|z^{(i)}; \mu, \Sigma) P(z^{(i)}; \phi) \\ &= \sum_{i=1}^m [\log P(x^{(i)}|z^{(i)}; \mu, \Sigma) + \log P(z^{(i)}; \phi)]\end{aligned}$$

此时 $z^{(i)}$ 已经确定，不需要再对 $z^{(i)}$ 从 1 开始求和到 k。

此时再求解最大似然时，令对 ϕ_j, μ_j, Σ_j 的导数均为 0，就能很方便解出：

$$\begin{aligned}\phi_j &= \frac{1}{m} \sum_{i=1}^m 1\{z^{(i)} = j\}, \\ \mu_j &= \frac{\sum_{i=1}^m 1\{z^{(i)} = j\} x^{(i)}}{\sum_{i=1}^m 1\{z^{(i)} = j\}}, \\ \Sigma_j &= \frac{\sum_{i=1}^m 1\{z^{(i)} = j\} (x^{(i)} - \mu_j)(x^{(i)} - \mu_j)^T}{\sum_{i=1}^m 1\{z^{(i)} = j\}}\end{aligned}$$

这下我们获得一种启发：既然已知 $z^{(i)}$ 之后就能轻松求得参数的最大似然估计值，那我们是不是可以引入一种方法来预先为每一个样本 $x^{(i)}$ 来设定其对应的类别值 $z^{(i)}$ 这样可能会便于我们求解？

这样便引出了 EM 算法的核心思想：为每一个样本当前设定一个“软”类别值（也就是按照一定的概率值分类到不同类别去）作为当前这一步的“假设”便于这一步作出优化，然后迭代地进入下一步继续通过新的“假设”来更新，最后应该就会收敛到“真实的”最优值。

• A quick overview of the EM algorithm

• ▶ E-step : Guess the value of $z^{(i)}$

Set :

$$w_j^{(i)} = P(z^{(i)} = j | x^{(i)}; \phi, \mu, \Sigma)$$

注意这是一种“软分类”，把每一个样本点按照不同的概率值分到不同的分布中去。（而不是像 k-means 一样每一步迭代中每一个样本点一定有且仅有属于一个分布中）

And we use Baye's rule which is similar to generative learning algorithm :

$$w_j^{(i)} = \frac{P(x^{(i)}|z^{(i)} = j) \cdot P(z^{(i)} = j)}{\sum_{l=1}^k P(x^{(i)}|z^{(i)} = l) P(z^{(i)} = l)}$$

$P(x^{(i)}|z^{(i)} = j)$ comes from Guassian density.
 $P(z^{(i)} = j)$ comes from our assumption about $z \sim \text{Multinomial}(\phi)$.

The terms at the denominator also comes from Guassian density and ϕ .

We can also write $w_j^{(i)}$ like this :

$$w_j^{(i)} = \frac{\mathcal{N}(x^{(i)}; \mu_j, \Sigma_j) \cdot \phi_j}{\sum_{l=1}^k \mathcal{N}(x^{(i)}; \mu_l, \Sigma_l) \phi_l}$$

We can see that $w_j^{(i)}$ is dependent on (old) parameters.

So in E-step, we try to guess each example's $z^{(i)}$ and we store the probabilities in $w_j^{(i)}$. (实际上 $w_j^{(i)}$ 就是 $z^{(i)}$ 的后验分布)

$w_j^{(i)}$: “how much $x^{(i)}$ is assigned to the μ_j Gaussian”

• ▶ M-step : use MLE to give estimations of the parameters

Using $w_j^{(i)}$ and MLE, we can get estimates of the parameters : (注意 Σ_j 用更新后的 μ_j 算!)

$$\begin{aligned}\phi_j &= \frac{1}{m} \sum_{i=1}^m w_j^{(i)} \\ \mu_j &= \frac{\sum_{i=1}^m w_j^{(i)} x^{(i)}}{\sum_{i=1}^m w_j^{(i)}} \\ \Sigma_j &= \frac{\sum_{i=1}^m w_j^{(i)} (x^{(i)} - \mu_j)(x^{(i)} - \mu_j)^\top}{\sum_{i=1}^m w_j^{(i)}}\end{aligned}$$

note that : $w_j^{(i)} = E[1\{z^{(i)} = j\}]$

Iteratively, we then update $w_j^{(i)}$ again and plug in the steps above. ($w_j^{(i)}$ is dependent on parameters from last time !)

One intuition about mixture of Gaussians model is that it's like a k-means but with a soft assignment. In k-means once we've update the centroids then we each point is assigned to a centroid. But EM uses the probabilities as weights to assign each point to a cluster, then update corresponding means.

• Rigorous Derivation of EM Algorithm

Now we want to derive EM algorithm rigorously, about why it's reasonable, why it's a MLE algorithm, and why it will converge.

• ▶ Tool : Jensen's Inequality

Let f be a convex function (e.g. $f'' > 0$) ;

Let X be a random variable ;

We have :

$$f(E[X]) \leq E[f(X)]$$

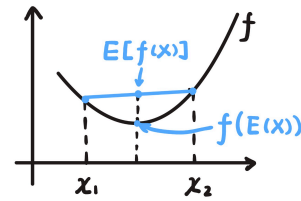


Figure 105: system pipeline

Further, if $f'' > 0$ (f is strictly convex), then

$$f(E[X]) = E[f(X)] \Leftrightarrow X \text{ is a constant}$$

When f is changed from convex to concave, then the whole conclusions will be the opposite way.

We have a model for $P(x, z; \theta)$ (θ is the parameter)

And we only observe $x : \{x^{(1)}, \dots, x^{(m)}\}$

$$\begin{aligned}\ell(\theta) &= \sum_{i=1}^m \log P(x^{(i)}; \theta) \\ &= \sum_{i=1}^m \log \left[\sum_{z^{(i)}} P(x^{(i)}, z^{(i)}; \theta) \right]\end{aligned}$$

Now we want to solve $\max_{\theta} \ell(\theta)$, and we'll derive an algorithm that works iteratively so find the MLE estimate of θ . (由于 log 里面有一个求和, 直接求导没有闭式解。因此我们想找一个下界, 通过最大化下界来间接最大化 $\ell(\theta)$ 并找到对应参数, 所以目标思想还是 MLE)

- **EM intuitive geometric steps**

Let's draw a picture of this iterative optimization process :

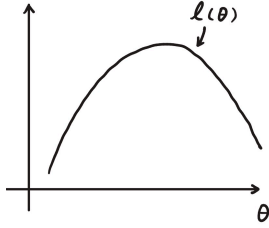


Figure 106: system pipeline

First, we initialize θ randomly :

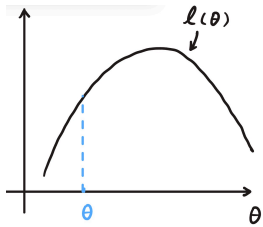


Figure 107: system pipeline

1) E-step :

We construct a lower bound for this log likelihood curve (green line), which has two properties : ① *lower* ② it's *equal* to log-likelihood at θ

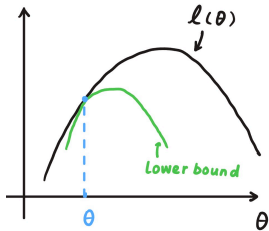


Figure 108: system pipeline

2) M-step :

Find the value for a new θ that maximizes the green lower bound. We update θ to that new value.

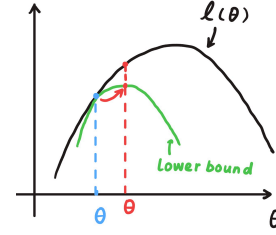


Figure 109: system pipeline

And we find a new lower bound of log-likelihood at θ again and do the update iteratively. Finally it'll converge to a local minimum.

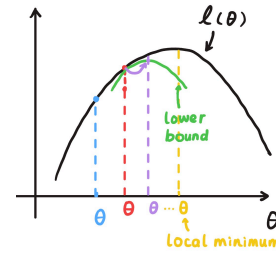


Figure 110: system pipeline

- **Mathematical Process**

Now let's look at mathematical of the process above :

$$\begin{aligned} \text{Goal : } & \max_{\theta} \sum_i \log P(x^{(i)}; \theta) \\ \Rightarrow & \sum_i \log \sum_{z^{(i)}} P(x^{(i)}, z^{(i)}; \theta) \\ \Rightarrow & \sum_i \log \sum_{z^{(i)}} Q_i(z^{(i)}) \left[\frac{P(x^{(i)}, z^{(i)}; \theta)}{Q_i(z^{(i)})} \right] \end{aligned}$$

where $Q_i(z^{(i)})$ is a probability distribution that : $\sum_{z^{(i)}} Q_i(z^{(i)}) = 1$

So it becomes (note that we see the term in square brackets as a function of $z^{(i)}$, so it ends up as a expectation of $z^{(i)}$):

$$\Rightarrow \sum_i \log E_{z^{(i)} \sim Q_i} \left[\frac{P(x^{(i)}, z^{(i)}; \theta)}{Q_i(z^{(i)})} \right]$$

Now we use Jensen's inequality (log function is concave !):

$$\Rightarrow \geq \sum_i E_{z^{(i)} \sim Q_i} \left[\log \left(\frac{P(x^{(i)}, z^{(i)}; \theta)}{Q_i(z^{(i)})} \right) \right]$$

Then we expand it out :

$$\Rightarrow \geq \sum_i \sum_{z^{(i)}} Q_i(z^{(i)}) \log \left(\frac{P(x^{(i)}, z^{(i)}; \theta)}{Q_i(z^{(i)})} \right)$$

So we have :

$$\ell(\theta) \geq \sum_i \sum_{z^{(i)}} Q_i(z^{(i)}) \log \left(\frac{P(x^{(i)}, z^{(i)}; \theta)}{Q_i(z^{(i)})} \right)$$

Note that the expression on the right is a function of θ ! ($x^{(i)}$ is data and we take the sum over $z^{(i)}$) Then it can serve as the lower bound of $\ell(\theta)$!!!

Don't forget that we want the lower bound is equal to $\ell(\theta)$ at the current θ .

On a given iteration of EM (with parameter θ), **we want Jensen's inequality to attain equality here :**

$$\begin{aligned} \sum_i \log E_{z^{(i)} \sim Q_i} \left[\frac{P(x^{(i)}, z^{(i)}; \theta)}{Q_i(z^{(i)})} \right] &= \sum_i E_{z^{(i)} \sim Q_i} \left[\log \left(\frac{P(x^{(i)}, z^{(i)}; \theta)}{Q_i(z^{(i)})} \right) \right] \\ \Leftrightarrow \log E_{z^{(i)} \sim Q_i} \left[\frac{P(x^{(i)}, z^{(i)}; \theta)}{Q_i(z^{(i)})} \right] &= E_{z^{(i)} \sim Q_i} \left[\log \left(\frac{P(x^{(i)}, z^{(i)}; \theta)}{Q_i(z^{(i)})} \right) \right] \end{aligned}$$

For this to hold :

$$\frac{P(x^{(i)}, z^{(i)}; \theta)}{Q_i(z^{(i)})} = \text{constant} \quad (\forall z^{(i)})$$

So we can set :

$$Q_i(z^{(i)}) \propto P(x^{(i)}, z^{(i)}; \theta)$$

Remember that Q_i is a probability distribution we choose for $z^{(i)}$, so we can simply set :

$$\begin{aligned} Q_i(z^{(i)}) &= \frac{P(x^{(i)}, z^{(i)}; \theta)}{\sum_{z^{(i)}} P(x^{(i)}, z^{(i)}; \theta)} \\ &= P(z^{(i)} \mid x^{(i)}; \theta) \end{aligned}$$

note that $\sum_{z^{(i)}} P(x^{(i)}, z^{(i)}; \theta) = P(x^{(i)})$

(this is the **posterior probability**)

• ▶ Summary

E-step :

Set

$$Q_i(z^{(i)}) = P(z^{(i)} \mid x^{(i)}; \theta)$$

(note that previously we set $w_j^{(i)}$ which is now in our distribution Q_i)

M-step :

$$\theta := \arg \max_{\theta} \sum_i \sum_{z^{(i)}} Q_i(z^{(i)}) \log \left(\frac{P(x^{(i)}, z^{(i)}; \theta)}{Q_i(z^{(i)})} \right)$$

(we find the maximum point of our lower bound and update θ to that point !)

So now we know that EM algorithm is a maximum likelihood estimation algorithm with optimization solved by constructing lower bounds and optimizing lower bounds.

1. Proof of K-Means' Convergence

我们之前在 k-means 处定义的损失函数是 (distortion function):

$$\mathcal{J}(c, \mu) = \sum_{i=1}^m \|x^{(i)} - \mu_{c^{(i)}}\|^2$$

It's the sum of squared distances between each training example $x^{(i)}$ and the cluster centroid $\mu_{c^{(i)}}$ to which it has been assigned.

显然, $\mathcal{J}(c, \mu) \geq 0$, 所以我们只需要证明 k-means 的每一次迭代都会让这个损失函数严格下降即可, 单减有下界的函数一定收敛!

K-means 中参数分为两部分, 一部分是 $c^{(i)}$ 表示当前第 i 个样本 $x^{(i)}$ 被分配给哪个簇; 另一部分是 μ_j 表示第 j 个簇的位置。K-means 目标就是选取最优的这两组参数, 让 distortion function 最小化。

既然有两组参数需要优化, k-means 采取的是坐标优化法 (coordinate descent): 每次固定其他变量, 只优化其中一个变量 (或一组变量), 使目标函数下降。

于是, 在内循环中“内循环做的是:

① 固定 μ , 优化 c : 对于每一个样本选取最合适的分类标签

② 固定 c , 优化 μ : 由于此时各个簇之间相互独立, 因此所以可以分别对每个 μ_j 最小化, 即选取各个簇内的质心即可。

由于内循环中每一步都是精确的单减优化, 因此损失函数不会增加。

Notice :

① 虽然 $\mathcal{J}$ 会收敛, 但严格来说, c 和 μ 不一定收敛到唯一值, 理论上可能出现: 算法在几个不同的聚类结果之间来回跳, 但这些结果的 $\mathcal{J}$ 完全相同。但实际中一般不会出现这种情况。

② $\mathcal{J}$ 非凸, 所以收敛到的是局部最优, 不保证全局最优。

2. Detailed Calculus in EM steps :

1) . Why no closed form solution in MLE : ?

$$\begin{aligned} \ell(\phi, \mu, \Sigma) &= \sum_{i=1}^m \log P(x^{(i)}; \phi, \mu, \Sigma) \\ &= \sum_{i=1}^m \log \left(\sum_{j=1}^k \phi_j \cdot \mathcal{N}(x^{(i)}; \mu_j, \Sigma_j) \right) \end{aligned}$$

① 定义后验责任度

$$w_j^{(i)} = P(z^{(i)} = j | x^{(i)}; \phi, \mu, \Sigma) = \frac{\phi_j \mathcal{N}(x^{(i)}; \mu_j, \Sigma_j)}{\sum_{l=1}^k \phi_l \mathcal{N}(x^{(i)}; \mu_l, \Sigma_l)}.$$

表示在当前参数下, 第 i 个样本属于第 j 个高斯分布的概率

② 对 μ_j 求偏导

$$\frac{\partial}{\partial \mu_j} \log \mathcal{N}(x; \mu_j, \Sigma_j) = \Sigma_j^{-1} (x - \mu_j).$$

$$\Rightarrow \frac{\partial \ell}{\partial \mu_j} = \sum_{i=1}^m w_j^{(i)} \Sigma_j^{-1} (x^{(i)} - \mu_j) = 0.$$

因为 Σ_j^{-1} 可逆, 得到

$$\sum_{i=1}^m w_j^{(i)} (x^{(i)} - \mu_j) = 0,$$

$$\Rightarrow \mu_j = \frac{\sum_{i=1}^m w_j^{(i)} x^{(i)}}{\sum_{i=1}^m w_j^{(i)}}.$$

③ 对 Σ_j 求导

高斯对数密度对 Σ_j 的矩阵导数为

$$\frac{\partial}{\partial \Sigma_j} \log \mathcal{N}(x; \mu_j, \Sigma_j) = -\frac{1}{2} \Sigma_j^{-1} + \frac{1}{2} \Sigma_j^{-1} (x - \mu_j)(x - \mu_j)^\top \Sigma_j^{-1}.$$

令导数为零, 得:

$$\Sigma_j = \frac{\sum_{i=1}^m w_j^{(i)} (x^{(i)} - \mu_j)(x^{(i)} - \mu_j)^\top}{\sum_{i=1}^m w_j^{(i)}}.$$

④ 对 ϕ_j 求导

由于这个概率值变量自带归一化约束 $\sum_{j=1}^k \phi_j = 1$ ，用拉格朗日乘子可得

$$\phi_j = \frac{1}{m} \sum_{i=1}^m w_j^{(i)}.$$

⑤ 为什么得不到闭式解？

上面得到三个驻点方程：

$$\begin{aligned}\mu_j &= \frac{\sum_i w_j^{(i)} x^{(i)}}{\sum_i w_j^{(i)}}, \\ \Sigma_j &= \frac{\sum_i w_j^{(i)} (x^{(i)} - \mu_j)(x^{(i)} - \mu_j)^\top}{\sum_i w_j^{(i)}}, \\ \phi_j &= \frac{1}{m} \sum_i w_j^{(i)}.\end{aligned}$$

但关键问题是：

$$w_j^{(i)} = \frac{\phi_j N(x^{(i)}; \mu_j, \Sigma_j)}{\sum_{l=1}^k \phi_l N(x^{(i)}; \mu_l, \Sigma_l)}$$

本身依赖于所有待估参数 ϕ, μ, Σ 。

因此这些方程不是显式解，而是关于 ϕ, μ, Σ 的非线性隐式耦合方程组。不能像完全数据情形那样，直接一步算出 μ_j, Σ_j, ϕ_j 。

更准确地说：不是数学上证明了绝对不存在任何闭式表达式，而是直接对观测似然求导得到的驻点方程是隐式的，无法通过有限步初等运算显式解出。因此通常说 GMM 的 MLE 没有闭式解。这也说明为什么只能用迭代化算法来做

2). 为什么我们发现上面第一个问题中我们求偏导得到的驻点问题的方程其实就是在 EM 算法中 M-step 使用的更新公式？两者为什么长得一模一样？

解释：

① 首先长得一样但是含义不一样：

驻点方程中 $w_j^{(i)}$ 依赖于正在被优化的参数 ϕ, μ, Σ ，而这些参数里面又含 $w_j^{(i)}$ ，所以属于隐式耦合关系，无法解出；

而在 M-step 中 $w_j^{(i)}$ 先由旧参数算出，而后作为一个常量去更新 ϕ, μ, Σ 参数，这是两个步骤，所以可以用迭代方法不断更新收敛到最优值

② 事实上二者是统一的：

由于

$$\ell(\theta) = \sum_{i=1}^m \log P(x^{(i)}; \theta)$$

$$\begin{aligned}\frac{\partial \ell}{\partial \theta} &= \sum_{i=1}^m \frac{1}{P(x^{(i)}; \theta)} \cdot \frac{\partial}{\partial \theta} P(x^{(i)}; \theta) \\ &= \sum_{i=1}^m \frac{1}{P(x^{(i)}; \theta)} \sum_{z^{(i)}} \frac{\partial}{\partial \theta} P(x^{(i)}, z^{(i)}; \theta) \\ &= \sum_{i=1}^m \frac{1}{P(x^{(i)}; \theta)} \sum_{z^{(i)}} P(z^{(i)} | x^{(i)}; \theta) \cdot \frac{1}{P(z^{(i)} | x^{(i)}; \theta)} \cdot \frac{\partial}{\partial \theta} P(x^{(i)}, z^{(i)}; \theta) \\ &= \sum_{i=1}^m \sum_{z^{(i)}} P(z^{(i)} | x^{(i)}; \theta) \cdot \frac{1}{P(x^{(i)}, z^{(i)}; \theta)} \cdot \frac{\partial P(x^{(i)}, z^{(i)}; \theta)}{\partial \theta} \\ &= \sum_{i=1}^m \sum_{z^{(i)}} P(z^{(i)} | x^{(i)}; \theta) \cdot \frac{\partial}{\partial \theta} \log P(x^{(i)}, z^{(i)}; \theta).\end{aligned}$$

这个式子就是在说：

观测数据对数似然的梯度 = 以 $P(z^{(i)} | x^{(i)}; \theta)$ (也即是 $w_j^{(i)}$) 为权重，对完全数据对数似然的梯度做加权平均

这样我们就在“无闭式解”（直接对观测数据求偏导得到不可解的隐式耦合方程）与 EM Algo（用后验责任度 $w_j^{(i)}$ 对完全数据对数似然

做加权，然后求导）之间建立了联系，所以会发现最后的式子形式上居然是一样的！

§ Lec XV

Factor Analysis Model

outline :

- EM convergence
- Guassians properties
- Factor Analysis Model
- EM steps for factor analysis model

1. EM Convergence

Recall that in E-step, we compute (posterior)
(这个是我们最后得到结论之后的做法, 即将 Q_i 分布选取为后验概率) :

$$w_j^{(i)} = Q_i(z^{(i)} = j) = P(z^{(i)} = j \mid x^{(i)}; \phi, \mu, \Sigma)$$

Then for the M-step, we do this maximization :

$$\max_{\phi, \mu, \Sigma} \sum_i \sum_{z^{(i)}} Q_i(z^{(i)}) \log \frac{P(x^{(i)}, z^{(i)}; \phi, \mu, \Sigma)}{Q_i(z^{(i)})}$$

注意: $Q_i(z^{(i)})$ (即 $w^{(i)}$) 是后验概率; 而 $P(z^{(i)} = j) = \phi_j$ 是先验概率

代入 GMM 具体形式, 上式可再写为:

$$\begin{aligned} &\Rightarrow \max_{\phi, \mu, \Sigma} \sum_i \sum_{z^{(i)}} Q_i(z^{(i)}) \log \frac{P(x^{(i)} \mid z^{(i)}) P(z^{(i)})}{Q_i(z^{(i)})} \\ &\Rightarrow \max_{\phi, \mu, \Sigma} \sum_i \sum_{j=1}^k w_j^{(i)} \log \frac{\mathcal{N}(x^{(i)}; \mu_j, \Sigma_j) \cdot \phi_j}{w_j^{(i)}} \end{aligned}$$

- Another equivalent view of EM :

Define :

$$J(\theta, Q) = \sum_i \sum_{z^{(i)}} Q_i(z^{(i)}) \log \frac{P(x^{(i)}, z^{(i)}; \phi, \mu, \Sigma)}{Q_i(z^{(i)})}$$

We know that

$$\ell(\theta) \geq J(\theta, Q) \quad \forall \theta, Q$$

So this view of EM :

- ① E-step : maximize J with respect to Q
- ② M-step : maximize J with respect to θ

This method is also **coordinate ascent** !

It turns out that in E-step, we'll choose Q that makes $\ell(\theta) = J(\theta, Q)$ (最后结论: 最优的 Q_i 选择就是后验分布); then in M-step we'll choose θ that maximizes $J(\theta, Q)$.

所以这是与 EM 应用于 GMM 除了 Jensen's inequality 之外的另一种等价视角, 即将 EM 算法视为坐标上升法的优化框架。并且在这个视角下回答了为什么 EM 算法会收敛? 因为:

$$\ell(\theta^{t+1}) \geq J(\theta^{t+1}, Q^t) \geq J(\theta^t, Q^t) = \ell(\theta^t)$$

所以每一次参数优化从 $\theta^t \rightarrow \theta^{t+1}$ 都会将 $\ell(\theta)$ 上升, 而这个函数应当是有上界的 (这一点尚待说明严谨), 故单增有上界必然会收敛。

2. Problem with GMM

e.g.¹ :

We have a 2-dimensional feature dataset, and 100 data points. So we can fit mixture Gaussians.

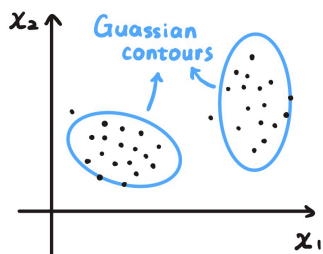


Figure 111: GMM fit e.g.

(examples > feature dimension)

Where we'd not use mixture of Gaussian but factor analysis is when $m \approx n$ or $m \ll n$ (n is the feature dimension, and m is the number of examples)

• What's wrong with single Gaussian model ?

1) In this case, we try to model single Gaussian distribution :

$$x \sim \mathcal{N}(\mu, \Sigma)$$

When we do MLE in this case :

$$\mu = \frac{1}{m} \sum_{i=1}^m x^{(i)}$$

$$\Sigma = \frac{1}{m} \sum_{i=1}^m (x^{(i)} - \mu)(x^{(i)} - \mu)^T$$

Σ 是 m 个中心化向量的外积之和, 这 m 个向量张成的空间维度最大为 m .

Thus if $m \leq n$, the covariance matrix would be singular (non-invertible) (rank $\leq m < n$).

If we take a look at the Gaussian density formula :

$$\frac{1}{(2\pi)^{\frac{n}{2}} |\Sigma|^{\frac{1}{2}}} \exp(-(\dots)\Sigma^{-1}(\dots)\dots)$$

$|\Sigma| = 0$ and Σ^{-1} doesn't exist.

For a simple example to illustrate this :

$$m = 2, n = 2$$

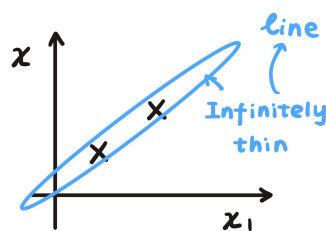


Figure 112: GMM fit problem

If we draw the Gaussian contour in this case, it'll be a infinitely thin line. (actually we've fit a line to that)

2) Another simple example to lead in :

Suppose we have a dataset of 30 persons, but we're gonna measure 100 psychological attributes and model $p(x)$. (now $x \in \mathbb{R}^{100}$)

既然直接拟合单个高斯分布不行 (因为协方差矩阵奇异的原因), 那么我们拟合 GMM (多个高斯混合分布) 显然更加不可行。那能否对协方差矩阵加一些限制让我们能够拟合出单个高斯分布呢 ?

• ▶ Option 1 : constrain Σ to be diagonal :

$$\Sigma = \begin{bmatrix} \sigma_1^2 & \dots & 0 \\ 0 & \sigma_2^2 & \dots \\ \vdots & \vdots & \sigma_n^2 \end{bmatrix}$$

Actually this corresponds to constraining the Gaussian to have axes align contours (高斯分布的等高线是轴对齐的椭圆) .

After MLE, we'll get:

$$\sigma_j^2 = \frac{1}{m} \sum_i \left(x_j^{(i)} - \mu_j \right)^2$$

Now Σ only has n entries.

However, the problem with it is that **this modeling constraint assumes that all of the features are uncorrelated, which is not reasonable.** (会导致模型欠拟合)

• **Option 2 : constrain $\Sigma = \sigma^2 I$**

Now it only has one parameter.

And then the MLE result is :

$$\sigma^2 = \frac{1}{m} \frac{1}{n} \sum_i \sum_j \left(x_j^{(i)} - \mu_j \right)^2$$

这个限制不仅假设各特征相互独立，而且各自的方差还相等，这无疑是更不合理的。但可以引出 factor analysis 的核心思路。

以上这个 Option 2 对应的模型假设是：

$$x \sim \mathcal{N}(\mu, \Sigma) \\ \Rightarrow x = \mu + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2 I)$$

这个假设意味着数据在所有维度上的变化都是等价的、独立的，即这个变化完全依赖于噪声，但这个假设太强，不合理。

一种自然的想法是：如果数据的变化并不是完全取决于噪声，而也同时来自于一种共同的隐藏因素？这个因素不是噪声，而是一个真实的、结构性的变化来源。

于是，在后续我们会把生成模型改成：

$$x = \mu + \underbrace{\Lambda z}_{\text{结构部分}} + \underbrace{\varepsilon}_{\text{噪声}}$$

其中：

- $z \in \mathbb{R}^d$ ：低维潜在因子，驱动数据的共同变化；
- Λ ：把低维因子映射到高维观测；
- $\varepsilon \sim N(0, \Psi)$ ：每个维度独立的噪声。

Insight：事实上这与矩阵的低秩分解紧密相关，也就是低维数据驱动得到的高维观测，这也对应着参数个数的减少。

FA 模型中低秩分解最直观的观察就在协方差矩阵的变化中：

$$\Sigma = \Lambda \Lambda^T + \Psi$$

其中 $\Lambda \Lambda^T \Rightarrow \mathbb{R}^{n \times d} \times \mathbb{R}^{d \times n}$ 就是一个低秩分解！

What factor analysis wants to do is to capture some correlation but doesn't run into the invertibility which naive Gaussian model does.

3. Factor Analysis Model

• **frame work**

$$P(x, z) = P(x | z)P(z)$$

z is latent

$$z \sim \mathcal{N}(0, I), z \in \mathbb{R}^d, (d < n)$$

$$x|z \sim \mathcal{N}(\mu + \Lambda z, \Psi)$$

模型骨架与 GMM 一样，是与生成式模型相同的框架。

以下则是 factor analysis model 建模的核心不同处：

• Assumptions

In factor analysis model, additionally, we have these parameters :

$$\mu \in \mathbb{R}^n, \Lambda \in \mathbb{R}(n \times d), \Psi \in \mathbb{R}^{n \times n}(\text{diagonal})$$

And we model x in another way :

$$x = \mu + \Lambda z + \varepsilon(\text{Guassian noise})$$

$$z \sim \mathcal{N}(0, I), \quad \varepsilon \sim \mathcal{N}(0, \Psi)$$

或者可以等价地写为：

$$x|z \sim \mathcal{N}(\mu + \Lambda z, \Psi)$$

Intuitions about this modeling :

- ① We believe there're d main factors that drive the value of x , so x is a linear function of $z(\in \mathbb{R}^d)$.
- ② Noises observed on all examples x are independent, so we set Ψ to be a diagonal matrix.

• 补充：两种模型参数个数比较

① 此时 factor analysis model 的参数个数：

$$O(\mu + \Lambda + \Psi) = O(n + nd + n) \\ \Rightarrow O(nd)$$

② 回顾一下 GMM 的参数个数：

$\phi_j \in \mathbb{R}(j = 1, \dots, k)$ 但概率值满足归一化;

$$\mu_j \in \mathbb{R}^n(j = 1, \dots, k);$$

$$\Sigma_j \in \mathbb{R}(n \times n)(j = 1, \dots, k) \text{ 且对称}$$

所以 GMM 总参数个数为

$$O(\phi + \mu + \Sigma) = O\left((k-1) + nk + k \cdot \frac{n(n+1)}{2}\right) \\ = O(kn^2)$$

直观比较之后显然 factor analysis model 总的自由参数个数比 GMM 少很多

• An Illustration Example

用一个简单的典例展示 factor analysis model 中“高维数据其实落在低维子空间附近”这一核心思想。

Suppose :

$$x \in \mathbb{R}^2(n = 2), z \in \mathbb{R}^1(d = 1), \text{ and } m = 7$$

Since here $z \sim N(0, 1)$, we first sample 7 z 's out on the axis :

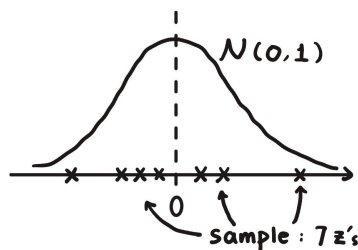


Figure 113: z 's sampled from a Gaussian

Say :

$$\Lambda = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \mu = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Now the linear function is :

$$x = \mu + \Lambda z \\ = \begin{bmatrix} 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 2 \\ 1 \end{bmatrix} z$$

So without the Gaussian noise ε , x 's are gonna be fit on a line :

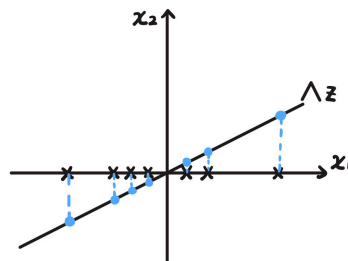


Figure 114: x 's without Gaussian noises

Now say :

$$\Psi = \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix}$$

which means that x_2 has higher noise variance than x_1 .

Then, we add Gaussian noise ε :

$$x = \mu + \Lambda z + \varepsilon$$

It corresponds to adding a Gaussian contour on each x .

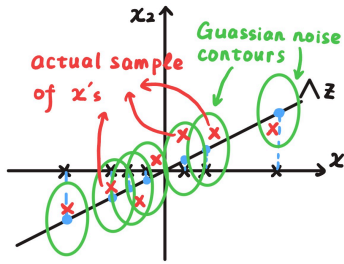


Figure 115: actual sampled x 's

Then we sample from these Gaussians and get the the red crosses as a typical example drawn from this model.

So here we let $n = 2, d = 1$, which means we have two-dimensional data, but most of the data lies on a one-dimensional subspace (with little noises).

When we have such high-dimensional data in a such small dataset, we can't fit very complex models through it. So it may be reasonable to fit them in a subspace.

• Properties of Multivariate Gaussians

$$x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \in \mathbb{R}^{r+s}, x_1 \in \mathbb{R}^r, x_2 \in \mathbb{R}^s$$

If $x \sim \mathcal{N}(\mu, \Sigma)$,

$$\mu = \begin{bmatrix} \mu_1 \\ \mu_2 \end{bmatrix} \in \mathbb{R}^{r+s}, \mu_1 \in \mathbb{R}^r, \mu_2 \in \mathbb{R}^s$$

$$\Sigma = \begin{bmatrix} \Sigma_{11} & \Sigma_{12} \\ \Sigma_{21} & \Sigma_{22} \end{bmatrix}, \Sigma_{11} \in \mathbb{R}^{r \times r}, \Sigma_{22} \in \mathbb{R}^{s \times s}$$

• ▶ 1) Marginal Probability

$P(x_1) = ?$

$$P(x) = P(x_1, x_2)$$

$$\begin{aligned} \Rightarrow P(x_1) &= \int_{x_2} P(x_1, x_2) dx_2 \\ &= \int_{x_2} P(x) dx_2 \end{aligned}$$

Then we plug in the Gaussian probability density of $P(x)$:

$$= \int_{x_2} \frac{1}{(2\pi)^{\frac{n}{2}} |\Sigma|^{\frac{1}{2}}} \exp \left(-\frac{1}{2} \begin{bmatrix} x_1 - \mu_1 \\ x_2 - \mu_2 \end{bmatrix}^T \begin{bmatrix} \Sigma_{11} & \Sigma_{12} \\ \Sigma_{21} & \Sigma_{22} \end{bmatrix}^{-1} \begin{bmatrix} x_1 - \mu_1 \\ x_2 - \mu_2 \end{bmatrix} \right) dx_2$$

Then we do the integration with respect to x_2 .
We'll find :

$$x_1 \sim \mathcal{N}(\mu_1, \Sigma_{11})$$

• ▶ 2) Conditional Probability

$P(x_1 | x_2) = ?$

$$\begin{aligned} P(x_1 | x_2) &= \frac{P(x_1, x_2)}{P(x_2)} \\ &= \frac{P(x)}{P(x_2)} \end{aligned}$$

It turns out that it also follows a Gaussian distribution.

$$\Rightarrow x_1|x_2 \sim \mathcal{N}(\mu_{x_1|x_2}, \Sigma_{x_1|x_2})$$

$$\text{with : } \mu_{x_1|x_2} = \mu_1 + \Sigma_{12}\Sigma_{22}^{-1}(x_2 - \mu_2)$$

$$\Sigma_{x_1|x_2} = \Sigma_{11} - \Sigma_{12}\Sigma_{22}^{-1}\Sigma_{21}$$

where :

$$\Sigma_{zz} = \text{Cov}(z) = I;$$

$$\begin{aligned}\Sigma_{zx} &= E[(z - E[z])(x - E[x])^T] = E[z(\Lambda z + \varepsilon)^T] \\ &= E[zz^T]\Lambda^T + E[z\varepsilon^T] = \Lambda^T\end{aligned}$$

$$\Sigma_{xz} = \Lambda$$

$$\begin{aligned}\Sigma_{xx} &= E[(x - E[x])(x - E[x])^T] \\ &= E[(\Lambda z + \varepsilon)(\Lambda z + \varepsilon)^T] \\ &= E[\Lambda zz^T \Lambda^T + \varepsilon z^T \Lambda^T + \Lambda z \varepsilon^T + \varepsilon \varepsilon^T] \\ &= \Lambda \Lambda^T + \Psi\end{aligned}$$

So we get :

$$\Sigma_{x,z} = \begin{bmatrix} I & \Lambda^T \\ \Lambda & \Lambda \Lambda^T + \Psi \end{bmatrix}$$

$$\begin{bmatrix} z \\ x \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} \vec{0} \\ \mu \end{bmatrix}, \begin{bmatrix} I & \Lambda^T \\ \Lambda & \Lambda \Lambda^T + \Psi \end{bmatrix}\right)$$

Now we can write down $P(x)$ (it's this Gaussian density) and try to take the derivative of the log likelihood with respect to each parameter and will find out that there's no closed form solutions.

• Derivation of EM algorithm for Factor Analysis Model

• ► Preparations :

Apply the properties of multivariate Gaussian to factor analysis model.

Derive the joint distribution : $P(x, z)$

$$\begin{bmatrix} z \\ x \end{bmatrix} \sim \mathcal{N}(\mu_{x,z}, \Sigma_{x,z})$$

$$z \sim \mathcal{N}(0, I) \in \mathbb{R}^d, \quad \varepsilon \sim \mathcal{N}(0, \Psi) \in \mathbb{R}^n$$

$$x = \mu + \Lambda z + \varepsilon \in \mathbb{R}^n$$

We can derive that :

$$E[x] = \mu \in \mathbb{R}^n \Rightarrow \mu_{x,z} = \begin{bmatrix} \vec{0} \\ \mu \end{bmatrix} \in \mathbb{R}^{d+n}$$

Similarly, we can get :

$$\Sigma_{x,z} = \begin{bmatrix} \Sigma_{zz} & \Sigma_{zx} \\ \Sigma_{xz} & \Sigma_{xx} \end{bmatrix}$$

So in order to fit the parameters, we'll resort to EM algorithm.

• ► E-step

We'd compute :

$$Q_i(z^{(i)}) = P(z^{(i)} | x^{(i)}; \theta)$$

When we're fitting a mixture of Gaussian distributions, $z^{(i)}$ was discrete. But in this case, $z^{(i)}$ is a continuous density.

It turns out that we can use the conditional probability density to represent this continuous density.

$$z^{(i)}|x^{(i)} \sim \mathcal{N}(\mu_{z^{(i)}|x^{(i)}}, \Sigma_{z^{(i)}|x^{(i)}})$$

Again we apply the previously derived properties :

$$\begin{aligned}\mu_{z^{(i)}|x^{(i)}} &= \mu_z + \Sigma_{zx} \Sigma_{xx}^{-1} (x^{(i)} - \mu_x) \\ &= \vec{0} + \Lambda^T (\Lambda \Lambda^T + \Psi)^{-1} (x^{(i)} - \mu) \\ \Sigma_{z^{(i)}|x^{(i)}} &= \Sigma_{zz} - \Sigma_{zx} \Sigma_{xx}^{-1} \Sigma_{xz} \\ &= I - \Lambda^T (\Lambda \Lambda^T + \Psi)^{-1} \Lambda\end{aligned}$$

So in E-step we'd compute these, and store them as variables and represent Q_i as a Gaussian density. (E-step 完成后, 我们得到每个样本 $x^{(i)}$ 对应的后验高斯分布)

• • M-step

$$\begin{aligned}\theta &= \arg \max_{\theta} \sum_i \int_{z^{(i)}} Q_i(z^{(i)}) \log \frac{P(x^{(i)}, z^{(i)})}{Q_i(z^{(i)})} dz^{(i)} \\ \Rightarrow & \sum_i E_{z^{(i)} \sim Q_i} \left[\log \frac{P(x^{(i)}, z^{(i)})}{Q_i(z^{(i)})} \right]\end{aligned}$$

For the nominator and denominator we'd plug in the Gaussian density. (usually when there's a log ahead, we'd plug in the Gaussian density)

After the plugging, this'll be a quadratic expression, and we'll take derivatives to find out the parameters.

与 GMM 类似, factor analysis model 所应用的 EM 算法同样也是一个坐标上升法。固定 $J(\theta, Q)$ 之后先在 E-step 固定 θ 优化 Q , 再在 M-step 固定 Q 来选取最优化的参数 θ

对比项	GMM	因子分析
模型框架	生成式模型 骨架	生成式模型 骨架
隐变量 z	discrete : $z \in \{1, \dots, k\}$	continuous : $z \in \mathbb{R}^d$
z 含义	簇标签	低维潜在因子
先验 $P(z)$	$P(z = j) = \varphi_j$	$z \sim N(0, I)$
条件分布 $P(x z)$	$x z = j \sim N(\mu_j, \Sigma_j)$	$x z \sim N(\mu + \Lambda z, \Psi)$
边缘分布 $P(x)$	混合高斯 : $\sum_{j=1}^k \phi_j N(\mu_j, \Sigma_j)$	单个高斯 : $N(\mu, \Lambda \Lambda^T + \Psi)$
EM 步骤	① 算后验概率 $w_j^{(i)}$ ② 进行加权 MLE	① 算出后验高斯分布 ② 再对高斯期望求导
建模目标	聚类、密度估计	降维、高维小样本密度估计

§ Lec XVI

Principal Component Analysis & Independent Component Analysis (partial)

1. PCA

Recall that factor analysis model tries to model $P(x)$ which is in a high-dimensional space. However, PCA is not a probabilistic and it doesn't model $P(x)$, but it still allows you to figure out whether the data is in a low-dimensional space.

• Example

We have an unlabeled dataset : $\{x^{(1)}, \dots, x^{(m)}\} \in \mathbb{R}^n$. We want to reduce the dimension from n to k , $k \ll n$.

比如说我们有一个 inch - centimeter 的二维数据集，由于长度单位之间是可以转换的，因此这个数据集实际上应当处于一个低维子空间。

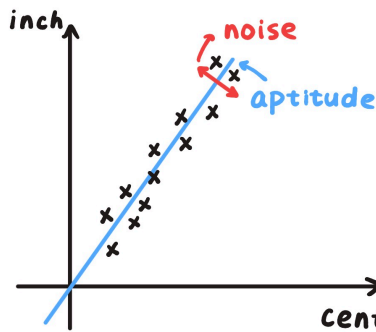


Figure 116: PCA e.g.¹ dataset

PCA 算法要做的事情就是找到图中那个倾斜的维度方向，那也是数据变化的主轴。并且在与之正交的维度上只会存在一些噪声。当我们把数据投影到这根轴上，二维数据就会变为一维数据。

• Pre-processing

Before PCA, we're gonna process our data :

① Zero out mean :

$$\mu = \frac{1}{m} \sum_{i=1}^m x^{(i)}$$
$$x^{(i)} \leftarrow x^{(i)} - \mu$$

② Standardize variance to 1 :

$$\sigma_j^2 = \frac{1}{m} \sum_{i=1}^m (x_j^{(i)})^2$$
$$x_j^{(i)} \leftarrow \frac{x_j^{(i)}}{\sigma_j}$$

- PCA illustration & intuition

If we've got this dataset after pre-processing :

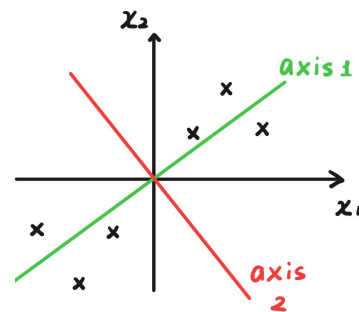


Figure 117: pre-processed dataset

It looks like that the green line is a pretty good variation axis (the one-dimensional subspace) for the given dataset. Intuitively, the red line is a bad subspace.

Why ?

Reason 1 : 如果我们把所有数据点分别（正交）投影到绿线和红线上面，可以看到所有数据点到绿线的距离的平方和应当是很小的。这可能是定义 PCA 方法的一种方式。

Reason 2 : 如果我们仅在绿/红两线上看投影点之间的位置关系, 我们可以看到绿线上投影点之间相隔较远, 而红线上的投影点之间挤成一团。所以我们或许可以这样定义 PCA : 找到一个子空间, 数据点投影到子空间上后尽可能保持分散, 这样能保留更多的数据变异性。

事实上, 上面两种直觉在数学上是等价的。

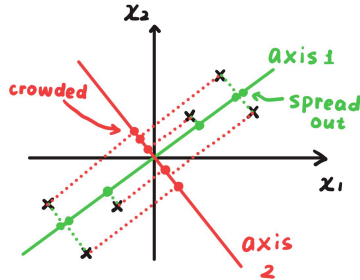


Figure 118: PCA intuition

If $\|u\| = 1$, then the projection of $x^{(i)}$ onto u is :

$$\text{Prj} = u^T x^{(i)}$$

In PCA, we want to choose u to maximize :

$$\begin{aligned} & \max_{u: \|u\|=1} \frac{1}{m} \sum_{i=1}^m (x^{(i)T} u)^2 \\ &= \max_{u: \|u\|=1} \frac{1}{m} \sum_{i=1}^m u^T x^{(i)} x^{(i)T} u \\ &= \max_{u: \|u\|=1} u^T \left(\frac{1}{m} \sum_{i=1}^m x^{(i)} x^{(i)T} \right) u \end{aligned}$$

Note that the covariance matrix for x is Σ_{xx}

$$\max_{u: \|u\|=1} u^T \Sigma_{xx} u$$

If we do the maximization, it turns out that u is **principal eigenvector** of Σ_{xx} . (注意, 由于 Σ_{xx} 是对称的, 所以其有一组正交基, 所以我们找到的所有 u_i 其实可以构成一组正交基, 并张成一个低维子空间。

(proof by Lagrange multipliers) :

$$\max_u u^T \Sigma_{xx} u, \quad \text{s.t. } u^T u = 1$$

$$\mathcal{L}(u, \lambda) = u^T \Sigma_{xx} u + \lambda(u^T u - 1)$$

$$\frac{\partial \mathcal{L}}{\partial u} = (\Sigma_{xx}^T + \Sigma_{xx})u + 2\lambda u$$

$$\frac{\partial \mathcal{L}}{\partial \lambda} = u^T u - 1$$

令两个偏导数均为 0 且代入约束条件, 且由于 $\Sigma_{xx} = \Sigma_{xx}^T$, 于是得:

$$\Sigma_{xx} u = -\lambda u$$

即 u 就是 Σ_{xx} 的特征向量。

综上, 如果我们要用一个一维子空间来近似数据, 那么选取的子空间方向就应当是这个对应的特征向量的方向。

• General case

If wish to project data to k -dimensional space, then we set $u_1, u_2, \dots, u_k$ to be the top- k eigenvectors of Σ_{xx}

Now let's say we have a very high dimensional dataset :

$$x^{(i)} \in \mathbb{R}^n \quad (\text{say } n = 1000)$$

And we want to reduce the dimension from n to k ($k \ll n$), so we need to find :

$$u_1, u_2, \dots, u_k \in \mathbb{R}^n \quad (\text{say } k = 10)$$

Then we'll have a new representation :

$$x^{(i)} \rightarrow \begin{bmatrix} u_1^T x^{(i)} \\ u_2^T x^{(i)} \\ \vdots \\ u_k^T x^{(i)} \end{bmatrix} = y^{(i)} \in \mathbb{R}^k$$

So now instead of using a thousand number to represent the training example, we're using 10 numbers to represent each training data point.
(注意：现在的 10 维空间是由 10 个特征向量所构建，因此原向量 x 在每一维上的坐标就是在这一维对应特征向量上的投影大小)

If we want to go back from $y^{(i)}$ to $x^{(i)}$, it turns out that :

$$x^{(i)} \approx y_1^{(i)} u_1 + y_2^{(i)} u_2 + \dots + y_k^{(i)} u_k \in \mathbb{R}^n$$

(此处仍有一个待解决问题：为什么在数据预处理要减去均值并除以标准差??? 几何直观上的意义???)

• Applications

① Visualization :

Project from n -D to 1-D or 2-D.

② Compression for ML efficiency :

$$x^{(i)} \in \mathbb{R}^{10000} \xrightarrow{\text{compress}} y^{(i)} \in \mathbb{R}^{1000}$$

Running the learning algorithm on a lower dimensional dataset can be more efficient.

③ Reduce overfitting (questionable)

Maybe regularization is more suitable in this case.

④ Outlier detection (matching)

It was once used in gace detection, where people use PCA to project the pixel vector to a low dimension and measure the Euclidean distance between two pictures. However, we tend to not use it anymore.

• ▶ Rule of Thumb

Before using PCA, consider just using the original data

If do use PCA, then in test set we use the same set of eigenvectors that we've found in train set.

Note that the direction of each eigenvector is really unstable, and if we study the meaning of directions that'll usually be hallucinations. But the resulting subspace is usually stable.

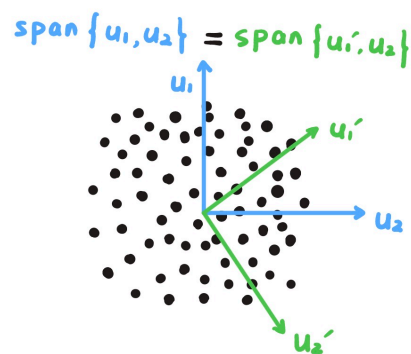


Figure 119: unstable eigenvectors and stable span

• Comparison

Now we can draw a comparison table and sort the properties of the 4 unsupervised learning algorithms that we've learned:

Model type	model $P(x)$ e.g. anomaly detection	non-probabilistic e.g. compression, visualization
Subspaces	factor analysis model	PCA
Clusters	Mixture of Gaussians	K-means

- **How to choose k ?** (dimension of the subspace)

If we choose :

$$\frac{\lambda_1 + \dots + \lambda_k}{\lambda_1 + \dots + \lambda_k + \dots + \lambda_n} = c\%$$

Then we'd say : **“retained $c\%$ of vairiance of the data”**. (usually we take $c\% = 0.90 / 0.95 / 0.98$...)

We observe :

$$x^{(i)} = As^{(i)}, \quad x^{(i)} \in \mathbb{R}^n$$

(n microphones)

Each microphone captures a linear combination of the different voices from the speakers.

$$x_j^{(i)} = \text{recording of microphone } j \text{ at time } t,$$

$$j = 1, \dots, n$$

Because :

$$x_j^{(i)} = \sum_k A_{jk} s_k^{(i)}$$

Our goal is to find

$$W = A^{-1}$$

so that

$$s^{(i)} = Wx^{(i)}$$

2. Independent Component Analysis (ICA)

- **Cocktail party example to lead in**

e.g. It can be used for seperating different independent voices from a mixed sample.

Suppose we have this original source :

$$s \in \mathbb{R}^n \quad (n \text{ speakers})$$

$$s_j^{(i)} = \text{signal from speaker } j \text{ at time } i$$

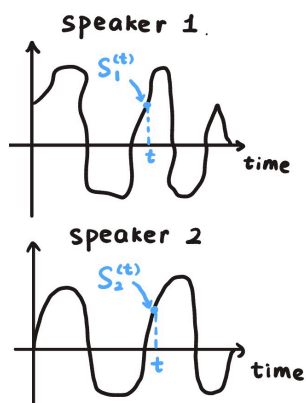


Figure 120: source signals

Note that the two samples are timestamps consistent.

- **ICA goal**

The whole algorithm is given the data x to find the matrix W . (注意：标准 ICA 中我们要求 s 与 x 的维数相同，因为这样线性变换矩阵是一个方阵，才有可能矩阵可逆!)

Notation :

$$W = \begin{bmatrix} \text{---} w_1^T \text{---} \\ \text{---} w_2^T \text{---} \\ \vdots \\ \text{---} w_n^T \text{---} \end{bmatrix}$$

$$\text{So } s_j^{(i)} = w_j^T x^{(i)}.$$

- **A visualization of ICA :**

Let's say the data resources are (2 speakers in total) : (each timestamp a speaker is emitting a random number between $(-1, 1)$)

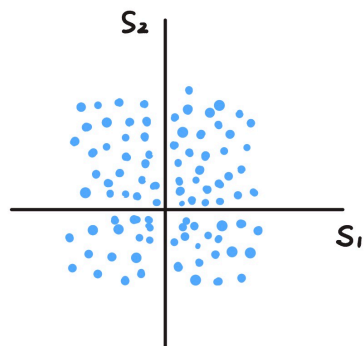


Figure 121: pre-processed source

Recall how does s change to x :

$$x^{(i)} = As^{(i)}$$

which means take each s through a linear transformation and end up to be x :

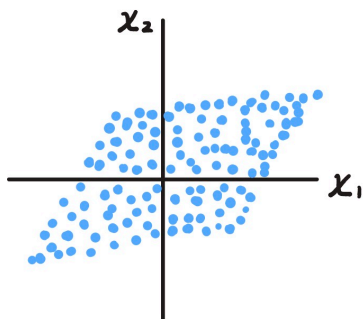


Figure 122: observed data

So in practice we are observing x 's and try to find a linear transformation that change x back to s .

- **Two ambiguities :**

① **Axis Ambiguity** : When we do the linear transformation from x back to s , we don't know the order of s_1 , s_2 , so we just come to the resources with a random order.

② **Sign Ambiguity** : In linear transformation, flipping happens. So when we come back to the resources, we may end up with $\pm s_1$, $\pm s_2$. But in practice that doesn't really matter.

§ Lec XVII

Independent Component Analysis

outline :

- CDFs (cumulative distribution functions)
- ICA model

• Two Ambiguities

In the last part of Lec16, we know that the goal of ICA algorithm is to find the unmixing matrix W that :

$$s^{(i)} = Wx^{(i)}$$

承接上个 Lec 最后的 ICA 部分，我们说到 ICA 具有 permutation ambiguity + scaling ambiguity. (对观测使用 unmixing matrix W 之后我们得到的 s 的各个维度无法区分顺序；对观测作用 W 之后得到的 s 各维度可能是被放缩之后的。

(additional proof of the two ambiguities) :

① Permutation ambiguity :

This corresponds to permutation matrixs :
e.g.

$$P = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

In ICA, if we've verify an unmixing matrix W that is already got :

$$s = Wx$$

Then if we let :

$$W' = PW$$

那么 W' 同样是一个该数据下的合理解混矩阵。也就是说，我们可以对 W 进行任意的行

变换得到的仍是满足条件的解混矩阵。

② Scaling ambiguity :

在 ICA 中我们可以对已经得到的满足条件的 W 的某一行进行放缩，得到的矩阵仍然是一个满足条件的解混矩阵。

这样，我们最后分离出来的独立成分（独立声源）只是会被相应地缩放相应倍数（非零），依然满足 ICA 目标。

• Pre-processing

说明了 ICA 的两个 ambiguity，现在来讲解的 ICA 前续步骤：

① 均值归零 (zero out means) :

$$x \rightarrow x - E[x]$$

由于建模为：

$$s = Wx$$

因此我们预计将分离出来的独立源也均值归零：

$$E[s] = \vec{0}$$

② 白化 (standardize variance to 1)

$$x \leftarrow Vx, \quad s.t. \text{Cov}(Vx) = I$$

这一步将观测数据方差归一化。由于均值归零后的 x 的协方差矩阵为 $\Sigma_x = E[xx^T]$ ($\in \mathbb{R}^n$, symmetric)

先将协方差矩阵 Σ_x 做特征值分解：

$$\Sigma_x = U\Lambda U^T$$

(Λ is diagonal, U is orthogonal)

我们可以直接使用一个矩阵 V 将 x 方差变换为 I 。使用：

$$V = \Lambda^{-\frac{1}{2}} U^T$$

(验证：

$$\begin{aligned} \text{Cov}(Vx) &= V\Sigma_x V^T \\ &= \Lambda^{-\frac{1}{2}} U^T \Sigma_x U \Lambda^{-\frac{1}{2}T} \\ &= \Lambda^{-\frac{1}{2}} U^T U \Lambda U^T U \Lambda^{-\frac{1}{2}} \\ &= \Lambda^{-\frac{1}{2}} (U^T U) \Lambda (U^T U) \Lambda^{-\frac{1}{2}} \\ &= \Lambda^{-\frac{1}{2}} I \Lambda I \Lambda^{-\frac{1}{2}} \\ &= I \end{aligned}$$

)

所以对 x 使用 V 之后 Vx 协方差矩阵变为 I 。

原始观测数据 x 经过上述 ①均值归零 ②白化两个先续处理步骤之后均值变为 $\vec{0}$ ，方差变为单位矩阵 I 。

• ICA Restrictions : non-Gaussian output

下面开始 ICA 模型的正式处理部分。

首先回顾 ICA 目标：寻找最优的解混矩阵 W 以从观测数据还原至原始独立声源：

$$s = Wx$$

假如我们对于最后的参数 W 有一个估计取值，怎样知道这个估计是否好呢？这依赖于我们对于原始独立数据的先验假设：

$$s_1, s_2, \dots, s_n \text{ are independent}$$

这是我们最为核心的判别法则，即对于估计的参数 W 对应的输出 s ，我们将根据其各维度的独立性判断当前的参数 W 选取的好坏。

然而当前判据并不足够，我们需要考虑下面一种特殊情况：

假设选取一个 W 以后，我们得到的 s 各个维度服从一个多元正态分布 $s \sim \mathcal{N}(\vec{0}, I)$ ，（由于前面对 x 均值归零，故 $E[s] = \vec{0}$ ；由于 scaling ambiguity 所以我们不妨可以假设 s 满足的多元正态分布协方差矩阵为 I ），此时我们能否判断当前选取的参数 W 好坏？

首先，因为 $s \sim \mathcal{N}(\vec{0}, I)$ ，故此时输出的 s 各维依然相互独立，满足我们最开始给出的判别方法。那么此时选取的 W 一定最好吗？（抛开 permutation + scaling 变体）

但是，我们发现此时满足条件的参数矩阵 W 具有无数多个！（即便已经抛开 permutation + scaling 所产生的变体）

事实上，我们可以找到这些所有的参数矩阵 W' 与现在我们已选择的这个参数矩阵 W 之间的关系：

$$W' = RW$$

R 是任意的正交矩阵

证明：

若已有：

$$s = Wx \sim \mathcal{N}(\vec{0}, I)$$

那么更改参数矩阵后：

$$\begin{aligned} s' = W'x &= RWx \sim \mathcal{N}(R\vec{0}, RIR^T) \\ &\sim \mathcal{N}(\vec{0}, I) \end{aligned}$$

注意：正交矩阵作用到一个 $\mathcal{N}(0, I)$ 的变量上面后结果仍然是 $\mathcal{N}(0, I)$ ！

补充：我们可以借助正交矩阵线性变换的几何直观来理解：

正交矩阵只包含 翻转 + 旋转 这两种操作及其复合，而以二维正态分布为例，其等高线为圆，翻转、旋转后等高线不变，故输出的分布不变！

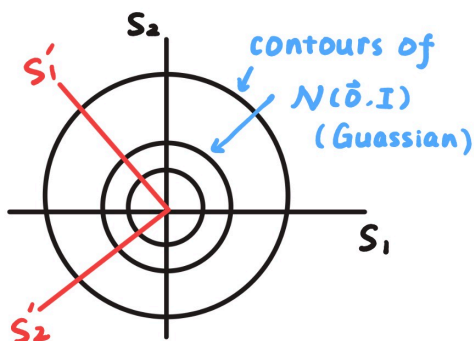


Figure 123: standard Gaussian contour

因此，如果输出 s 满足多元高斯分布，那么我们可以在已有参数 W 基础上选择无数多种参数矩阵 W ，最后的输出依然一样。因此此时参数的选取有无数种！无法找到正确的那一类参数 W ！

(补充知识：实际上我们除了独立性还有其他损失函数可以评判，但在此处高斯分布依然会导致损失评判恒为 0 而使得无法找到最优的参数 W)

为避免上述情况产生，我们对于最后的输出 s 加一条评判标准，即输出的 s 不应满足多维高斯分布！

In both ICA and PCA, we'll first zero out means. Assume here we get s_1, s_2 for later processing.

• CDF + Probability Relation

Given that, we'll develop ICA under the circumstance that the data is non-Gaussian. We first have to figure out what's the density of s .

补充知识：An equivalent way to represent the probability of the density of continuous random variables is via CDF :

$$F(s) = P(S \leq s)$$

(S is a random variable and s is a constant)

e.g : If S is a Gaussian random variable, then the CDF is the function that increases from 0 to 1.

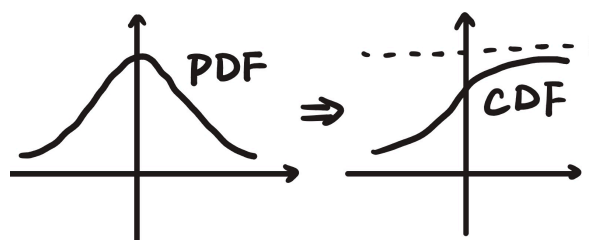


Figure 124: CDF to PDF

(这其实就是概率论中密度函数与分布函数关系)

In ICA, instead of specifying a PDF for the source data, we're gonna choose a specified CDF that is not a Gaussian density CDF.

Choose a CDF $F(s)$, then the density of s is $P_s(s)$;

$$\begin{aligned} x &= As = W^{-1}s \\ s &= Wx \end{aligned}$$

A tempting method is that maybe we can compute the density of x in this way ?

$$P_x(x) = P_s(Wx) \quad (\text{because } Wx = s)$$

However, it turns out that this is incorrect as it works only for discrete probability distributions and is incorrect for continuous probability densities.

(Why we have to find the density of x ?
Because in training, we can only observe x for finding the maximum likelihood estimate parameters. So we need to know the density of x to choose the optimal parameter W .)

• **A simple illustration example**

Say :

$$P_s(s) = 1\{0 \leq s \leq 1\} \quad (s \sim U(0, 1))$$

Let's say :

$$\left(\begin{array}{l} x = 2s \\ \text{(here } A = 2, W = \frac{1}{2}, n = 1 \text{ (one-dimensional))} \end{array} \right)$$

Thus :

$$x \sim U(0, 2)$$

$$P_x(x) = \frac{1}{2} \cdot 1\{0 \leq x \leq 2\}$$

Then we draw densities for s and x :

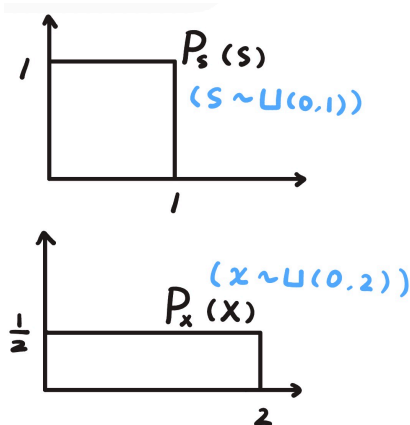


Figure 125: density from s to x

More generally, the correct formula for the equality relation between $P_x(x)$ and $P_s(s)$ is :

$$P_x(x) = P_s(Wx) \cdot |\det(W)|$$

$\det(W)$ is the determinant of W

And that ensures that the distribution still normalizes to 1.

Note that in the simple example above, $P_x(x)$ is :

$$P_x(x) = \frac{1}{2} \cdot 1\{0 \leq x \leq 2\}$$

$$= 1\left\{0 \leq \frac{1}{2}x \leq 1\right\}$$

and $\frac{1}{2}x = s$, so we're back to $P_s(s)$

• **Choose $P_s(s) = ?$**

We need to choose a non-Gaussian for $P_s(s)$.

We can choose the sigmoid function for $F(s)$:

$$F(s) = P(S \leq s) = \frac{1}{1 + e^{-s}}$$

Then we take the derivative of it and get the corresponding PDF, it turns out that it has fatter tail than Gaussian. And this captures human voices or other natural phenomena better than a Gaussian density as there are a larger number of extreme outliers.

Also, double-sided exponential (Laplacian distribution) turns out to be good as a choice for $P_s(s)$.

- MLE

现在我们将进行 MLE 估计最优参数：(MLE 实质就是给定观测数据 x ，将数据概率值参数化为待估计的参数 W ，随后最大化这个概率来估计参数 W 的最优值)

Because : (independence of the sources is the key assumption of ICA)

$$P_s(s) = \prod_{i=1}^n P_s(s_i)$$

(s is the vector of total voice sources)

(n speakers are independent)

Therefore we have :

$$\begin{aligned} P_x(x) &= P_s(Wx) |\det(W)| \\ &= \prod_{j=1}^n P_s(W_j^T x) |\det(W)| \end{aligned}$$

So the ICA model is as above. Here we choose $P_s()$ to be the CDF of sigmoid, and we express $P_x(x)$ as a function of the parameter W . (此时我们代入的模型假设是： P_s 是 sigmoid 对应的 PDF 概率分布)

Now the MLE :

$$\ell(w) = \sum_{i=1}^m \log \left[\left(\prod_j P_s(W_j^T x^{(i)}) \right) |\det(W)| \right]$$

Then we use stochastic gradient ascent .

$$\nabla_w \ell(w) = \begin{bmatrix} 1 - 2g(W_1^T x) \\ \dots \\ 1 - 2g(W_n^T x) \end{bmatrix} x^{(i)T} + (W^T)^{-1}$$

$g()$ is the sigmoid function.

- Recap of the whole algorithm

① We have a whole training set of

$$x^{(1)}, \dots, x^{(m)}$$

where each of the training examples is a microphone recording.(and we can split a certain timestamp out)

(此时进行 均值归零 + 白化 预处理步骤)

② We'll initialize the unmixing W randomly, and run stochastic gradient ascent. When it converges, we'll have W and use it to recover the sources :

$$s = Wx$$

§ Lec XVIII - XXI

RL

这些章节内容主要为强化学习方面，而关于 RL 部分可以参见 西湖大学《强化学习的数学基础》(赵世钰) 这门课程，本网站也有对其完整的笔记 (关于 RL 非常严谨详细)。于是 CS229 RL 部分将不再收录。